



INVESTIGACIÓN

**Modelos de Machine Learning para la Predicción de Trayectorias de Transporte Urbano
BRT: Sistema de Transmilenio, Bogotá D.C**

Autor

JAIME ANDRÉS CUCHIA RUIZ

Director/Tutor

PhD. ROBERTO FERRO ESCOBAR

Universidad Distrital Francisco José De Caldas

Maestría en Ciencias de la Información y las Comunicaciones

Énfasis en Ingeniería de Software

Bogotá, Colombia

mayo de 2026

Contenido

Resumen	7
Palabras Clave.....	8
Introducción	9
1. Problema de investigación	12
1.1 Planteamiento del problema	12
1.1.1 Diagnóstico de la situación actual	12
1.1.2 Pronóstico	13
1.2 Formulación del problema	14
1.3 Sistematización del problema	14
2. Objetivos	15
2.1 Objetivo general	15
2.2 Objetivos específicos.....	15
3. PLANTEAMIENTO de la hipótesis.....	16
4. CAPITULO i. marco de referencia	17
4.1 Sistema de Transporte Tapido por Buses (<i>bus rapid transit, BRT</i>).	17
4.1.1 <i>Contexto Global y Latinoamericano</i>	18
4.1.2 <i>Transmilenio: Implementación y Alcance Operativo</i>	19
4.1.3 <i>Ciudades Inteligentes (CI) y Transporte Urbano</i>	21
4.2 Marco Conceptual.....	23
4.2.1 <i>Calidad de los Datos en Sistemas de Transporte</i>	23
4.2.2 <i>Inteligencia Artificial (IA)</i>	24
4.2.3 <i>Machine Learning (ML)</i>	25

4.2.4	Relevancia del ML para la predicción de trayectorias espaciotemporales.	26
4.3	Trayectorias Espaciotemporales.....	27
4.4	Algoritmos Relevantes para La Predicción de Trayectorias	30
4.5	Validación y Evaluación del Modelo de ML.....	33
4.5.1	Métricas de evaluación para predicción de trayectorias	34
4.5.2	Métodos de Validación	36
4.6	Investigaciones Previas	39
5.	CAPITULO II: EXPLORACIÓN Y PREPARACIÓN de datos espaciotemporales	42
5.1	Identificación de Datos Espaciotemporales.....	42
5.1.1	Delimitación espacial a la Troncal 80	43
5.2	Fuentes de Datos	45
5.3	Recolección de Datos.....	47
5.4	Filtrado por troncal calle 80.....	51
5.5	Limpieza de Datos	53
5.6	Estandarización e Integración.....	57
5.6.1	Resultado de la integración.....	60
5.7	Generación de Variables Derivadas	61
5.8	Análisis Exploratorio de Datos	62
6.	Capitulo III : Selección, Implementación y Evaluación de Modelos de ML	66
6.1	Bosques Aleatorios (Random Forest)	67
6.2	Regresión Mediante Potenciación de Gradiente (xgboost)	71
6.3	Máquinas de Vectores de Soporte para Regresión (SVR)	74
6.4	Red Neuronal Artificial (ann/mlp regressor)	78
6.5	Long Short Term Memory (LSTM)	82
7	CAPÍTULO IV: RESULTADOS Y DISCUSIÓN	89

7.1	Comparativa general de los cinco modelos	89
7.2	Análisis espacial de los resultados	90
7.3	Análisis temporal de los resultados.....	91
7.4	Importancia de variables.....	92
7.5	Discusión.....	92
8	CONCLUSIONES	94
9.	PRINCIPALES APORTES DE LA INVESTIGACIÓN.....	96
10.	recomendación Y TRABAJOS FUTUROS.....	97
	Referencias.....	98
	Anexos.....	102
	Anexo A — Estructura de Directorios y Configuración Inicial	102
	Anexo B — Extracción de Archivos Excel con openpyxl.....	102
	Anexo C — Limpieza mediante Rango Intercuartílico (IQR) por Estación	103
	Anexo D — Protocolo de Entrenamiento y Evaluación (Random Forest)	103
	Anexo E — Arquitectura y Entrenamiento del Modelo LSTM	104

Índice de Figuras

Figura 1.	Limitaciones Estructurales Identificadas.....	20
Figura 2	Mapa de Troncales Sistema Transmilenio Bogotá D.C – Troncal calle 80....	45
Figura 3.	Archivos Organizados	48
Figura 4.	Diagrama de flujo proceso limpieza de datos	53
Figura 5.	Proceso de Integración	57
Figura 6.	Diagrama entidad-relación del dataset espaciotemporal integrado.....	60
Figura 7.	Distribución de ingresos al sistema por tipo de día. Troncal 80.	62
Figura 8.	Perfil horario de ingresos promedio por tipo de día y por estación.	63
Figura 9.	Mapa de calor de ingresos promedio por estación y franja horaria.....	64
Figura 10.	Serie temporal de ingresos y salidas diarias del corredor. Troncal 80	65
Figura 11.	Ratio ingreso/salida y flujo neto promedio por estación (días laborales.....	65

Figura 12. Random Forest: Dispersión real vs. predicho y distribución de residuos . .	69
Figura 13. Random Forest : Importancia de features.	70
Figura 14. Random Forest — Métricas por estación y perfil horario predicho vs. real.	70
Figura 15. XGBoost — Dispersión real vs. predicho y distribución de residuos	72
Figura 16. XGBoost — Importancia de features (gain normalizado).	73
Figura 17. Comparativo XGBoost vs. Random Forest por franja horaria y R^2 global. .	74
Figura 18. Dispersión real vs. predicho y distribución de residuos (conjunto prueba).	76
Figura 19. Comparativo SVR vs. XGBoost vs. Random Forest por franja y R^2 global.	77
Figura 20. Red Neuronal : Arquitectura MLP y curva de convergencia	80
Figura 21. Red Neuronal — Dispersión real vs. predicho y distribución de residuos. .	80
Figura 22. Comparativo final — 4 modelos de Machine Learning. Troncal 80.	81
Figura 23. LSTM: Dispersión real vs. predicho y distribución de residuos.	85
Figura 24. LSTM — Curvas de entrenamiento y validación. Troncal 80 (2025-2026)..	86
Figura 25. LSTM : Métricas de evaluación por estación. Conjunto de prueba	87
Figura 26. Desempeño del LSTM Comparación final	88

Índice de Tablas

Tabla 1 Desafíos críticos en la implementación de modelos de Machine Learning	22
Tabla 2 Subcomponentes de la calidad de datos en sistemas de transporte masivo BRT	24
Tabla 3 Paradigmas del aprendizaje automático y su aplicación en el sistema TransMilenio	26
Tabla 4 Capacidades analíticas del análisis de trayectorias espaciotemporales en sistemas	29
Tabla 5. Comparativa de métodos de validación para modelos de regresión	38
Tabla 6. Indicadores operacionales del Sistema TransMilenio	43
Tabla 7. Características técnicas de cada fuente	47
Tabla 8. Resultado del filtrado por corredor Troncal 80 (TZ005)	52
Tabla 9. Resultados del proceso de limpieza de datos por fuente).	57
Tabla 10. Variables del dataset final.	61
Tabla 11. Métricas de evaluación.	68

Tabla 12. Métricas de evaluación — XGBoost Regressor.....	73
Tabla 13. Métricas de evaluación — SVR.	76
Tabla 14. Métricas de evaluación — Red Neuronal ANN/MLP	81
Tabla 15. Arquitectura del modelo LSTM	83
Tabla 16. Ranking final — cinco modelos de Machine Learning.....	89
Tabla 17. MAE por estación y modelo.....	90
Tabla 18. MAE por franja horaria y modelo).	91

RESUMEN

El sistema TransMilenio de Bogotá D.C. enfrenta limitaciones estructurales crecientes asociadas a la alta incertidumbre en la demanda de pasajeros, la congestión en estaciones críticas y la dificultad para anticiparse a las condiciones cambiantes de la operación. Ante este escenario, la presente investigación tuvo como objetivo analizar e implementar modelos de *Machine Learning* para la predicción de trayectorias espaciotemporales en el corredor Troncal 80, evaluando su potencial como herramienta de apoyo a la toma de decisiones operativas en el sistema.

Se implementaron y compararon cinco modelos de aprendizaje automático supervisado: *Random Forest*, *XGBoost*, *Support Vector Regression* con *kernel RBF*, Red Neuronal Artificial densa y Long Short-Term Memory. Los modelos fueron construidos sobre un dataset espaciotemporal de 493.662 registros elaborado a partir de fuentes de datos de acceso público para el período enero 2025 a marzo 2026, y evaluados bajo un protocolo experimental común basado en división temporal estricta, normalización sin fuga de información y búsqueda automatizada de hiperparámetros.

Los resultados demostraron que los modelos de ML permiten predecir la demanda de pasajeros por estación e intervalo de 15 minutos con niveles de precisión superiores a los reportados en la literatura internacional comparable para sistemas BRT. La Red Neuronal Artificial alcanzó el mejor desempeño con un R^2 de 0,9999996 y un MAPE de 0,089%, mientras que XGBoost y Random Forest obtuvieron MAPEs inferiores al 5%, superando los referentes documentados en estudios similares desarrollados en sistemas BRT de Asia y Europa.

Los resultados confirman la hipótesis de la investigación: la implementación de modelos de ML aplicados al análisis de trayectorias espaciotemporales mejora la capacidad predictiva sobre el comportamiento operativo del sistema TransMilenio y contribuye a la generación de herramientas analíticas más precisas para su gestión eficiente, a partir exclusivamente de datos históricos de acceso público.

PALABRAS CLAVE

Inteligencia Artificial (IA), Machine Learning (ML), trayectorias (Variables Espaciotemporales), Ciudades Inteligentes (CI), Random Forest, XGBoost, Support Vector Regression predicción, Transporte Rápido por Buses (BRT).

INTRODUCCIÓN

La movilidad urbana es uno de los fenómenos más complejos y determinantes en la calidad de vida de las ciudades contemporáneas. En Bogotá D.C., esta complejidad alcanza una dimensión singular: más de cuatro millones de personas dependen diariamente del sistema TransMilenio para desplazarse por una ciudad que no deja de crecer, que no deja de cambiar y que exige cada vez más de un sistema de transporte que fue concebido para una realidad urbana muy distinta a la actual. Con más de dos décadas de operación continua, TransMilenio ha sido un referente mundial en la implementación del modelo Bus Rapid Transit, pero esa trayectoria también ha dejado al descubierto sus límites: buses llenos, estaciones desbordadas, tiempos de espera impredecibles y una creciente sensación ciudadana de que el sistema no logra adaptarse a la demanda que enfrenta cotidianamente (Hidalgo et al., 2024).

Frente a este escenario, la pregunta que motivó esta investigación no es nueva pero sí resulta urgente: ¿es posible anticiparse al comportamiento del sistema TransMilenio con suficiente precisión como para que esa anticipación se convierta en una herramienta real de gestión operativa? Durante décadas, la respuesta a preguntas similares en el campo del transporte masivo dependió de modelos estadísticos tradicionales, series de tiempo clásicas y métodos de predicción que, si bien aportaron valor en su momento, presentan limitaciones conocidas ante la complejidad, la no linealidad y la alta dimensionalidad de los datos que generan los sistemas BRT modernos (Zhang et al., 2021). En los últimos años, sin embargo, el avance del aprendizaje automático ha abierto un horizonte diferente: algoritmos capaces de aprender patrones complejos a partir de grandes volúmenes de datos históricos, sin necesidad de imponer supuestos distribucionales sobre el comportamiento del sistema, y con una capacidad de generalización que los métodos tradicionales difícilmente pueden igualar (Goodfellow et al., 2016).

Esta investigación se sitúa precisamente en ese horizonte. Su propósito es analizar e implementar modelos de ML para la predicción de trayectorias espaciotemporales en el corredor Troncal 80 del sistema TransMilenio de Bogotá D.C., entendiendo las trayectorias como el flujo de pasajeros por estación e intervalo de tiempo, y evaluando el potencial de

estos modelos como herramienta de apoyo a la toma de decisiones operativas. La selección del corredor Troncal 80 como caso de estudio no fue arbitraria: con 13 estaciones activas, 7,38 kilómetros de extensión y operación continua desde el año 2000, este corredor ofrece el volumen histórico de datos, la diversidad funcional de estaciones y la representatividad operativa necesarios para construir y evaluar modelos de machine learning con rigor metodológico.

Una de las decisiones más importantes de la investigación fue trabajar exclusivamente con datos de acceso público. Los portales institucionales de TransMilenio S.A. y de la Alcaldía de Bogotá publican de forma rutinaria los registros de validación electrónica, los datos de salidas de pasajeros por estación, los atributos de infraestructura y los datos de oferta programada en formato GTFS. Esta información, que el sistema ya genera y publica cotidianamente sin costo adicional para la institución, fue el único insumo de datos utilizado en toda la investigación. Esta decisión tiene implicaciones metodológicas y prácticas importantes: si los modelos construidos exclusivamente con datos públicos logran alcanzar niveles de precisión comparables o superiores a los reportados en la literatura internacional, entonces la barrera de acceso a la información deja de ser un obstáculo para la implementación de sistemas predictivos en el argumento institucional colombiano.

Para responder al objetivo planteado, se implementaron y compararon cinco modelos de ML que representan paradigmas metodológicos distintos: Random Forest y XGBoost como representantes del paradigma de ensamble de árboles de decisión, Support Vector Regression con kernel RBF como representante del paradigma de máquinas de vectores de soporte, una Red Neuronal Artificial densa como representante del paradigma de aprendizaje profundo con features de contexto explícitas, y una red Long Short-Term Memory como representante del paradigma de aprendizaje secuencial recurrente. La evaluación comparativa de estos cinco enfoques sobre el mismo dataset y bajo el mismo protocolo experimental permite extraer conclusiones más sólidas y transferibles que las que ofrecería la evaluación aislada de un único algoritmo.

El documento se organiza en cuatro capítulos. El Capítulo I presenta el marco de referencia teórico y conceptual que sustenta la investigación, abordando el sistema TransMilenio y el modelo BRT, los fundamentos del ML aplicado al transporte, y los métodos de validación empleados. El Capítulo II documenta el proceso completo de

exploración y preparación del dataset espaciotemporal, desde la identificación y recolección de las fuentes hasta la generación de variables derivadas y el análisis exploratorio de los datos. El Capítulo III describe el diseño, implementación y evaluación individual de los cinco modelos, con sus respectivos análisis de desempeño global, por estación y por franja horaria. El Capítulo IV presenta los resultados comparativos entre los cinco modelos, la discusión de los hallazgos en el contexto de la literatura especializada, el análisis del alcance y las limitaciones de los modelos como herramienta de apoyo a la toma de decisiones, y las conclusiones del trabajo

1. PROBLEMA DE INVESTIGACIÓN

1.1 PLANTEAMIENTO DEL PROBLEMA

1.1.1 Diagnóstico de la situación actual

Desde la puesta en marcha del sistema de TM en la ciudad de Bogotá D.C en el año 2000, se ha convertido en la columna vertebral de la movilidad de la ciudad, siendo la principal alternativa de transporte movilizand o a más de 4,5 millones de usuarios diarios y en especial para los usuarios que residen en zonas periféricas y dependen de una conectividad eficiente con el resto de la ciudad. Hay que tener claro que el sistema ya tiene más de dos décadas de operación, y el paso del tiempo, la no modernización del sistema, la extensión de la ciudad y el aumento poblacional ha generado al sistema diversas limitaciones operativas y estructurales, lo cual ha afectado negativamente al desempeño y la percepción del servicio entre los usuarios (Hidalgo et al., 2024).

Uno de los principales puntos de insatisfacción por parte de los usuarios, se ve identificado a la alta incertidumbre en los tiempos de llegada de los buses, particularmente en las horas pico es cuando la demanda supera la capacidad planificada, sumándose la escasez de rutas disponibles para algunos sectores, las frecuentes demoras por congestión en estaciones, la aglomeración de usuarios y la imposibilidad del sistema en anticiparse a diversos problemas externos. Esto da como resultado una experiencia de viaje caracterizado como negativo dado a los altos niveles de insatisfacción por parte de los usuarios.

Pese a lo anterior, TM sigue siendo indispensable para la ciudad ya que no se cuenta con un sistema de transporte alternativo como el Metro. Esta situación refuerza la exigencia por parte de la ciudadanía en repensar el modelo operativo desde un enfoque que permita adaptarse de forma dinámica a las condiciones cambiantes de la demanda, optimizar la planificación de rutas y reducir la incertidumbre operativa que afecta diariamente a millones de usuarios

1.1.2 Pronóstico

De mantenerse las condiciones actuales descritas anteriormente, los niveles de insatisfacción ciudadana y la congestión operativa en TransMilenio continuaran en aumento. Dado a que el crecimiento urbano acelerado, el incremento proyectado en la demanda de transporte y la persistente falta de integración con futuros sistemas de transporte complementarios, como el metro (actualmente en fase de construcción), podrían agravar aún más las fallas del sistema de TM.

El sistema debe contar con herramientas predictivas que permitan anticiparse a variaciones tanto en el comportamiento del sistema como en la planificación de la prestación del servicio y si no se da esto, sin sistemas que apoyen decisiones dinámicas, como el ajuste de frecuencias, la redistribución de flotas o la detección temprana de cuellos de botella en las vías, se incrementara el riesgo de colapsos operativos.

La falta de respuestas efectivas por parte de la administración de la ciudad, y del sistema de transporte TM en la integración efectiva de una modernización, se han reflejado al uso del transporte informal como las bicicletas, patinetas eléctricas o hasta del vehículo particular, esto lejos de ser una solución ha traído consigo varios factores determinantes y con bastantes consecuencias negativas para la movilidad urbana y para la ciudad, más desorden en la movilidad, cultura ciudadana negativa, aumento en la contaminación del aire de la ciudad.

1.1.3 Control al pronóstico

Frente a las limitaciones operativas identificadas en el sistema TransMilenio y al riesgo en el desempeño del sistema, resulta pertinente considerar estrategias que fortalezcan la capacidad del sistema para anticipar y adaptarse a condiciones dinámicas., los modelos de ML representan una alternativa metodológica con potencial para aportar valor en la predicción de trayectorias y tiempos de llegada, aspectos críticos en la operación de sistemas de transporte urbano de gran escala.

La combinación de datos históricos y en tiempo real y métodos de análisis de trayectorias permitirá identificar y capturar patrones de operación sofisticados y derivar mejores conocimientos sobre el comportamiento del sistema actual. Contribuyendo a la

mejora significativa, agilizando ajustes operativos más mayo grado de eficiencia ante fluctuaciones en la demanda, incidentes en vía o condiciones externas imprevistas.

Este enfoque técnico se alinearía con las tendencias internacionales en movilidad urbana y así abrir oportunidades en la incorporación de herramientas analíticas avanzadas para la planificación y operación del sistema TM mejorando sustancialmente la calidad del servicio y en la capacidad institucional para responder a los retos de una ciudad como Bogotá D.C

1.2 FORMULACIÓN DEL PROBLEMA

¿En qué medida los modelos de *machine learning* aplicados al análisis de trayectorias, pueden contribuir al desarrollo de esquemas predictivos orientados a la evaluación operativa de sistemas de transporte urbano masivo, utilizando como caso de estudio los datos del sistema TransMilenio de Bogotá?

1.3 SISTEMATIZACIÓN DEL PROBLEMA

- ¿Qué tipo de datos espacio-temporales se requieren para desarrollar modelos de predicción de trayectorias en sistemas de transporte urbano como TransMilenio?
- ¿Qué técnicas de Machine Learning resultan más adecuadas para modelar trayectorias a partir de datos operativos históricos?
- ¿Cómo influye la calidad y estructura de los datos en el desempeño predictivo de los modelos de Machine Learning aplicados a trayectorias vehiculares?
- ¿Qué métricas permiten evaluar la precisión y robustez de los modelos de predicción de trayectorias en un sistema masivo como TransMilenio?
- ¿De qué manera los resultados obtenidos mediante técnicas de Machine Learning pueden ser utilizados para optimizar decisiones operativas en contextos urbanos de alta complejidad?

2. OBJETIVOS

2.1 OBJETIVO GENERAL

Analizar e implementar modelos de *Machine Learning* para la predicción de trayectorias en sistemas de transporte urbano, a partir del estudio de datos del sistema TransMilenio de Bogotá D.C., con el fin de evaluar su potencial para mejorar la capacidad de anticipación operativa en contextos urbanos complejos.

2.2 OBJETIVOS ESPECÍFICOS

- Identificar los tipos de datos espaciotemporales necesarios para la construcción de modelos de predicción de trayectorias en sistemas de transporte urbano masivo.
- Seleccionar y aplicar técnicas de Machine Learning adecuadas para modelar trayectorias vehiculares a partir de datos históricos
- Evaluar el desempeño de los modelos de Machine Learning implementados mediante métricas cuantitativas, considerando su precisión, robustez y capacidad de generalización en la predicción de trayectorias espaciotemporales
- Analizar los resultados obtenidos para determinar el alcance y las limitaciones del uso de modelos de Machine Learning como apoyo a la toma de decisiones operativas en el sistema TransMilenio.

3. PLANTEAMIENTO DE LA HIPÓTESIS

La implementación de modelos de Machine Learning aplicados al análisis de trayectorias (espaciotemporales) permite mejorar la capacidad predictiva sobre el comportamiento operativo de los vehículos en sistemas de transporte urbano masivo BRT, lo cual contribuye a la generación de herramientas analíticas más precisas para una gestión más eficiente de TM.

4. CAPITULO I. MARCO DE REFERENCIA

4.1 Sistema de Transporte Tapido por Buses (*bus rapid transit, BRT*).

Los Sistemas de Transporte Rápido por Buses (*Bus Rapid Transit, BRT*) son un modelo de transporte público masivo basado en buses, los cuales mantienen su operación por medio de carriles exclusivos (right of way), integrado a la infraestructura y el espacio público urbano, con la particularidad de su nivel de adaptación a las necesidades propias de cada ciudad. Por medio de la definición de (Cervero, 2013), *un BRT se precisa a “un sistema de transporte público basado en buses de alta calidad, que proporciona una movilidad urbana rápida, cómoda además de rentable mediante infraestructura segregada, operaciones frecuentes y rápidas, y altos estándares de mercadeo y servicio al cliente”*. En esencia, este sistema busca, combinar la capacidad y velocidad de los sistemas ferroviarios y metros con la flexibilidad, los bajos costos y fácil mantenimiento característico de los autobuses convencionales (The International Association of Public Transport, 2025).

Para diferenciar un sistema BRT de un servicio de autobuses convencionales, por medio de la literatura especializada se ha podido identificar cinco (5) elementos esenciales que definen a un sistema como BRT, los cuales contribuyen de manera decisiva a la reducción de demoras causadas por la congestión, los conflictos con otros vehículos y las interrupciones derivadas al ascenso y descenso de pasajeros, incrementando así la eficiencia y la confiabilidad, al tiempo que se reducen los costos de operación (Institute for Transportation & Development Policy, 2024). Los cinco elementos esenciales del BRT son:

1. Carril exclusivo y derecho de vía: Constituye el elementó primario bajo la infraestructura del sistema, el cual consiste en la habilitación de carriles físicamente segregados del tráfico general y destinado exclusivamente para la circulación de las flotas BRT, garantizando que los vehículos del sistema eviten la congestión vehicular mixta y así poder alcanzar velocidades operativas superiores a las del transporte convencional (ITDP, 2024).
2. Alineación del carril para autobús: La ubicación del carril dentro de un sistema BRT está definido bajo alineación central del eje de las calzadas, pensado

estratégicamente, dado que los bordes de vía son zonas de alta conflictividad (maniobras de giro del tráfico mixto, frenados, estacionamientos, cargue y descargue de mercancías o pasajeros de otros medios de transporte) y evitando que los buses queden expuestos a esta serie de interrupciones y así poder garantizar factores de velocidad, regularidad y confiabilidad en el sistema. (ITDP, 2024).

3. Validación de pago previo al abordaje: El recaudo del pago se realiza por medio de torniquetes y validadores de tarjetas inteligentes por radiofrecuencia (RFID) esquema conocido como Pago fuera del bus (*Off board fare collection*) (ITDP, 2024).
4. Tratamiento de intersecciones.
5. Abordaje a nivel de plataforma: Las plataformas de las estaciones se construyen al mismo nivel del acceso a los buses (*level boarding*) evitando escalones de acceso entre el andén y los vehículos así garantizando la accesibilidad para personas con movilidad reducida, usuarios en silla de ruedas u otras condiciones, mujeres embarazadas, coches de bebé, etc. (ITDP, 2024).

4.1.1 Contexto Global y Latinoamericano

La primera referencia documentada al concepto se deriva de 1937, cuando el Plan de Chicago propuso transformar las líneas de tránsito ferroviario en corredores de buses expresos sobre autopistas urbanas, con distribución en superficie en las áreas centrales de la ciudad (Harrington et al., 1937, citado en Levinson et al., 2003). No obstante, aquella propuesta no pasó de ser una idea adelantada a su tiempo dado a la infraestructura del momento. Es por ello que fueron necesarias casi cuatro décadas para que el modelo tomara forma concreta y demostrara, en la práctica, que era viable.

En 1974, se efectuó la primera implementación real del modelo BRT en Curitiba, Brasil, con la puesta en marcha de la *Red Integrada de Transporte* (RIT), marcando técnicamente el uso de carriles exclusivos, marcando así un antes y un después en el diseño de estos sistemas: ubicar los carriles de bus en la posición central de la calzada, en lugar de en los bordes de la vía como se acostumbraba hasta entonces (Ingvardson & Nielsen, 2017). Esa decisión aparentemente sencilla, transformó radicalmente la eficiencia del servicio y sentó las bases del modelo como una alternativa inteligente: alta capacidad, bajo costo relativo y

tiempos de implementación considerablemente más cortos que los del metro o el tren ligero (UITP, 2019).

Para la década del 2000, más de veinticinco ciudades en el mundo ya operaban sistemas BRT (Hidalgo, 2013), pero fue un hito puntual el que realmente catapultó al modelo al escenario global: la apertura de TransMilenio en Bogotá, Colombia, a finales del año 2000. El sistema bogotano no se instauro como otro corredor BRT más ; ha sido una apuesta urbana de gran escala demostrando que este tipo de infraestructura puede transformar profundamente la movilidad de una metrópolis latinoamericana.

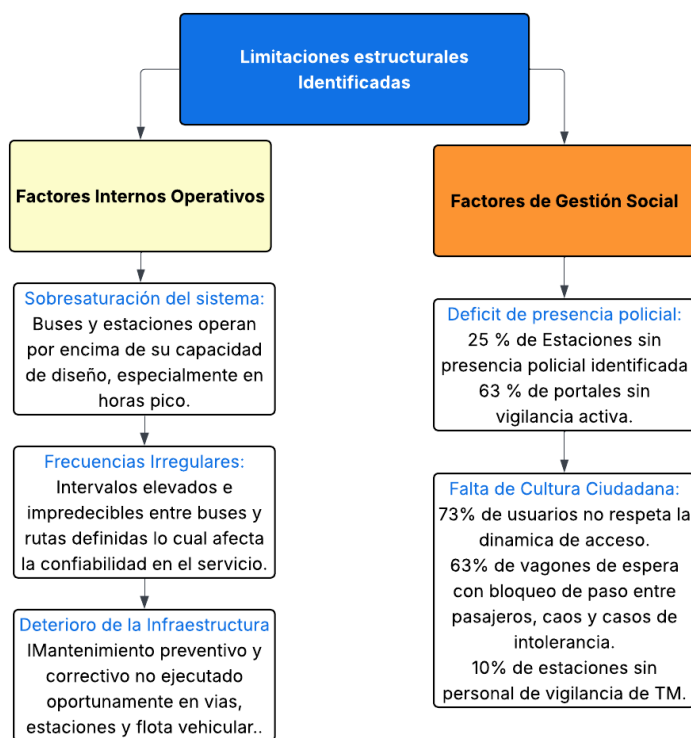
Los sistemas BRT operan hoy en 187 ciudades alrededor del mundo y transportan 34,5 millones de pasajeros cada día. De ese total, el 58 % corresponde a usuarios de América Latina y Asia, mientras que Europa concentra el 25 % restante. America Latina, concentra un tercio de los kilómetros de ruta BRT a nivel global, 63 % de los pasajeros totales del sistema en el mundo (Global BRT Data, 2025). Más de sesenta ciudades de la región cuentan actualmente con sistemas en operación, lo que convierte a Latinoamérica no solo en la cuna histórica del modelo, sino en su principal escenario de desarrollo y consolidación

4.1.2 Transmilenio: Implementación y Alcance Operativo

En Bogotá D.C., el sistema TransMilenio (TM), fue adoptado desde inicios del año 2000, y ha sido un ejemplo entre las implementaciones más representativas del modelo BRT a nivel mundial como se señaló anteriormente. Específicamente TM opera más de 114.4 kilómetros de vías troncales, 143 estaciones y 9 portales, movilizand o aproximadamente a 4.03 millones de pasajeros diarios mediante una flota compuesta por más de 4.000 vehículos, entre articulados, biarticulados y alimentadores (TransMilenio, 2024).

A pesar de ser un referente y con más dos décadas de operación, el sistema TM actualmente enfrenta importantes limitaciones estructurales, entre factores alternos e internos que comprometen su desempeño. Entre las fallas más relevantes se identifican las siguientes segmentadas por Factores internos Operativos y Factores de Gestión Social. (Figura 1):

Figura 1. Limitaciones Estructurales Identificadas



Nota. Fuente Personería de Bogotá. D.C; secretaria Distrital de movilidad – Elaboración Propia

Este tipo de problemáticas se presentan de carácter recurrente y afectan directamente la eficacia del sistema bajo un contexto urbano dinámico y de alta demanda como el de Bogotá D.C (Distrital De Movilidad De Bogotá). Las fallas inherentes al sistema TM dan trascendencia a la importancia de replantear los enfoques y estrategias tradicionales, apostando al desarrollo y análisis del sistema a través de soluciones tecnológicas integrales e innovadoras , y a la vez fomentar significativamente la participación ciudadana. Los ciudadanos, como ente fundamental, son precisamente los que deben exigir que los procesos y servicios se presten, con garantías de equidad y calidad (De Bem Machado et al., 2023)

4.1.3 Ciudades Inteligentes (CI) y Transporte Urbano

El concepto de CI, ha evolucionado considerablemente desde sus primeras formulaciones, en la que se entendía su uso particularmente al objetivo estructural por medio de sensores, conectividad y automatización, derivándose a un enfoque o visión más integral, en el que el uso de tecnologías en respuesta al mejoramiento de la calidad de vida de los ciudadanos, optimización de los recursos públicos y fortalecimiento en la capacidad de respuesta de las instituciones ante desafíos propios de la vida actual (Trencher, 2019; Singh & Edla, 2022).

Dado a lo anterior, el transporte público representa uno de los mayores retos de las CI porque por este medio, los habitantes de una ciudad se relacionan directamente con el espacio urbano bajo factores tales como, el tiempo en que tardan en desplazarse de un lugar a otro, que zonas son más accesibles y una percepción general en términos de calidad de servicio prestado por el Estado. Un sistema de transporte deficiente no solo representa un problema de movilidad; es una limitante estructural para el avance y desarrollo económico, equidad social y sostenibilidad ambiental de una ciudad (Maestre, 2015). No obstante, la construcción de una CI en materia de Transporte no depende únicamente de la disponibilidad tecnológica, requiere también una gobernanza sólida que sea capaz de articular todos los actores involucrados (operadores, autoridades, usuarios), bajo marcos normativos que orienten el uso responsable de datos a largo plazo, garantizando que las soluciones implementadas sean sostenibles, inclusivas y adaptables (Trencher, 2019), dado a que sin estos elementos la tecnología por sí sola corre el riesgo de aumentar aun mas las brechas en las desigualdades existentes en temas de movilidad.

La articulación entre tecnología, gestión urbana y participación ciudadana determina el escenario donde los modelos de Aprendizaje Automático (Machine Learning – ML) se posicionan y encuentran un foco de aplicación. Los sistemas de transporte masivo a nivel general producen grandes volúmenes de datos, que representan información cruda con poca representación práctica, la capacidad de los modelos de ML tiene como finalidad de transformar la heterogeneidad de datos producidos por el sistema en conocimiento accionable para la gestión propia del sistema (Zhao, 2017 ; Zhong, 2022 ;Lee & Yoon, 2022).

Sin embargo, es importante reconocer que su implementación no está exenta de desafíos y se han identificado enfrenta al menos 3 desafíos críticos (ver tabla 1):

Tabla 1 Desafíos críticos en la implementación de modelos de Machine Learning en sistemas BRT

Desafío	Problema central	Implicación práctica
1. Calidad de los datos	Los errores en la recolección sensores defectuosos, registros incompletos o inconsistentes afectan directamente la precisión de los modelos entrenados.	Exigen ajustes continuos que demandan recursos técnicos y humanos especializados, incrementando los costos operativos del sistema.
2. Adaptabilidad en entornos dinámicos	Los sistemas de transporte urbano cambian constantemente en términos de demanda, rutas y condiciones de operación, lo que afecta la vigencia de los modelos.	Los modelos deben ser reentrenados de forma periódica para mantener su validez y capacidad predictiva ante nuevas condiciones operativas.
3. Interpretabilidad de los modelos	Los algoritmos más avanzados operan frecuentemente como cajas negras (black box), sin ofrecer explicación visible sobre sus decisiones.	Genera desconfianza institucional al momento de integrar estos modelos en procesos formales de toma de decisiones (UITP, 2025).

Nota. Elaboración propia con base en UITP (2025).

Estas limitaciones, anteriormente descritas, no deslegitiman la utilidad y el potencial del *Machine Learning*; al contrario, enfatizan la importancia de contar con datos de alta calidad y un análisis particular de los modelos a implementar (desde su aplicabilidad, complejidad y eficiencia). Bogotá exige respuestas ágiles y la fiabilidad tanto de los datos como de los modelos predictivos. Estas demandas requieren sistemas robustos, dado que solo con conjuntos de datos bien estructurados y actualizados (tiempos de parada, número de usuarios por estación, número de buses por ruta, velocidades, eventos externos, etc) se podrán generar resultados en la predicción de trayectorias con un alto margen de precisión.

Esta investigación no solo permitirá anticiparse a comportamientos del sistema como tal, sino también aportará a la producción de conocimiento accionable por parte de la entidad que gestiona el servicio. Su implementación exitosa dependerá de comprender la brecha entre la teoría algorítmica propia de los modelos de ML y la comprensión de los factores particulares del sistema TM, ya que es un sistema que genera más de 4.033.991 viajes diariamente y es una de las ciudades con altas fallas y complejidad en América Latina (Bogota.gov.co, 2024).

4.2 Marco Conceptual

La presente investigación se fundamenta en un conjunto de conceptos relacionados entre sí, constituyen la base teórica sobre la cual se sustenta el análisis y la predicción de trayectorias en el sistema TransMilenio. El recorrido conceptual propuesto parte de la naturaleza de los datos producidos por los sistemas de transporte, avanza hacia los fundamentos del aprendizaje automático y concluye con los criterios metodológicos para la evaluación de los modelos predictivos.

4.2.1 *Calidad de los Datos en Sistemas de Transporte*

La calidad de los datos constituye la condición estructural de todo proceso analítico. De acuerdo con (Batini & Scannapieco, 2016), hace referencia a la precisión, consistencia, integridad y relevancia de la información utilizada durante el análisis. Bajo el contexto particular del aprendizaje automático, la calidad ejerce un papel fundamental, directamente en las fases de entrenamiento de un modelo particular, como en su aplicación a nuevos datos o escenarios reales.

Los sistemas de transporte masivo, como Transmilenio, producen en todo momento grandes volúmenes de datos provenientes de fuentes heterogéneas (no provienen de una única fuente, no tienen el mismo formato, frecuencia o naturaleza): trazas de la flota, accesos y salidas de usuarios, eventos viales y datos operativos del sistema (rutas, valores de pasajes, estructura del sistema, mapas etc.). Para que esta información pueda ser fuente de alimentación adecuada en un algoritmo de ML, es indispensable aplicar un preprocesamiento que mantenga unos parámetros rigurosos en limpieza de datos, imputación de valores faltantes, filtrado y alineación temporal de las fuentes de datos. La ausencia de estos pasos conduciría inevitablemente a la obtención de resultados apartados de la realidad con una alta probabilidad de error y baja capacidad de generalización en la predicción (Soomro, 2019).

Batini & Scannapieco (2016) descomponen este concepto en múltiples dimensiones que, analizadas en conjunto, permiten establecer si un conjunto de datos es apto para alimentar un modelo predictivo confiable, (ver tabla 2):

Tabla 2 Subcomponentes de la calidad de datos en sistemas de transporte masivo BRT

Subcomponente	Definición	Aplicación en TransMilenio
Precisión	Grado en que los datos representan fielmente los valores reales del sistema observado.	Un error en la posición GPS de un bus puede generar errores acumulativos en la predicción de su trayectoria y tiempo de llegada a la siguiente estación.
Integridad	Disponibilidad de todos los valores necesarios para describir adecuadamente una situación o evento operativo.	La ausencia de registros sobre tiempos de parada o desviaciones de ruta limita la capacidad del modelo para identificar patrones operativos relevantes en TransMilenio.
Consistencia	Ausencia de declaraciones contradictorias o conflictivas entre distintas fuentes de datos.	Una discrepancia entre el tiempo registrado por el GPS del vehículo y el tiempo del sistema de control central puede generar conflictos en la interpretación algorítmica del dato.
Calidad temporal	Correspondencia de los datos con el estado actual del sistema, garantizando que la información refleje las condiciones vigentes y no situaciones pasadas.	En un entorno dinámico como TransMilenio, datos desactualizados sobre frecuencias o demanda por estación invalidan las predicciones basadas en ellos.
Validez y representatividad	Los datos deben reflejar el comportamiento real del sistema tanto en condiciones normales como en situaciones anómalas, garantizando la capacidad de generalización del modelo.	Un modelo entrenado únicamente con datos de días laborables típicos fallará al predecir trayectorias durante eventos masivos, cierres viales o jornadas atípicas de demanda.

Nota. Elaboración propia con base en Batini & Scannapieco (2016) y Soomro (2019).

4.2.2 *Inteligencia Artificial (IA)*

La Inteligencia Artificial (IA) como área interdisciplinaria del conocimiento, tiene como objetivo principal el desarrollo de sistemas computacionales capaces de simular habilidades cognitivas humanas, tales como: razonamiento, planificación, aprendizaje, percepción y adaptación a nuevas situaciones. (Russell & Norvig, 2020) Desde los planteamientos filosóficos más que técnicos, Alan Turing planteó en 1950 la posibilidad de que las máquinas pudieran pensar, y posteriormente con la formalización del término en 1956, la IA ha evolucionado desde enfoques simbólicos hasta sistemas de aprendizaje estadístico que permiten a las máquinas mejorar su desempeño a partir de los datos.

En particular, los modelos basados en redes neuronales han progresado rápidamente dado al crecimiento exponencial en el volumen de datos disponibles y al desarrollo constante de la capacidad computacional (OECD, 2019). En el sector de transporte y movilidad urbana, estos avances han permitido el diseño de soluciones que analizan datos en tiempo real, detectan patrones operativos y proponen decisiones automatizadas que búsqueda de mejorar la eficiencia y la calidad de los sistemas de transporte. (UITP, 2025).

4.2.3 *Machine Learning (ML)*

El Aprendizaje Automático (Machine Learning, ML) constituye uno de los subcampos más dinámicos y de mayor impacto dentro de la Inteligencia Artificial. A diferencia de los sistemas computacionales tradicionales, que operan siguiendo reglas explícitamente programadas, los sistemas de ML adquieren la capacidad de mejorar su desempeño a partir de la exposición a datos, sin requerir instrucciones detalladas para cada tarea específica. Esta característica los convierte en herramientas especialmente valiosas para abordar problemas donde la complejidad, la variabilidad y el volumen de información superan la capacidad de los enfoques convencionales (Ghazal et al., 2021).

Desde una perspectiva formal, el ML puede definirse como el proceso mediante el cual un sistema computacional generaliza relaciones a partir de ejemplos observados, con el objetivo de producir predicciones o decisiones válidas sobre datos no vistos previamente. Esta generalización no ocurre de forma aleatoria: se sustenta en técnicas estadísticas avanzadas que permiten al modelo identificar patrones, estructuras y dependencias en los datos, y trasladarlos a nuevos contextos con un margen de error controlado (Russell & Norvig, 2020). En este sentido, es un proceso matemático verificable cuya efectividad puede medirse y compararse de manera cuantitativa.

Particularmente en el caso de TransMilenio, el ML encuentra un campo de aplicación especialmente pertinente. Un sistema que opera más de cuatro millones de viajes diarios genera de forma continua flujos de datos que ningún analista humano podría procesar en tiempo real. Los modelos de aprendizaje automático ofrecen la capacidad de absorber esa complejidad, detectar regularidades en el comportamiento operativo del sistema y anticipar eventos antes de que se manifiesten como fallas visibles (retrasos acumulados, desbalance en la distribución de la flota, sobrecarga en estaciones críticas). Esta capacidad predictiva

es la que justifica su incorporación como eje metodológico central de la presente investigación (UITP, 2025; Lee & Yoon, 2022).

Dentro de la literatura especializada clasifica los algoritmos de ML en tres grandes paradigmas, cada uno definido por la naturaleza de los datos disponibles y el tipo de tarea que se busca resolver (ver tabla 3) (Russell & Norvig, 2020; Geron, 2022).

Tabla 3 Paradigmas del aprendizaje automático y su aplicación en el sistema TransMilenio

Paradigma	Definición	Aplicación en TransMilenio	Referencia
Aprendizaje supervisado	El modelo aprende la relación entre variables de entrada y una salida conocida (etiqueta), generalizando dicha relación a nuevas observaciones cuya salida se desconoce.	Predicción del tiempo de llegada y posición futura de buses a partir de variables operativas: posición GPS, velocidad, tiempo de parada, afluencia por estación y condiciones del tráfico.	Geron 2022, Russell & Norvig (2020)
Aprendizaje no supervisado	Opera sobre datos sin etiquetas predefinidas, buscando estructuras o agrupamientos latentes en la información sin predefinir una salida específica.	Identificación de segmentos de rutas con comportamientos similares, agrupación de estaciones según perfiles de demanda y detección de configuraciones operativas anómalas.	Geron, 2022
Aprendizaje por refuerzo	Un agente toma decisiones secuenciales e interactúa con su entorno recibiendo recompensas o penalizaciones, aprendiendo la política que maximiza su desempeño a largo plazo.	Optimización dinámica de frecuencias y despachos en corredores BRT, ajustando decisiones operativas en tiempo real según las condiciones del sistema.	Geron, 2022

Nota. Elaboración propia con base en Russell & Norvig (2020), Geron (2022) y Zhong et al. (2022).

4.2.4 Relevancia del ML para la predicción de trayectorias espaciotemporales.

Lo que distingue al aprendizaje automático de otras aproximaciones analíticas en el contexto de esta investigación, no es únicamente su capacidad para manejar grandes volúmenes de datos, sino su aptitud para capturar la naturaleza espaciotemporal de las

trayectorias vehiculares. A diferencia de los modelos estadísticos clásicos (que asumen relaciones lineales y estacionarias entre variables), los modelos de ML pueden aprender dependencias no lineales, efectos de interacción entre variables y patrones que varían según el contexto (hora del día, día de la semana, condiciones externas), sin requerir que el analista los especifique de antemano (Zhao et al., 2017).

Esta flexibilidad tiene implicaciones prácticas directas para TransMilenio. El comportamiento de un bus en una ruta troncal no sigue una función matemática simple: está mediado por decisiones humanas, condiciones del entorno, interacciones con otros vehículos y variaciones en la demanda que difícilmente pueden representarse mediante ecuaciones predefinidas. Los modelos de ML, en cambio, aprenden esas relaciones directamente de los datos históricos del sistema, construyendo representaciones internas que capturan la complejidad real de la operación (Panovski & Zaharia, 2020; Zhong et al., 2022).

El aprendizaje automático no se incorpora a esta investigación como una herramienta de tendencia tecnológica, sino como la respuesta metodológica más adecuada a las características específicas del problema: alta dimensionalidad de los datos, naturaleza espaciotemporal de las trayectorias, variabilidad dinámica del entorno operativo y necesidad de predicciones que orienten la gestión eficiente del sistema TransMilenio.

4.3 Trayectorias Espaciotemporales

Antes de abordar los criterios de evaluación de los modelos predictivos, es necesario precisar qué se entiende por *trayectoria* en el contexto de esta investigación, dado que este concepto constituye el objeto central sobre el cual operan los algoritmos de aprendizaje automático propuestos.

Una trayectoria, en el ámbito del transporte urbano, no se limita a la ruta geográfica que recorre un vehículo entre dos puntos. En su dimensión más completa, una trayectoria es una secuencia ordenada de estados espaciales y temporales que describe el comportamiento de un vehículo (flujo de vehículos a lo largo del tiempo). Cada estado está definido por al menos tres coordenadas fundamentales: la posición geográfica, el instante de tiempo en que esa posición es registrada, y un conjunto de variables operativas

asociadas a ese momento (velocidad, ocupación, tiempo de parada, desvíos, entre otras) (Zheng, 2015). La combinación de estas dimensiones es lo que convierte a una trayectoria en un objeto espaciotemporal, es decir, una entidad que no puede comprenderse ni modelarse si se analiza únicamente su componente espacial o únicamente su componente temporal de forma aislada.

Esta naturaleza dual del dato trayectoria tiene implicaciones metodológicas directas. Desde el punto de vista espacial, la posición de un bus en un momento dado no es un evento independiente: está condicionada por la geometría de la red vial, la ubicación de las estaciones, la distribución de la demanda a lo largo del corredor y la interacción con otros vehículos del sistema. Desde el punto de vista temporal, el comportamiento del vehículo en un instante determinado está fuertemente influenciado por lo que ocurrió en los instantes anteriores (efecto de memoria) con cierto grado de regularidad, lo que ocurrirá en los instantes siguientes. Es precisamente esta dependencia secuencial la que hace que los modelos capaces de capturar patrones temporales de largo plazo, como las redes LSTM, sean especialmente adecuados para este tipo de predicción (Zhao et al., 2017).

En el caso específico de TransMilenio, la trayectoria de un bus articulado en una ruta troncal no es estática ni perfectamente predecible. Está sujeta a perturbaciones continuas: variaciones en la demanda por estación y por franja horaria, eventos externos como manifestaciones o accidentes, condiciones climáticas, fallas mecánicas y el ya mencionado fenómeno de *bunching* (la acumulación de buses de la misma ruta en un mismo punto de la vía). Estas perturbaciones introducen incertidumbre en la trayectoria esperada y constituyen, al mismo tiempo, la razón de ser de los modelos predictivos: si el comportamiento del sistema fuera perfectamente regular y predecible, no existiría necesidad de modelarlo con algoritmos de aprendizaje automático.

El estudio de trayectorias espaciotemporales en sistemas BRT aporta al menos tres capacidades fundamentales para la gestión del sistema consolidadas en la tabla 4:

Tabla 4 Capacidades analíticas del análisis de trayectorias espaciotemporales en sistemas BRT

Capacidad analítica	Descripción	Aplicación en TransMilenio
Detección de patrones y anomalías Zhang et al. (2021)	El análisis sistemático de trayectorias históricas permite identificar comportamientos recurrentes (como picos de demanda en estaciones específicas durante determinadas franjas horarias) y detectar desviaciones respecto a esos patrones que pueden ser indicativas de fallas operativas emergentes, antes de que se consoliden y afecten la prestación del servicio.	Identificación anticipada de acumulación de buses (bunching), retrasos sostenidos y sobrecarga en estaciones críticas del sistema TransMilenio.
Predicción del tiempo de llegada Panovski & Zaharia (2020)	Al modelar la trayectoria como una secuencia espaciotemporal, es posible estimar con mayor precisión el tiempo en que un vehículo llegará a una estación determinada, considerando no solo la distancia recorrida, sino también las condiciones operativas actuales y los patrones históricos asociados a ese tramo, horario y día de la semana.	Generación de estimaciones de tiempo de llegada confiables por estación y por ruta, mejorando la información disponible para el usuario y para los operadores del sistema.
Modelado de la demanda y la ocupación Zhong et al. (2022)	Las trayectorias permiten correlacionar la posición del vehículo con la afluencia de pasajeros por estación, facilitando la construcción de modelos que anticipen niveles de ocupación y la evolución de la demanda a lo largo del corredor en distintas franjas horarias.	Orientación de la programación de frecuencias de despacho y distribución eficiente de la flota en los corredores troncales de TransMilenio según la demanda proyectada.

Nota. Elaboración propia con base en Zhang et al. (2021), Panovski & Zaharia (2020) y Zhong et al. (2022).

Comprender la trayectoria como un objeto espaciotemporal, es el punto de partida conceptual que permite justificar la elección de modelos de ML con capacidad de procesamiento secuencial, así como definir con precisión las variables de entrada, las métricas de evaluación y el alcance predictivo de la investigación. Es sobre esta base conceptual que se estructura el proceso metodológico descrito en los capítulos siguientes.

4.4 Algoritmos Relevantes para La Predicción de Trayectorias

La selección de algoritmos de ML para la predicción de trayectorias en sistemas de transporte masivo no es una decisión trivial. Cada algoritmo presenta ventajas, limitaciones y condiciones de aplicabilidad particulares que deben evaluarse en función de la naturaleza de los datos disponibles, la complejidad del entorno operativo y el tipo de variable que se desea predecir. A continuación, se caracterizan los algoritmos con mayor respaldo empírico en la literatura especializada para este tipo de tarea.

- **K-Vecinos más cercanos (k-NN)**

El algoritmo de *K-Vecinos* más cercanos es uno de los clasificadores no paramétricos más simples y ampliamente utilizados en problemas de predicción con datos de transporte. Su principio de funcionamiento consiste en asignar a una nueva observación la etiqueta o valor de sus k vecinos más próximos en el espacio de características, calculados mediante la distancia euclídea u otras métricas de similitud (Sari, 2020). En términos de predicción de tiempos de viaje entre estaciones, el k-NN puede identificar tramos con condiciones operativas similares (misma franja horaria, nivel de demanda, horas pico) y extrapolar los tiempos históricos observados en esas condiciones a nuevas situaciones. Su principal ventaja radica en su simplicidad e interpretabilidad; sin embargo, su rendimiento se degrada significativamente ante grandes volúmenes de datos y en presencia de variables de alta dimensionalidad, lo que limita su escalabilidad en sistemas de la complejidad de TransMilenio (Zhong et al., 2022).

- **Árboles de decisión y métodos de ensamble: Random Forest y XGBoost**

Los árboles de decisión constituyen la base de una familia de modelos especialmente potentes para la predicción en sistemas de transporte. Un árbol de decisión divide el espacio de características mediante reglas jerárquicas, construyendo una estructura en la que cada nodo interno aplica un criterio de partición y cada hoja terminal contiene una predicción (Rodrigo, 2020). Si bien un árbol individual puede ser interpretable, tiende a sobre ajustarse a los datos de entrenamiento, limitando su capacidad de generalización a nuevas observaciones.

Los métodos de ensamble superan esta limitación combinando múltiples árboles. El *Random Forest* (Breiman, 2001) construye un conjunto de árboles entrenados sobre subconjuntos aleatorios de datos y características (técnica denominada *bagging*), agregando sus predicciones mediante votación o premediación. Este enfoque reduce la varianza del modelo sin sacrificar su capacidad de capturar relaciones no lineales complejas (Rodrigo, 2020). En aplicaciones de predicción de tiempos de viaje en buses, el *Random Forest* ha demostrado consistentemente un desempeño superior al de los modelos lineales clásicos, especialmente cuando se incorporan variables temporales, espaciales y meteorológicas como predictores (Zhong et al., 2022).

- **El XGBoost (eXtreme Gradient Boosting)**

Propuesto por Chen & Guestrin (2016), representa la evolución más eficiente de los métodos de ensamble secuencial. A diferencia del *Random Forest*, que entrena los árboles en paralelo, el *XGBoost* los construye de forma secuencial: cada árbol nuevo aprende a corregir los errores residuales del árbol anterior, aplicando una expansión de Taylor de segundo orden sobre la función de pérdida, lo que mejora tanto la precisión como la velocidad de convergencia del modelo. Considerando características temporales, espaciales y meteorológicas como variables de entrada, (Zhong et al., 2022) demostraron que el *XGBoost* supera al k-NN, a redes neuronales de propagación hacia adelante y al *LightGBM* en términos de precisión, ajuste y estabilidad en la predicción del tiempo de viaje entre estaciones de bus. Su capacidad para manejar datos heterogéneos, resistir el sobreajuste mediante regularización y ofrecer interpretabilidad a través de métricas de importancia de características lo convierten en uno de los algoritmos más adecuados para el contexto operativo de TransMilenio.

- **Máquinas de Vectores de Soporte (SVM)**

Las Máquinas de Vectores de Soporte, formalizadas por Cortes & Vapnik (1995), son algoritmos de aprendizaje supervisado aplicables tanto a tareas de clasificación como de regresión. Su objetivo es encontrar el hiperplano óptimo que maximice el margen de separación entre clases (o que minimice el error de regresión) en un espacio n-dimensional, empleando funciones de núcleo (*kernel*) para mapear los datos a espacios de mayor dimensionalidad cuando las relaciones entre variables son no lineales (Cortes, 1995;

Matlab, 2025). En la predicción de tiempos de llegada de buses, las SVM han mostrado buen desempeño cuando se incorporan factores de ponderación temporal que asignan mayor relevancia a observaciones recientes, mejorando su capacidad para adaptarse a la variabilidad dinámica del sistema (Zhong et al., 2022). No obstante, su alto costo computacional ante grandes volúmenes de datos y su sensibilidad a la elección del *kernel* representan limitaciones relevantes para su aplicación a sistemas de alta frecuencia y gran escala operativa.

- **Redes Neuronales Recurrentes y arquitecturas LSTM**

Las Redes Neuronales Artificiales (ANN) constituyen el núcleo de los enfoques de aprendizaje profundo (*deep learning*) más avanzados en la predicción de trayectorias. A diferencia de los modelos anteriores, que tratan cada observación de forma relativamente independiente, las redes neuronales recurrentes (RNN) incorporan conexiones de retroalimentación que les permiten procesar secuencias de datos manteniendo una memoria interna de los estados anteriores (Goodfellow et al., 2016). lo que las convierte en herramientas naturalmente adecuadas para el análisis de series de tiempo operativas en sistemas de transporte.

Dentro de este grupo, las redes *Long Short-Term Memory* (LSTM), propuestas originalmente por Hochreiter & Schmidhuber (1997), representan la arquitectura más consolidada para la predicción de trayectorias vehiculares. Xie et al. (2021) señalan que las LSTM se especializan en memorizar secuencias largas y evitan eficazmente el problema del desvanecimiento del gradiente (característico de las RNN convencionales) mediante el uso de bloques de memoria con tres puertas de control: entrada (*input gate*), salida (*output gate*) y olvido (*forget gate*), que regulan selectivamente el flujo de información a lo largo del tiempo. Esta propiedad les permite capturar dependencias temporales de largo alcance: el comportamiento de un bus en un momento dado está influenciado no solo por su estado inmediatamente anterior, sino por patrones acumulados a lo largo de múltiples ciclos operativos (horas, días, semanas).

Agafonov & Yumaganov (2019) demostraron que los modelos LSTM que incorporan información heterogénea sobre la situación del tráfico superan a otros algoritmos típicos de predicción, logrando errores absolutos medios (MAE) de aproximadamente 0,4 minutos en

la estimación del tiempo de llegada de buses. Estudios más recientes han explorado arquitecturas híbridas que combinan LSTM con SVM (aprovechando la capacidad de las LSTM para capturar patrones temporales y la robustez de las SVM ante relaciones no lineales), obteniendo mejoras significativas en la precisión de los despachos programados respecto a los modelos individuales (Liu, 2020). En el contexto de TransMilenio, donde las trayectorias están condicionadas por múltiples variables secuenciales (posición, tiempos de parada, afluencia acumulada, condiciones del corredor), las arquitecturas LSTM y sus variantes representan los candidatos más prometedores para la construcción de modelos predictivos robustos y escalables.

4.5 Validación y Evaluación del Modelo de ML

La construcción de un modelo de aprendizaje automático no concluye con su entrenamiento. Para que un modelo sea útil en un contexto operativo real (como la predicción de trayectorias en TransMilenio) debe someterse a un proceso riguroso de evaluación que permita determinar con precisión cuánto y en qué condiciones el modelo es capaz de generalizar sus predicciones a datos no vistos durante el entrenamiento. Esta capacidad de generalización es, en definitiva, la que distingue un modelo robusto de uno que simplemente ha memorizado los patrones del conjunto de entrenamiento sin extraer conocimiento transferible (Russell & Norvig, 2020).

Evaluar un modelo de ML implica trabajar sobre dos dimensiones complementarias e interdependientes. La primera corresponde a las métricas cuantitativas de error, que miden la discrepancia entre los valores predichos por el modelo y los valores reales observados. La segunda corresponde a los métodos de validación, que garantizan que esa medición sea metodológicamente confiable y no esté sesgada por la forma en que los datos fueron divididos entre entrenamiento y prueba. Ambas dimensiones son inseparables: una métrica calculada sobre datos inadecuadamente divididos produce una falsa sensación de precisión, mientras que un método de validación robusto aplicado con una métrica mal elegida puede desorientar por completo la interpretación de los resultados (Kohavi, 2001; Russell & Norvig, 2020).

4.5.1 Métricas de evaluación para predicción de trayectorias

En tareas de predicción continua (como la estimación del tiempo de llegada o la posición futura de un vehículo), las métricas más utilizadas en la literatura especializada se explican a continuación.

- **Error Absoluto Medio (*Mean Absolute Error, MAE*)**

Calcula el promedio de las diferencias absolutas entre los valores predichos por el modelo y los valores realmente observados. Al tratar todos los errores de forma lineal (sin elevarlos al cuadrado), el MAE es especialmente adecuado cuando se desea una medida intuitiva de la discrepancia típica del modelo, sin penalizar de forma desproporcionada los errores extremos. En la predicción de tiempos de llegada de buses, un MAE de 1,5 minutos significa que, en promedio, el modelo se equivoca en ese margen temporal, lo que resulta directamente interpretable por operadores y usuarios del sistema (Olumide, 2023).

- **Error Cuadrático Medio (*Mean Squared Error, MSE*)**

Eleva al cuadrado las diferencias entre predicciones y valores reales antes de promediarlas. Esta operación penaliza con mayor intensidad los errores de gran magnitud, haciéndolo especialmente sensible a valores atípicos. Aunque su principal limitación radica en que no es directamente interpretable en las unidades originales de la variable predicha (al estar expresado en unidades al cuadrado), resulta útil durante el proceso de optimización del modelo, dado que sus propiedades matemáticas facilitan el cálculo del gradiente en algoritmos de entrenamiento (Zhang et al., 2021).

- **Raíz del Error Cuadrático Medio (*Root Mean Squared Error, RMSE*)**

Resuelve el problema de interpretabilidad del MSE al extraer su raíz cuadrada, devolviendo la métrica a las mismas unidades de la variable predicha. Mantiene la propiedad de penalizar los errores grandes con mayor intensidad que el MAE, siendo preferible en contextos donde las desviaciones extremas tienen consecuencias operativas significativas —como un bus con un retraso muy superior al promedio que genera acumulación en las estaciones aguas abajo (Olumide, 2023). En la predicción de trayectorias en sistemas BRT, Agafonov & Yumaganov (2019) reportaron valores de RMSE

consistentemente inferiores en modelos LSTM frente a modelos de regresión lineal y redes neuronales convencionales, confirmando la superioridad de las arquitecturas con memoria para este tipo de tarea.

- **Error Porcentual Absoluto Medio (*Mean Absolute Percentage Error, MAPE*)**

Expresa el error en términos relativos (como porcentaje del valor real), lo que facilita la comparación entre modelos que operan en distintas escalas o unidades y permite comunicar el desempeño del modelo de forma intuitiva a audiencias no técnicas (Oracle, 2025). Sin embargo, su aplicabilidad se ve limitada cuando los valores reales son cercanos a cero, situación en la que el porcentaje de error se dispara artificialmente. En el contexto de TransMilenio, donde los tiempos de recorrido entre estaciones raramente son cero, el MAPE resulta una métrica complementaria válida para reportar el desempeño general del modelo.

El Coeficiente de Determinación (R^2) cuantifica la proporción de varianza en la variable dependiente que es explicada por el modelo, tomando valores entre 0 y 1. Un valor de R^2 cercano a 1 indica que el modelo captura la mayor parte de la variabilidad del fenómeno estudiado, mientras que un valor cercano a 0 sugiere que el modelo no aporta información explicativa relevante más allá del promedio (Zhang et al., 2021). Es importante subrayar que un R^2 alto no garantiza por sí solo que el modelo sea preciso en todas las predicciones individuales (puede existir alta varianza explicada con errores puntuales significativos), razón por la cual debe interpretarse siempre en conjunto con el MAE o el RMSE para obtener una visión completa del desempeño predictivo.

La elección de la métrica adecuada no es neutral: responde a las prioridades operativas del sistema que se desea predecir. En el caso de TransMilenio, donde tanto la precisión promedio como la capacidad de detectar retrasos extremos son relevantes para la gestión del servicio, se recomienda el uso combinado de MAE y RMSE como métricas principales, complementadas con MAPE para la comunicación de resultados y R^2 para la evaluación de la capacidad explicativa global del modelo.

4.5.2 Métodos de Validación

La validación de modelos de aprendizaje automático constituye una etapa metodológica indispensable que determina si el desempeño reportado refleja la capacidad real de generalización del modelo o constituye un artefacto estadístico de los datos de entrenamiento. En el contexto de series temporales operativas (como las trayectorias de demanda del corredor Troncal 80 de TransMilenio), la elección del método de validación es especialmente crítica, dado que la naturaleza secuencial y auto correlacionada de los datos impone restricciones adicionales sobre los protocolos convencionales de evaluación (Zhong et al., 2022). Evaluar un modelo con los mismos datos con los que fue entrenado constituye un error metodológico fundamental (data leakage) que produce estimaciones artificialmente irrepresentables del desempeño real en condiciones operativas (Russell & Norvig, 2020; Kohavi, 2001).

Es importante señalar que los algoritmos empleados en esta investigación —Random Forest, XGBoost, SVR y Red Neuronal Artificial— son de propósito general y pueden aplicarse tanto a problemas de clasificación como de regresión. En esta investigación se implementan en su variante de regresión, dado que la variable objetivo son variables numéricas continuas que puede tomar cualquier valor entero no negativo. Por lo cual, las métricas de evaluación empleadas corresponden exclusivamente al paradigma de regresión (MAE, RMSE, R^2 y MAPE), descartando las métricas propias de clasificación como precisión, exhaustividad o F1-score, que son aplicables únicamente cuando la variable objetivo es categórica (Russell & Norvig, 2020).

- **División entrenamiento, validación y prueba (split temporal)**

El método de división entrenamiento prueba (*train ,test split*) es el protocolo fundamental de validación en aprendizaje automático supervisado para problemas de regresión. Los datos disponibles se dividen en tres subconjuntos mutuamente excluyentes con propósitos diferenciados: el conjunto de entrenamiento (que comprende típicamente entre el 70% y el 80% de las observaciones disponibles) se utiliza para ajustar los parámetros internos del modelo; el conjunto de validación (aproximadamente el 10% y el 15% se emplea durante el desarrollo para la selección de hiperparámetros y el seguimiento del ajuste); y el conjunto de prueba (10% y 15% restante), se reserva exclusivamente para la evaluación final del

desempeño sobre observaciones que el modelo no ha visto en ninguna etapa previa (Russell & Norvig, 2020). Este tipo de división garantiza que la estimación del desempeño sea estadísticamente imparcial y no esté contaminada por las decisiones de diseño tomadas durante el desarrollo del modelo.

En problemas de regresión sobre series temporales operativas, la división de los datos debe respetar estrictamente el orden cronológico de las observaciones mediante un split temporal. A diferencia del split aleatorio convencional, el split temporal asigna los períodos más recientes al conjunto de prueba y los períodos anteriores al entrenamiento, replicando la condición real de predicción en la que el modelo dispone únicamente de la historia pasada para anticipar el comportamiento futuro del sistema. La violación de este principio introduce data leakage temporal (situación en la que el modelo aprende implícitamente de observaciones futuras durante el entrenamiento), produciendo métricas de desempeño que no son reproducibles en condiciones operativas reales (Zhong et al., 2022). Adicionalmente, cualquier transformación de preprocesamiento (normalización de variables mediante MinMaxScaler), debe ajustarse exclusivamente sobre el conjunto de entrenamiento y aplicarse de forma transformativa sobre los conjuntos de validación y prueba, eliminando toda forma de contaminación de información entre conjuntos.

- **Validación cruzada K-fold**

La validación cruzada de k pliegues (k-fold cross-validation), formalizada por Kohavi (2001), complementa el split temporal durante la fase de selección de hiperparámetros. El conjunto de entrenamiento se divide en k particiones de igual tamaño denominadas pliegues (folds); el modelo se entrena y evalúa k veces, utilizando en cada iteración una partición distinta como conjunto de validación interna y las k-1 particiones restantes como datos de entrenamiento. El desempeño final reportado corresponde al promedio de las k evaluaciones independientes, lo que reduce significativamente el sesgo y la varianza introducidos por una única división aleatoria de los datos (Kohavi, 2001).

En el contexto de series temporales, la validación cruzada debe aplicarse respetando el orden cronológico dentro del conjunto de entrenamiento para prevenir el data leakage temporal. La combinación de validación cruzada para la selección de hiperparámetros (aplicada internamente sobre el conjunto de entrenamiento) con un conjunto de prueba

temporal independiente para la evaluación final constituye el protocolo de validación más robusto disponible para modelos de predicción de demanda en sistemas de transporte (Bergstra & Bengio, 2012). Bergstra y Bengio (2012) demostraron empíricamente que la búsqueda aleatoria de hiperparámetros (RandomizedSearchCV) obtiene aproximadamente el 95% del rendimiento óptimo del GridSearch exhaustivo con una fracción del costo computacional, resultado que justifica su adopción en investigaciones con grandes volúmenes de datos.

Tabla 5. Comparativa de métodos de validación para modelos de regresión en series temporales

Método	Propósito	Ventaja principal	Limitación
Split temporal (70/15/15)	Evaluación final imparcial del modelo sobre datos no vistos durante el entrenamiento ni la validación.	Replica la condición real de predicción futura al respetar el orden cronológico de los datos, evitando la fuga de información temporal (data leakage).	Requiere un volumen suficiente de datos en el período de prueba para que la evaluación sea estadísticamente representativa del comportamiento real del sistema.
k-fold cross-validation	Selección y ajuste de hiperparámetros durante la fase de desarrollo del modelo, reduciendo el sesgo de una única división aleatoria.	Garantiza que todas las observaciones participen en la validación en alguna iteración, lo que produce estimaciones de desempeño más estables y menos dependientes de la partición inicial.	No debe aplicarse de forma aleatoria en series temporales: requiere respetar el orden cronológico de los datos en cada pliegue para evitar que el modelo aprenda del futuro al predecir el pasado (Kohavi, 2001; Zhong et al., 2022).
RandomizedSearchCV	Búsqueda eficiente en el espacio de hiperparámetros del modelo mediante muestreo aleatorio de combinaciones candidatas.	Obtiene aproximadamente el 95 % del rendimiento óptimo que lograría una búsqueda exhaustiva (GridSearch), con una fracción significativamente menor del costo computacional (Bergstra & Bengio, 2012).	Al basarse en muestreo aleatorio, no garantiza la identificación del óptimo global del espacio de hiperparámetros. Los resultados pueden variar entre ejecuciones si no se fija una semilla de aleatoriedad.

Nota. Elaboración propia con base en Kohavi (2001), Russell y Norvig (2020), Bergstra y Bengio (2012) y Zhong et al., 2022).

La integración sistemática de los métodos de validación y las métricas de evaluación descritos constituye una condición de rigor científico, no un requisito formal. En el contexto de esta investigación, la evaluación comparativa de los cuatro modelos de regresión candidatos mediante este conjunto de herramientas permitirá seleccionar, con evidencia cuantitativa sólida, el algoritmo con mejor capacidad predictiva para las trayectorias de

demanda del corredor Troncal 80, garantizando que los resultados obtenidos sean generalizables a condiciones operativas reales y no solo a los datos históricos utilizados durante el entrenamiento.

4.6 Investigaciones Previas

El Machine Learning dentro de la literatura refleja un alto grado de aumento por su versatilidad en aplicación, como en la robustez de sus resultados, en el campo del transporte urbano masivo como los BRT, ha sido ampliamente implementado en recientes investigaciones como herramienta clave en la optimización de fallas críticas de este tipo de sistemas, y como metodología innovadora para la predicción de la demanda, los tiempos de llegada y la planificación operativa.

Estas investigaciones, han sido desarrolladas en diferentes contextos alternos al que se aborda con esta investigación, pero proporcionan marcos metodológicos y hallazgos que resultan útiles para abordar los desafíos que proporciona el sistema de TransMilenio.

Las investigaciones que se abordan a continuación son estudios relevantes que con sus contribuciones tanto teóricas como practicas aportan a la consolidación metodológica, así como para la construcción y orientación de la presente propuesta investigativa:

El artículo ,denominado: "*Comparative Analysis of Different Machine Learning Techniques for Travel Mode Prediction*" desarrollado por Bhosle et al. (2024) genero una comparación entre varios algoritmos de clasificación supervisada, incluyendo *k-Nearest Neighbors*, *Support Vector Machines*, *Random Forest* y *Extra Trees*, aplicados a una base de datos proporcionada por Microsoft. El objetivo general fue determinado mediante la identificación del modelo más eficaz para predecir qué modo de transporte es utilizado por los usuarios mediante datos de coordenadas GPS de origen, Coordenadas GPS de destino, hora de inicio y finalización del viaje.

Los resultados obtenidos generaron que el algoritmo Extra Trees logró una precisión del 97 %, superando a las demás modelos evaluadas dentro de la investigación. Este estudio evidencia el potencial del ML en la modelación de comportamientos de movilidad, aunque su alcance se centró únicamente en la identificación del modo de transporte sin

abordar la dimensión espaciotemporal de las trayectorias aporta un punto de partida en la visión y aplicabilidad que tienen ciertos tipos de modelos en diversos contextos y dar un acercamiento a que modelos son capaces de anticipar de forma dinámica el comportamiento operativo del sistema de TM.

Por otro lado, la investigación "Equality of Opportunity in Travel Behavior Prediction with Deep Neural Networks and Discrete Choice Models" de Zheng et al. (2021) generan una perspectiva más sólida en el uso de técnicas avanzadas de predicción de trayectorias en diversos contextos de movilidad urbana.

En esta investigación, los modelos DNN se compararon visualmente con DCM con encuestas nacionales de transporte en los EE. UU. El objetivo principal fue evaluar la precisión y la equidad de ambos modelos para predecir diferentes comportamientos durante un viaje. Al hablar de equidad dentro de este contexto, se refiere a que los modelos pueden ser injustos en ciertos panoramas y diferentes grupos vulnerables, ya que pueden llegar a subestimar la demanda a zonas marginales de las ciudades y se demuestra por medio de que las DNN ofrecen mayor capacidad de predicción en demanda y en la planificación del servicio, siguen presentando variables significativas al predecir patrones de viaje para grupos vulnerables, personas con cierto tipo de discapacidad o usuarios de bajos ingresos económicos.

Este trabajo aporta en la forma de cómo hay que comprender que la calidad de los datos es indispensable para el éxito de cualquier modelo de ML, especialmente en sistemas con alta diversidad demográfica como TM., aunque la presente propuesta de investigación no tiene un enfoque en la evaluación de la equidad en el sistema, sí se adopta para enfatizar la necesidad de datos limpios, y más representativos para el análisis de trayectorias y que modelos pueden ser usados.

Bajo el ámbito de la planificación operacional en los sistemas de transporte masivo, Zhang et al. (2021) en su artículo "Analysis of Spatial Temporal Characteristics of Operations in Public Transit Systems", desarrollan un estudio enfocado en identificar las variaciones espacio temporales del transporte público, basándose en registros GPS y de demanda en diferentes franjas horarias. El análisis permitió identificar patrones operativos y zonas de congestión que afectan el desempeño del sistema en distintas horas del día. Su aporte

resalta en la relevancia de considerar variables dinámicas y condiciones externas (como la ubicación, la hora o la densidad de pasajeros) para lograr una gestión más eficiente. En esa línea, esta tesis extiende dicho enfoque integrando estos datos dentro de modelos de ML, lo cual permite una predicción más precisa y adaptativa de trayectorias en un entorno tan complejo como el de TransMilenio.

Finalmente, la investigación "Real Time Public Transportation Prediction with Machine Learning Algorithms" de Panovski & Zaharia (2020) representa una contribución directa al campo de la predicción de tiempos de llegada. La investigación aplicó y comparó los algoritmos de regresión lineal, máquinas de vectores de soporte (SVM) y redes neuronales completamente conectadas (FNN) con datos de rutas de autobuses en tiempo real.

Sin embargo, el contexto aplicado comparado con el sistema de TM es un sistema que no tiene tanta afluencia de pasajeros, y cabe destacar que dentro de las recomendaciones y trabajos futuros está su aplicación a sistemas mucho más robustos y mayor complejidad operativa.

5. CAPITULO II: EXPLORACIÓN Y PREPARACIÓN DE DATOS ESPACIOTEMPORALES

La calidad y representatividad del dataset constituyen el fundamento sobre el cual se sustenta la capacidad predictiva de cualquier modelo de aprendizaje automático. En proyectos aplicados al transporte público, esta premisa adquiere especial relevancia dado el alto volumen de registros diarios, la heterogeneidad de las fuentes de información y la complejidad de los patrones operacionales que deben ser capturados (Zhang et al., 2021; Zhong et al., 2022). El presente capítulo documenta el proceso completo de exploración y preparación de los datos espaciotemporales empleados en esta investigación, estructurado en cinco etapas secuenciales: identificación de datos, recolección, transformación y limpieza, estandarización e integración de fuentes, y generación de variables derivadas.

5.1 Identificación de Datos Espaciotemporales.

Los datos espaciotemporales describen los objetos en movimiento que participan directamente en la operación del sistema, capturando y etiquetando el comportamiento de las variables en un espacio determinado a través del tiempo (Zheng, 2015). En el contexto operativo de TransMilenio, estas variables registran dinámicamente el dónde y el cuándo de cada evento: la validación de un usuario mediante tarjeta Tu Llave en una estación específica, el egreso de pasajeros por un acceso físico determinado y el flujo de demanda en un intervalo de tiempo definido. Los parámetros clave capturados son la ubicación del evento (portal, estación o troncal) y su instancia temporal exacta (fecha e intervalo de 15 minutos). La integración de ambas dimensiones en un mismo registro es la condición que habilita el análisis de trayectorias espaciotemporales en el sentido definido en esta investigación.

Para la selección de fuentes se establecieron tres criterios: cobertura espacial explícita mediante código de nodo (`cod_nodo`), resolución temporal de 15 minutos (96 intervalos diarios) y disponibilidad pública en portales institucionales de datos abiertos en cumplimiento de la Ley 1712 de 2014. Con base en estos criterios se identificaron cuatro

fuentes de datos complementarias que cubren las dimensiones espacial, temporal y operativa del corredor Troncal 80 (TZ005) para el período enero 2025 – marzo 2026.

5.1.1 Delimitación espacial a la Troncal 80

TransMilenio es el componente troncal del Sistema Integrado de Transporte Público de Bogotá D.C., Inaugurado el 4 de diciembre de 2000, el sistema se organiza en tres fases de implementación: Fase I (Calle 80, Av. Caracas, Autopista Norte y Calle 13), Fase II (Américas, NQS y Suba) y Fase III (Calle 26 y Carrera 10ª), con una demanda promedio de 4.000.000 de viajes diarios en el SITP completo y más de 50,8 millones de validaciones troncales registradas en el primer trimestre de 2025 (TransMilenio S.A., 2025; Bogotá.gov.co, 2024).

La red troncal opera con buses articulados (hasta 160 pasajeros y biarticulados de hasta 250 pasajeros), con una velocidad comercial promedio de 26 km/h y una flota eléctrica de 1.486 unidades (TransMilenio S.A., 2025; Hidalgo et al., 2024), en La Tabla 1 relaciona los principales indicadores operacionales del sistema.

Tabla 6. Indicadores operacionales del Sistema TransMilenio — 2025

Indicador	Valor	Fuente
Demanda promedio diaria (SITP)	4.000.000 viajes/día	Bogotá.gov.co (2024)
Validaciones troncales 1er trim. 2025	50,8 millones	TransMilenio S.A. (2025)
Flota eléctrica operativa	1.486 buses	Bogotá.gov.co (2024)
Velocidad comercial promedio	26 km/h	Hidalgo et al. (2024)

Nota. SITP = Sistema Integrado de Transporte Público. Datos correspondientes al primer trimestre de 2025. Elaboración propia con base en fuentes institucionales.

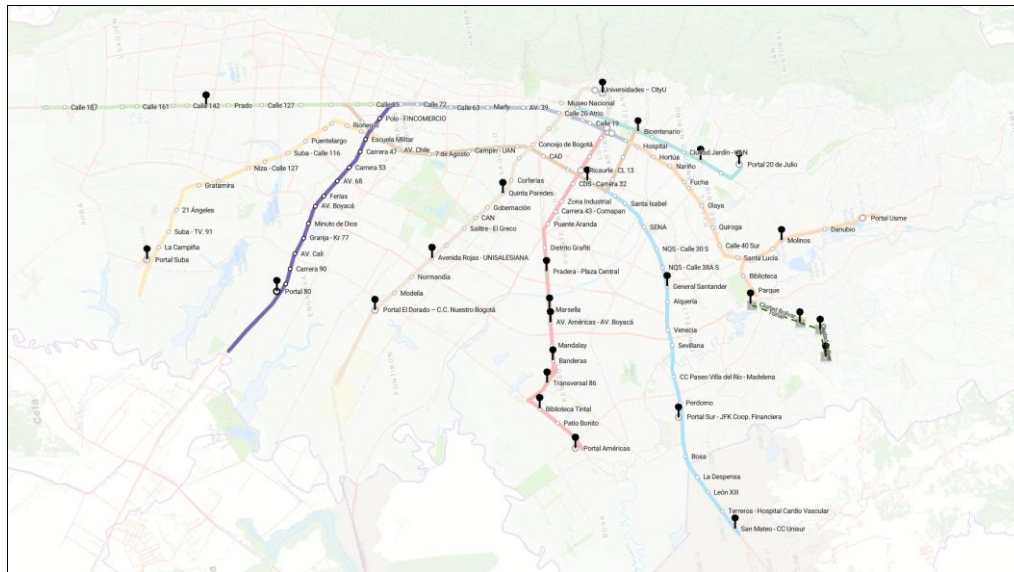
De los corredores que conforman la red troncal, esta investigación delimita su alcance al corredor Troncal 80 (TZ005) como caso de estudio, con base en cuatro criterios metodológicos.

1. El corredor pertenece a la Fase I del sistema (inaugurada el 4 de diciembre de 2000) lo que garantiza la mayor antigüedad y continuidad histórica de registros operacionales disponibles en el portal de datos abiertos de TransMilenio S.A., condición que Zhong et al. (2022) identifican como determinante para la construcción de modelos predictivos estadísticamente robustos en sistemas BRT.

2. Su extensión de 7,38 km con 13 estaciones activas entre Portal 80 (extremo occidental y Polo FINCOMERCIO extremo oriental) representa una escala espacial que equilibra la riqueza del análisis, evitando la complejidad de los corredores de mayor longitud sin sacrificar la representatividad de los patrones de demanda del sistema (Zhang et al., 2021).
3. El corredor articula de forma simultánea múltiples tipos funcionales de estación — nodo cabecera de alta demanda (Portal 80), nodos de transferencia intertronal (Av. Boyacá, Polo–FINCOMERCIO), estaciones de demanda media en zonas comerciales (Av. 68, Av. Ciudad de Cali) y estaciones de baja demanda en zonas residenciales (Escuela Militar, Carrera 47), diversidad que permite evaluar la capacidad de generalización de los modelos sobre patrones operativos cualitativamente distintos dentro de un mismo corredor, siguiendo la metodología de evaluación espacial propuesta por Lee y Yoon (2022).
4. El volumen de datos disponibles para este corredor proporciona una base estadística suficiente para el entrenamiento, validación y evaluación robusta de los cuatro modelos de aprendizaje automático desarrollados en esta investigación, en línea con los requerimientos mínimos de volumen de datos documentados por Cats et al. (2020) para modelos de predicción de demanda en sistemas de transporte masivo.

Desde el punto de vista funcional, la Troncal 80 cumple el rol de corredor de distribución occidental: recibe pasajeros procedentes de los municipios del occidente metropolitano de Bogotá (Cota, Funza, Mosquera a través del Portal 80 y los distribuye hacia el centro de la ciudad. La Figura 2 presenta la localización del corredor en la red troncal de TransMilenio.

Figura 2 Mapa de Troncales Sistema Transmilenio Bogotá D.C – Troncal calle 80



Nota. La troncal 80 se encuentra señalada dentro de la figura de color púrpura. Fuente <https://mapadigital.transmilenio.gov.co/>

5.2 Fuentes de Datos

Los datos empleados en esta investigación provienen de dos portales institucionales de acceso público: el portal de datos abiertos de TransMilenio S.A. y el portal de datos abiertos de Bogotá D.C. Ambas plataformas publican sus conjuntos de datos en cumplimiento de la Ley 1712 de 2014, norma que regula el derecho de acceso a la información pública en Colombia y obliga a las entidades del Estado a disponer su información en formatos abiertos y reutilizables por cualquier ciudadano o investigador.

La extracción de los datos fue realizada de forma manual mediante descarga directa desde cada portal. Esta modalidad de acceso fue la única viable, dado que ninguna de las dos plataformas dispone de una interfaz de programación de aplicaciones (API) que permita la consulta automatizada de los conjuntos de datos en los formatos y períodos requeridos por la investigación. El proceso implicó la identificación, selección y descarga individualizada de cada archivo disponible para el período comprendido entre enero de

2025 y marzo de 2026, acumulando un total de 34 archivos con 58.297.600 registros brutos antes de cualquier intervención de preprocesamiento o limpieza.

Para garantizar que las fuentes seleccionadas fueran técnicamente pertinentes para la construcción del dataset espaciotemporal, se establecieron tres criterios de selección de aplicación:

1. La cobertura espacial explícita mediante código de nodo (`cod_nodo`), campo que permite vincular cada registro a una ubicación geográfica concreta dentro del corredor y que constituye la condición mínima para el análisis de trayectorias.
2. La resolución temporal de 15 minutos, que equivale a aproximadamente 76 intervalos por día de operación (resultado de dividir las 19 horas de funcionamiento efectivo del sistema, comprendidas entre las 4:30 a.m. y las 11:30 p.m., entre intervalos de un cuarto de hora ($19 \times 4 = 76$)). Esta granularidad no es arbitraria: en sistemas de transporte masivo de alta frecuencia como TransMilenio, los patrones de demanda cambian de forma significativa dentro de una misma hora. Un intervalo de 60 minutos fusionaría en un único registro comportamientos operativos muy distintos (el inicio de la hora pico, su punto más alto y su declive) perdiendo la precisión necesaria para detectar patrones relevantes. Por el contrario, un intervalo inferior a 15 minutos produciría registros con volúmenes de pasajeros muy bajos en estaciones de menor afluencia, introduciendo ruido estadístico que afectaría la estabilidad de los modelos. Los 15 minutos representan el punto de equilibrio entre detalle suficiente para capturar la variabilidad real de la demanda y estabilidad estadística en los registros, condición necesaria para que los algoritmos de ML identifiquen patrones temporales significativos y no aprendan sobre fluctuaciones aleatorias sin valor predictivo (Zhong et al., 2022; Zhang et al., 2021).
3. La disponibilidad pública verificada en los portales institucionales mencionados, condición que garantiza la reproducibilidad de la investigación y su alineación con los principios de transparencia y ciencia abierta exigidos por la normativa colombiana vigente.

Con base a los criterios establecidos anteriormente, se identificaron cuatro conjuntos de datos que cubren las dimensiones espacial, temporal y operativa del corredor Troncal 80. En la Tabla 1 se consolidan las características técnicas de cada

fuente (formato, número de archivos, volumen bruto y dimensión cubierta) con el fin de ofrecer una visión de conjunto del material empírico sobre el que se sustenta la investigación.

Tabla 7. Características técnicas de cada fuente

Fuente	Formato	Archivos	Volumen bruto	Dimensión cubierta
Validaciones de ingreso (Tu Llave)	Excel (.xlsx)	14 mensuales	34.078.548 registros	Demanda: ingresos al sistema
Salidas de pasajeros	Excel (.xlsx)	14 mensuales	24.219.052 registros	Demanda: egresos del sistema
Atributos de estaciones	Excel/CSV	1 archivo	13 nodos del corredor	Espacial infraestructura
GTFS 2026 (5 archivos)	TXT (CSV)	5 archivos	37.894 stop_times	Oferta programada
TOTAL	—	34 archivos	58.297.600 registros	Espacial, temporal, operativa

Nota. Fuentes disponibles en el portal de datos abiertos de TransMilenio S.A. y el portal de datos abiertos de Bogotá D.C. Período: enero 2025 – marzo 2026. Elaboración propia.

5.3 Recolección de Datos

La recolección de datos comprende el conjunto de procedimientos técnicos mediante los cuales los archivos publicados en los portales institucionales fueron accedidos, procesados y preparados para su uso en las etapas posteriores de la investigación. Este proceso no se limitó a la descarga de los archivos: implicó decisiones técnicas específicas sobre las herramientas de extracción, los formatos de almacenamiento y los criterios de filtrado, cada una de las cuales incide directamente sobre la integridad y la usabilidad de los datos resultantes.

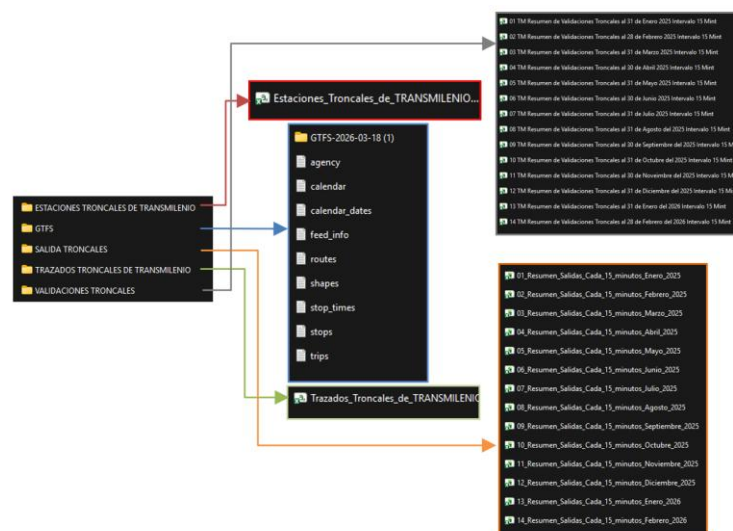
El lenguaje de programación utilizado en todas las etapas fue Python, elección que responde a tres razones concretas: su ecosistema de librerías especializadas en ciencia de datos y aprendizaje automático (pandas, scikit learn, TensorFlow, entre otras), su amplia adopción en la comunidad académica e investigativa, y su capacidad para integrar en un mismo entorno las etapas de extracción, preprocesamiento, modelado y evaluación sin necesidad de cambiar de herramienta. Lenguajes alternativos como R ofrecen capacidades estadísticas comparables, pero presentan limitaciones relevantes en la implementación de

modelos de aprendizaje profundo y en el manejo eficiente de volúmenes de datos superiores a los 50 millones de registros, como los que caracteriza esta investigación (McKinney, 2022; VanderPlas, 2016). A continuación, se describe el proceso de recolección de forma secuencial.

Paso 1. Descarga y organización de archivos

Los archivos fueron descargados manualmente desde los portales institucionales en sus formatos originales de publicación: Excel (.xlsx) para los registros de validaciones de ingreso y salidas de pasajeros, Excel/CSV para los atributos de estaciones y TXT con estructura CSV para los archivos GTFS. Una vez descargados, los archivos fueron organizados en una estructura de directorios local diferenciada por fuente y período mensual, con el fin de mantener la trazabilidad del origen de cada archivo durante las etapas de procesamiento.

Figura 3. Archivos Organizados



Nota. Se representa como quedaron cada uno de los archivos guardados y organizados. Elaboración propia

Esta práctica de organización es reconocida en la literatura de ciencia de datos como una condición básica de reproducibilidad del proceso analítico: sin una estructura de

directorios coherente y documentada, la verificación independiente del proceso por parte de otros investigadores resulta prácticamente inviable (Wilson et al., 2017).

Paso 2. Extracción y lectura de archivos Excel

La lectura de los archivos Excel fue implementada mediante la librería `openpyxl`, herramienta de código abierto ampliamente utilizada para la manipulación programática de archivos en formato `.xlsx` (`openpyxl Development Team`, 2023). La elección de esta librería sobre la alternativa más común (función `read_excel` de `pandas`) responde a una limitación técnica específica de los archivos fuente.

En los registros de salidas de pasajeros publicados por TransMilenio S.A., las variables de fecha están almacenadas como fórmulas de Excel y no como valores directos. Al utilizar `read_excel` con su configuración estándar, estas celdas devuelven valores nulos o cadenas de texto con la fórmula sin evaluar, corrompiendo la variable temporal más crítica del análisis. El acceso directo mediante `openpyxl` con los parámetros `data_only=True` que instruye a la librería para recuperar los valores calculados por las fórmulas en lugar de las fórmulas mismas y `read_only=False` que mantiene el archivo en modo de lectura completa, condición necesaria para que las fórmulas de fecha sean procesadas correctamente, fue la única configuración que permitió recuperar los valores temporales de forma confiable, garantizando la integridad de la dimensión temporal del dataset (`openpyxl Development Team`, 2023).

Paso 3. Almacenamiento en formato Apache Parquet

Una vez extraídos, los datos de cada fuente fueron convertidos y almacenados en formato Apache Parquet en lugar de conservarlos en sus formatos originales. Esta decisión responde a limitaciones técnicas concretas que los formatos CSV y Excel presentan cuando se trabaja con volúmenes de datos de la escala de esta investigación.

Cuando un archivo CSV se abre en Python, todas las columnas son tratadas inicialmente como texto, independientemente de si contienen fechas, números enteros, decimales o categorías. Esto implica que en cada lectura el programa debe inspeccionar el contenido de cada celda e intentar convertirlo al tipo de dato correcto, operación que consume tiempo de procesamiento y puede generar errores silenciosos cuando el formato

no es consistente (situación documentada en los archivos de salidas de TransMilenio, donde la columna de fecha presenta dos formatos distintos según el período (cadena de texto `_AAAA_MM_DD` para enero–noviembre 2025 y objeto `datetime` para diciembre 2025 en adelante)). El formato Excel agrega una restricción adicional: cada hoja admite un máximo de aproximadamente 1,04 millones de filas, lo que hace inviable el procesamiento unificado de los 58.297.600 registros brutos del sistema sin ningún tipo de fragmentación.

Apache Parquet resuelve estos problemas al ser un formato de almacenamiento en columnas (*columnar storage*) donde cada columna se guarda de forma contigua en disco con su tipo de dato nativo ya definido: `cod_nodo` como categoría, la columna de fecha como `datetime`, los conteos como enteros de 16 o 32 bits según su rango. Esto elimina la necesidad de reconversión en cada lectura y reduce el riesgo de errores de tipado. Adicionalmente, Parquet aplica compresión automática sobre cada columna (típicamente entre el 60% y el 80% de reducción respecto al CSV equivalente) y permite leer únicamente las columnas requeridas para cada operación sin cargar el archivo completo en memoria (Apache Software Foundation, 2024).

Este tipo estrategia de almacenamiento adoptada generó un archivo Parquet independiente por mes y por fuente, respetando la granularidad temporal de los archivos originales. De tal forma que los 14 archivos Excel mensuales de validaciones de ingreso produjeron 14 archivos Parquet individuales (ejemplo: `validaciones_2025-01.parquet`, `validaciones_2025-02.parquet`, ..., `validaciones_2026-03.parquet`), y de igual forma los 14 archivos de salidas de pasajeros. Cuando dos archivos Excel de distintos meses contenían registros del mismo período (situación que ocurrió en algunos meses con solapamiento de fechas) el pipeline aplicó de duplicación automática sobre la clave compuesta [`cod_nodo`, `fecha`, `intervalo`] antes de escribir el Parquet resultante. El archivo de atributos de estaciones y los cinco archivos GTFS fueron procesados individualmente dado su menor volumen. Esta organización por período facilita la lectura selectiva durante el procesamiento posterior: en lugar de cargar los 28 meses completos, cada operación de filtrado o agregación accede únicamente a los archivos del período requerido (McKinney, 2022). El resultado fue una reducción del espacio de almacenamiento de aproximadamente 4,2 GB en formatos originales a menos de 900 MB en el conjunto de archivos Parquet, con tiempos de lectura que pasaron de varios minutos (en el caso de los archivos Excel originales) a segundos en

cada operación de filtrado y agregación posterior. Esta mejora en el rendimiento fue especialmente relevante durante las etapas iterativas de preprocesamiento, donde los mismos archivos debieron leerse y procesarse múltiples veces para ajustar parámetros y verificar resultados.

5.4 Filtrado por troncal calle 80

TransMilenio S.A. publica todos sus conjuntos de datos de forma consolidada para la totalidad de la red, sin distinción previa por corredor. Esto aplica tanto para los archivos de demanda (validaciones de ingreso y salidas de pasajeros) como para el archivo de atributos de estaciones y el GTFS. En todos los casos, los archivos descargados contienen información de todos los corredores del sistema simultáneamente, por lo que fue necesario aplicar un proceso de filtrado sobre cada fuente para delimitar el conjunto de trabajo al corredor Troncal 80 (TZ005).

El criterio de filtrado se fundamenta en el campo `cod_nodo`, que identifica de forma unívoca la estación donde ocurrió cada evento registrado. Dentro del sistema de información de TransMilenio, cada estación física está identificada mediante este código único (funcionalmente equivalente a un número de identificación), que la distingue del resto de las estaciones de la red. La Troncal 80 está compuesta por 13 estaciones físicas con sus correspondientes códigos asignados (04000–04108). Cuando se descarga un archivo de validaciones de ingreso, ese archivo contiene registros de usuarios que ingresaron al sistema por cualquier estación de Bogotá (por ejemplo: Portal El Dorado, Museo del Oro, Calle 72...etc.). Cada registro tiene anotado el `cod_nodo` de la estación donde ocurrió el ingreso. El filtrado consistió entonces en conservar únicamente los registros cuyo `cod_nodo` correspondía a alguna de las 13 estaciones del corredor, descartando todos los demás. El resultado es conceptualmente equivalente a tomar el registro completo de ingresos al sistema durante 15 meses y quedarse exclusivamente con los que ocurrieron en las estaciones de un corredor específico.

Técnicamente, este filtrado no se aplicó sobre los archivos Excel originales sino sobre los 28 archivos Parquet generados en el paso anterior (14 de validaciones y 14 de salidas, uno por mes). Esta distinción es metodológicamente relevante: el formato Parquet

almacena cada columna de forma contigua en disco con su tipo de dato nativo ya definido, lo que permite acceder directamente a la columna `cod_nodo` y seleccionar los registros correspondientes sin necesidad de leer ni cargar en memoria el resto de las columnas del archivo. El filtrado se ejecutó de forma independiente sobre cada uno de los 28 archivos Parquet, procesando un mes a la vez, lo que redujo el tiempo de operación de varios minutos (tiempo que habría requerido un recorrido línea por línea sobre los Excel originales) a segundos por archivo.

Para las fuentes estructurales, el criterio de filtrado fue distinto según la fuente. En el archivo de atributos de estaciones, se seleccionaron únicamente los 13 registros correspondientes a los nodos del corredor. En el GTFS, se filtraron los registros de `stop_times` y `trips` asociados exclusivamente a las rutas identificadas como pertenecientes a la Troncal 80 en el archivo `routes.txt`, descartando las rutas del resto del sistema.

El resultado acumulado del filtrado sobre las cuatro fuentes fue el siguiente: de los 58.297.600 registros brutos disponibles para el período enero 2025–marzo 2026, el proceso delimitó el conjunto de trabajo a 2.961.104 registros correspondientes exclusivamente a las 13 estaciones de la Troncal 80, equivalentes al 5,1% del total del sistema. El volumen final fue distribuido en 28 archivos Parquet ya filtrados al corredor (constituye el insumo directo sobre el que se ejecutan los pasos siguientes de limpieza e integración del dataset espaciotemporal ver tabla 8).

Tabla 8. Resultado del filtrado por corredor Troncal 80 (TZ005) sobre las fuentes de datos adquiridas

Fuente	Archivos	Registros brutos total sistema	Registros filtrados Troncal 80 (TZ005)	Descartados (otros corredores)	% del sistema
Validaciones de ingreso (Tu Llave)	14	34.078.548	1.302.310	32.776.238	3,8%
Salidas de pasajeros	14	24.219.052	1.658.794	22.560.258	6,8%
TOTAL FUENTES DE DEMANDA	28	58.297.600	2.961.104	55.336.496	5,1%

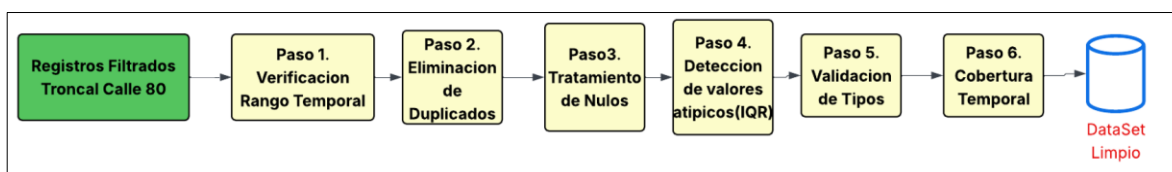
Atributos de estaciones	1	N/A (sistema completo)	13 nodos TZ005	Resto del sistema	—
GTFS 2026 (stop_times)	5	N/A (sistema completo)	37.894 entradas	Resto de rutas	—

Nota. El criterio de filtrado para las fuentes de demanda fue el campo `cod_nodo` (rango 04000–04108 para TZ005). Para el GTFS, se filtraron los registros de `stop_times` y `trips` asociados a las 20 rutas de la Troncal 80 identificadas en `routes.txt`. El filtrado se aplicó sobre los archivos Parquet generados por fuente y mes. N/A = no aplica como conteo de registros (archivos de referencia estructural). % del sistema calculado sobre los 58.297.600 registros brutos de las fuentes de demanda. Elaboración propia con base en datos abiertos de TransMilenio S.A. (2025-2026).

5.5 Limpieza de Datos

Dado que los datos provienen directamente del sistema de validación por medios electrónicos como la tarjeta tullave presentan frecuentemente inconsistencias derivadas de fallos en torniquetes, registros duplicados por errores de transmisión y valores atípicos asociados a eventos operacionales excepcionales (Zhong et al., 2022). Para garantizar la integridad del dataset filtrado, se implementó un proceso estructurado en seis pasos secuenciales aplicados de forma independiente sobre los 28 archivos Parquet del corredor. La Figura 3 resume el flujo y los resultados de este proceso.

Figura 4. Diagrama de flujo proceso limpieza de datos



Nota. Se describe los pasos secuenciales del proceso de limpieza de datos (Elaboración Propia)

Paso 1: La presencia de registros con fechas fuera del período de estudio o con valores malformados introduce ruido sistemático que distorsiona los patrones aprendidos por los modelos (Zhang et al., 2021). Se verificó que todos los registros pertenecieran estrictamente al período enero 2025–marzo 2026, descartando aquellos cuya fecha no pudiera convertirse a un objeto `datetime` válido o cuyo valor estuviera fuera del intervalo definido. Esta verificación fue necesaria por dos razones técnicas: los encabezados de fecha en los archivos Excel son frecuentemente el resultado de fórmulas con referencias entre hojas que, al ser evaluadas por `openpyxl`, pueden devolver valores nulos o fechas del año 1900

(la fecha base de Excel cuando una fórmula no resuelve correctamente); y el solapamiento entre archivos mensuales consecutivos puede introducir registros de fechas adyacentes al período de interés.

Paso 2: Un mismo evento operacional puede quedar registrado más de una vez cuando dos archivos mensuales consecutivos comparten fechas en sus extremos, generando duplicados que inflan artificialmente los conteos de demanda y sesgan el entrenamiento de los modelos (Zhong et al., 2022). Para eliminarlos, se identificó cada registro de forma unívoca mediante la clave compuesta [cod_nodo, fecha, intervalo] —que define con precisión la estación, el día y el intervalo de 15 minutos donde ocurrió cada evento— conservando únicamente la primera ocurrencia y descartando las restantes.

Paso 3: Un registro sin valor en alguna de las tres variables estructurales —cod_nodo, fecha e intervalo— no puede ser ubicado ni espacial ni temporalmente dentro del corredor, lo que lo hace analíticamente inútil e introduce ambigüedad en el proceso de integración posterior (Rahman et al., 2021). Se eliminaron todos los registros con valores faltantes en estas variables, dado que su imputación con valores aproximados introduciría información ficticia en las dimensiones más críticas del dataset.

Paso 4: La detección de valores atípicos en datos de demanda de transporte representa uno de los desafíos más delicados del preprocesamiento, dado que un valor extremo puede ser el resultado de un error de registro (un torniquete que contabilizó validaciones de forma errónea o un evento operacional legítimo como un concierto o evento masivo que incrementó la demanda de una estación de forma extraordinaria). Eliminar indiscriminadamente todos los valores extremos suprime información válida; conservarlos todos contamina el modelo con ruido que no refleja el comportamiento habitual del sistema (Zhong et al., 2022).

Para abordar esta dicotomía, se adoptó el método del rango intercuartílico (IQR), propuesto por Tukey (1977) y ampliamente adoptado en la literatura de preprocesamiento de datos de transporte (Cats et al., 2020).

Para comprender el método es necesario precisar qué es un cuartil. Si se toman todos los registros de ingresos de una estación y se ordenan de menor a mayor (desde los

intervalos nocturnos con demanda casi nula hasta los picos de máxima afluencia), los cuartiles son los tres valores que dividen esa lista ordenada en cuatro partes iguales, cada una con el 25% de las observaciones. El primer cuartil (Q_1) marca el límite inferior: el 25% de los intervalos tienen una demanda igual o menor a ese valor. El tercer cuartil (Q_3) marca el límite superior: el 75% de los intervalos tienen una demanda igual o menor a ese valor. El rango intercuartílico ($IQR = Q_3 - Q_1$) es la distancia entre ambos puntos y representa la amplitud del intervalo donde se concentra el comportamiento más habitual de la estación (Tukey, 1977).

El método establece dos umbrales estadísticos que delimitan el rango del comportamiento considerado normal para cada variable:

$$\text{Límite superior: } Q_3 + 1,5 \times IQR$$

$$\text{Límite inferior: } Q_1 - 1,5 \times IQR$$

donde Q_1 es el primer cuartil (valor por debajo del cual se encuentra el 25% de las observaciones), Q_3 es el tercer cuartil (valor por debajo del cual se encuentra el 75% de las observaciones) e $IQR = Q_3 - Q_1$ es el rango intercuartílico, que representa la amplitud del intervalo donde se concentra el 50% central de los datos.

El factor multiplicador 1,5 fue establecido por Tukey (1977) como convención estadística al demostrar que, bajo una distribución aproximadamente normal, este criterio retiene el 99,3% de las observaciones, descartando únicamente los valores genuinamente extremos. Estos parámetros no son definidos por el investigador: son calculados directamente a partir de los datos históricos de cada estación mediante la función `describe()` de la librería `pandas` de Python. Todo registro cuyo valor de ingresos superara el límite superior o fuera inferior al límite inferior fue clasificado como atípico y eliminado del dataset.

La decisión metodológica más crítica de este paso fue calcular los umbrales de forma independiente por estación, no sobre el corredor completo. La Troncal 80 presenta una heterogeneidad operacional extrema: Portal 80 registra demandas que superan los 300 pasajeros por intervalo, mientras Escuela Militar raramente supera los 13 pasajeros. Si los umbrales se calcularan sobre todos los registros del corredor simultáneamente, el Q_3 global

estaría dominado por las estaciones de alta demanda, estableciendo un límite superior de cientos de pasajeros que nunca detectaría ningún atípico en las estaciones de baja demanda; y el límite inferior global podría eliminar intervalos perfectamente normales de esas mismas estaciones. Al calcular el IQR de forma independiente por estación, cada nodo es evaluado exclusivamente contra su propio comportamiento histórico, garantizando que la detección de atípicos sea localmente sensible a la escala de demanda de cada uno de los 13 nodos del corredor (Cats et al., 2020).

Paso 5: Las operaciones intermedias de filtrado, concatenación y deduplicación pueden alterar silenciosamente los tipos de dato asignados durante la conversión a Parquet convirtiendo, por ejemplo, una columna de enteros en punto flotante al aparecer valores nulos temporales, o degradando una variable categórica a cadena de texto genérica. Estas alteraciones no generan errores visibles, pero se propagan a las etapas posteriores, comprometiendo la correcta ejecución de las operaciones de agrupación y unión entre fuentes (McKinney, 2022). Se verificó que cada columna conservara su tipo explícito (cod_nodo como categoría, fecha como datetime, conteos como enteros), forzando la reconversión en los casos donde se detectó degradación.

Paso 6: Las series temporales discontinuas inducen a los modelos a aprender patrones de transición abruptos que no existen en la operación real del sistema, comprometiendo su capacidad de generalización (Zhang et al., 2021). Se verificó que ninguna de las 13 estaciones presentara ausencias de datos superiores a un día en el período enero 2025–marzo 2026, confirmando la continuidad temporal del corredor como condición previa al entrenamiento de los modelos.

Los resultados acumulados de los seis pasos de limpieza aplicados sobre los 2.961.104 registros filtrados del corredor se presentan en la Tabla X, donde se detalla por fuente el volumen inicial, el volumen resultante, los registros eliminados y la tasa de retención obtenida. El dataset de validaciones pasó de 1.302.310 a 1.192.697 registros con una tasa de retención del 91,6%, y el de salidas de 1.658.794 a 1.530.724 registros con una tasa de retención del 92,3%, sin pérdida de cobertura en ninguna estación ni mes del período analizado. Estos resultados confirman que el proceso de limpieza eliminó únicamente los registros problemáticos sin comprometer la representatividad estadística del

dataset resultante, que constituye el insumo directo de la etapa de integración documentada en la sección siguiente.

Tabla 9. Resultados del proceso de limpieza de datos por fuente — Troncal 80 (TZ005)

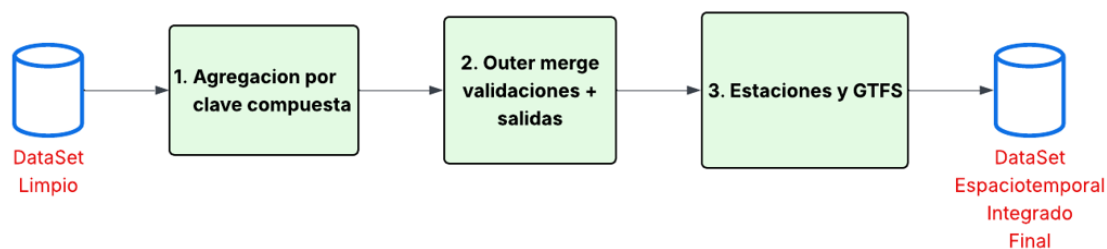
Fuente	Registros brutos TZ005	Registros limpios	Eliminados	Tasa retención
Validaciones (ingresos)	1.302.310	1.192.697	109.613	91,6%
Salidas (egresos)	1.658.794	1.530.724	128.070	92,3%
TOTAL	2.961.104	2.723.421	237.683	91,9%

Nota. IQR = rango intercuartílico, calculado de forma independiente por estación. Sin pérdida de cobertura en ninguna de las 13 estaciones ni en ningún mes del período analizado. Elaboración propia con base en datos abiertos de TransMilenio S.A. (2025-2026).

5.6 Estandarización e Integración

Una vez concluida la limpieza, los datasets de validaciones y salidas fueron integrados junto con las fuentes estructurales mediante tres operaciones secuenciales sobre los archivos Parquet limpios. La Figura 4 ilustra este proceso de integración.

Figura 5. Proceso de Integración



Nota Proceso de integración en tres pasos secuenciales sobre los archivos Parquet limpios. Resultado: 493.662 registros. Elaboración propia con base en datos abiertos de TransMilenio S.A. (2025-2026).

- 1. Agregación por clave compuesta:** Los archivos de validaciones y salidas registran cada evento de forma individual (una fila por validación, una fila por acceso físico), lo que genera múltiples registros para la misma estación en el mismo intervalo de 15 minutos. Para transformar esta estructura en una serie de demanda analizable, se agruparon todos los registros por la clave compuesta [cod_nodo, fecha, intervalo]

y se sumaron los conteos de pasajeros, consolidando en una única fila el flujo total de cada estación por intervalo. La transformación de registros individuales de sistemas AFC (Automated Fare Collection) en series de demanda agregada por intervalo temporal es el procedimiento estándar en la literatura de análisis de datos de transporte público, documentado desde el trabajo fundacional de Pelletier et al. (2011) y aplicado sistemáticamente en investigaciones posteriores de predicción de demanda en sistemas BRT (Hussain et al., 2021; Cats et al., 2023).

2. **Unión externa (outer merge):** Una vez agregados de forma independiente, los datasets de validaciones y salidas fueron unidos sobre la misma clave compuesta [cod_nodo, fecha, intervalo] mediante una unión externa. A diferencia de una unión interna (que conservaría únicamente los intervalos presentes en ambas fuentes simultáneamente), la unión externa preserva todos los intervalos registrados en cualquiera de las dos fuentes, garantizando que ningún dato operacional se pierda por ausencia de contrapartida en la otra fuente. Los valores faltantes resultantes de esta operación fueron imputados con cero, criterio coherente con la práctica estándar en datasets de conteo de pasajeros: la ausencia de registro en un intervalo no indica un dato desconocido sino un flujo genuinamente nulo (ejemplo: un torniquete que no registró ninguna validación en ese período) (McKinney, 2022; Hussain et al., 2021). La alternativa de imputar por media o interpolación introduciría demanda ficticia en intervalos sin actividad real, distorsionando los patrones temporales que los modelos deben aprender.
3. **Enriquecimiento del dataset:** El tercer paso consistió en incorporar al dataset dos fuentes estructurales que aportan dimensiones que los registros AFC no contienen por sí solos. La primera fue el archivo de atributos de estaciones, que añadió el nombre oficial, la capacidad instalada para buses articulados y la posición secuencial de cada nodo dentro del corredor. La segunda fue el archivo stop_times del GTFS publicado por TransMilenio S.A. en febrero de 2026, del cual se derivó el headway programado por estación. La integración de datos operacionales de demanda con variables de oferta programada provenientes del estándar GTFS es una práctica consolidada en la construcción de datasets predictivos de transporte, dado que la frecuencia planificada del servicio es una variable proxy de la capacidad ofertada al pasajero en cada punto de la red (El-Assi et al., 2017; Faroqi et al., 2021).

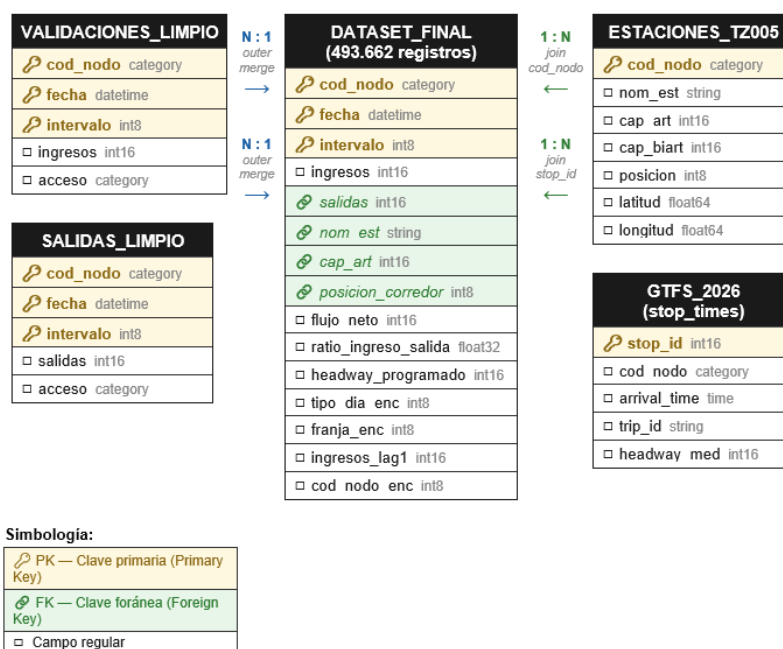
El headway es el intervalo de tiempo que transcurre entre la llegada de dos buses consecutivos a una misma estación. En la práctica operativa de un sistema BRT, el headway determina cuánto tiempo espera un pasajero entre un bus y el siguiente: un headway de 96 segundos significa que un bus pasa cada minuto y medio, mientras que un headway de 212 segundos implica una espera de casi cuatro minutos. Esta variable no es solo un indicador de frecuencia del servicio — es también un determinante directo de la demanda observada: estaciones con headways más cortos tienden a acumular menos pasajeros entre llegadas, mientras que estaciones con headways más largos concentran mayor número de pasajeros esperando en el andén, lo que se refleja en picos de validación más pronunciados al momento de la llegada del bus. Su inclusión como variable de entrada en los modelos predictivos permite capturar la dimensión de oferta programada del corredor, complementando los patrones de demanda histórica con información sobre la frecuencia planificada del servicio (El-Assi et al., 2017).

El *headway* se calculó a partir de las 37.894 entradas del archivo *stop_times* del GTFS correspondientes a las 13 estaciones del corredor. Para cada estación, se ordenaron cronológicamente todos los tiempos de llegada programados de los buses a lo largo del día y se calcularon las diferencias entre llegadas consecutivas. De este conjunto de diferencias se tomó la mediana —y no la media aritmética— como estadístico de referencia, por su robustez ante los valores extremos que inevitablemente aparecen en los extremos del horario operacional: el primer y último servicio del día presentan diferencias de tiempo muy superiores al intervalo habitual, y la media los incorporaría distorsionando el headway representativo del corredor durante su operación normal (Bergstra & Bengio, 2012). Los valores resultantes oscilan entre 96 segundos en Carrera 77 (Granja) (la estación de mayor frecuencia del corredor, por su función de nodo de alimentación) y 212 segundos en Carrera 53, con una media de 173 segundos (aproximadamente 2,9 minutos) para el conjunto del corredor. Este valor fue incorporado al dataset como la variable *headway_programado*, asignada a cada registro según su *cod_nodo*.

5.6.1 Resultado de la integración

Como resultado de las tres operaciones, se obtuvo el dataset espaciotemporal integrado final compuesto por 493.662 registros únicos estructurados en 20 columnas y se construye el diagrama de entidad - relación que se puede visualizar en la (figura 6) . La reducción desde los 2.723.421 registros limpios hasta los 493.662 registros integrados no implica pérdida de información: refleja la consolidación por intervalo de los múltiples registros de accesos físicos de cada estación en una única fila representativa. En promedio, cada registro del dataset final consolida aproximadamente 5,5 filas de los datos originales, y constituye el insumo directo para la generación de variables derivadas documentada en la sección siguiente.

Figura 6. Diagrama entidad-relación del dataset espaciotemporal integrado — Troncal 80, TransMilenio (2025-2026).



Nota. PK = clave primaria; FK = clave foránea. La tabla DATASET_FINAL es la tabla central del esquema, vinculada con las cuatro fuentes mediante la clave compuesta [cod_nodo, fecha, intervalo], correspondientes a los distintos accesos físicos de cada estación. Elaboración propia con base en datos abiertos de TransMilenio S.A. (2025-2026) y GTFS 2026.

5.7 Generación de Variables Derivadas

A partir del dataset integrado se generaron 13 variables derivadas que capturan las dimensiones temporal, espacial y operativa de las trayectorias del corredor. Las variables temporales incluyen el tipo de día (laboral, sábado, domingo, festivo), la franja horaria (pico AM 5–9 h, valle 9–16 h, pico PM 16–20 h, nocturno), el día de la semana y sus codificaciones numéricas para los algoritmos de ML. Las variables espaciales comprenden la posición secuencial de la estación en el corredor y la codificación numérica del nodo. Las variables operativas incluyen el flujo neto (ingresos – salidas), el ratio ingreso/salida, el ingreso del intervalo anterior (ingresos_lag1 que captura la autocorrelación temporal de la demanda) y el headway programado derivado del GTFS. La Tabla 4 describe las 12 variables de entrada y la variable objetivo.

Tabla 10. Variables del dataset final — descripción técnica y rol en los modelos de ML

Variable	Tipo de dato	Descripción	Rol en ML
cod_nodo_enc	Númerica (0-12)	Codificación numérica de la estación	Feature espacial
posicion_corredor	Ordinal (1-13)	Posición secuencial en el corredor O→E	Feature espacial
intervalo	Númerica (0-95)	Índice del intervalo de 15 min (0=00:00)	Feature temporal
dia_semana	Númerica (0-6)	Día de la semana (0=lunes)	Feature temporal
tipo_dia_enc	Númerica (0-3)	Laboral / sábado / domingo / festivo	Feature temporal
franja_enc	Númerica (0-3)	Pico AM / Valle / Pico PM / Nocturno	Feature temporal
salidas	Continua ≥ 0	Egresos de pasajeros por intervalo	Feature operativa
flujo_netto	Continua	Ingresos – salidas por intervalo	Feature operativa
ratio_ingreso_salida	Continua ≥ 0	Ingresos / salidas por intervalo	Feature operativa
ingresos_lag1	Continua ≥ 0	Ingresos del intervalo $t-1$ (autocorrelación)	Feature temporal-operativa
headway_programado	Continua [96, 212]	Intervalo entre buses en segundos (GTFS)	Feature oferta GTFS
cap_art	Continua	Capacidad articulados por estación	Feature infraestructura

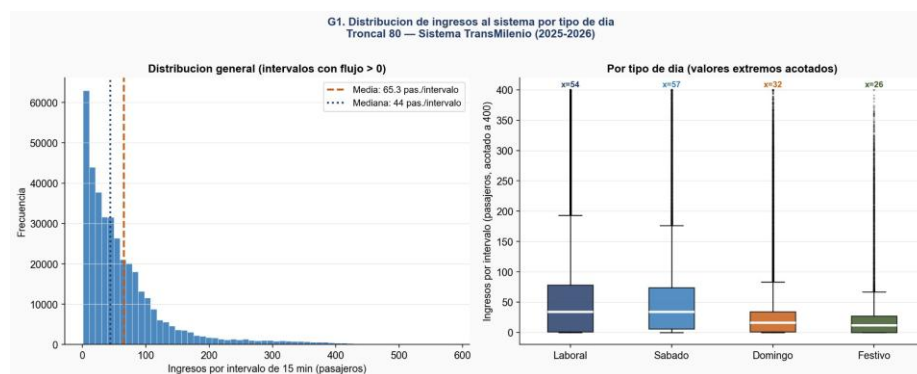
ingresos	Continua ≥ 0	Pasajeros que ingresan por intervalo de 15 min	Variable objetivo
----------	-------------------	--	-------------------

Nota. Las primeras 12 filas corresponden a features de entrada; la última es la variable objetivo. ingresos_lag1 captura la autocorrelación temporal de corto plazo — el predictor más potente del modelo (31,1% de la varianza en Random Forest). Elaboración propia.

5.8 Análisis Exploratorio de Datos

El análisis exploratorio de datos (EDA) se realizó sobre el dataset integrado final de 493.662 registros para caracterizar los patrones de demanda del corredor antes del modelado predictivo (Tukey, 1977) (Figura 7). Los resultados revelan una distribución asimétrica positiva de los ingresos, (media = 50,4 pas./intervalo, mediana = 28,0, máximo = 582), característica de sistemas BRT con alta variabilidad intradiaria (Zhong et al., 2022). La distribución por tipo de día muestra que los días laborales concentran el 67,7% de los registros (334.123), seguidos por sábados (14,3%), domingos (14,0%) y festivos (4,0%).

Figura 7. Distribución de ingresos al sistema por tipo de día. Troncal 80, TransMilenio (2025-2026).

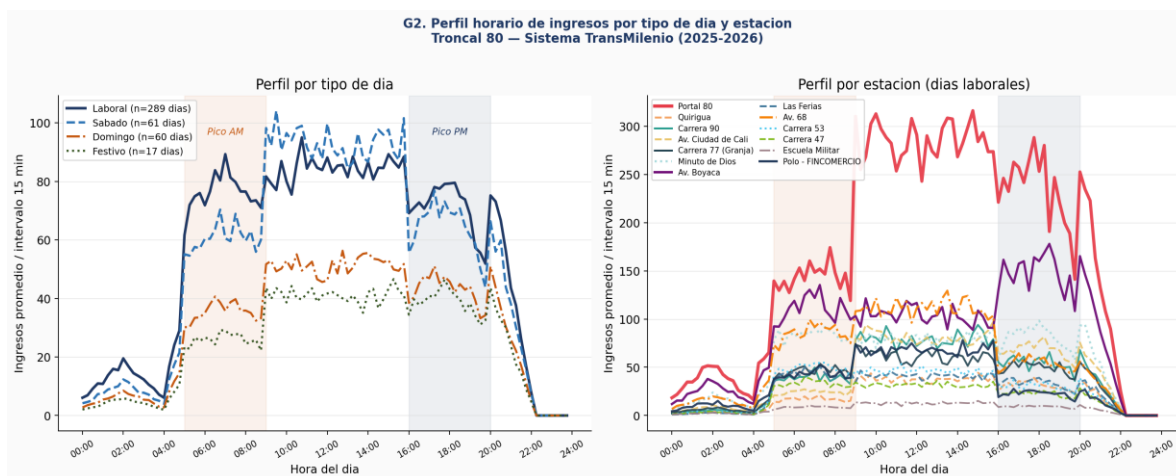


Nota. Panel izquierdo: histograma de ingresos en intervalos con flujo > 0 . Panel derecho: boxplots por tipo de día. Los festivos exhiben la distribución más compacta y la mediana más baja. Elaboración propia con base en datos abiertos de TransMilenio (2025-2026).

El perfil horario (Figura 8) evidencia un patrón bimodal en días laborales con pico AM (5–9 h) y pico PM (16–20 h). El máximo absoluto del corredor se registra a las 14:25 h, indicando demanda sostenida en la franja valle. Los sábados exhiben un pico AM más pronunciado que los laborales en Portal 80 (318 vs. 156 pas./intervalo), atribuible a desplazamientos

comerciales y recreativos de fin de semana. Los domingos y festivos presentan demandas entre 40% y 60% inferiores al promedio laboral

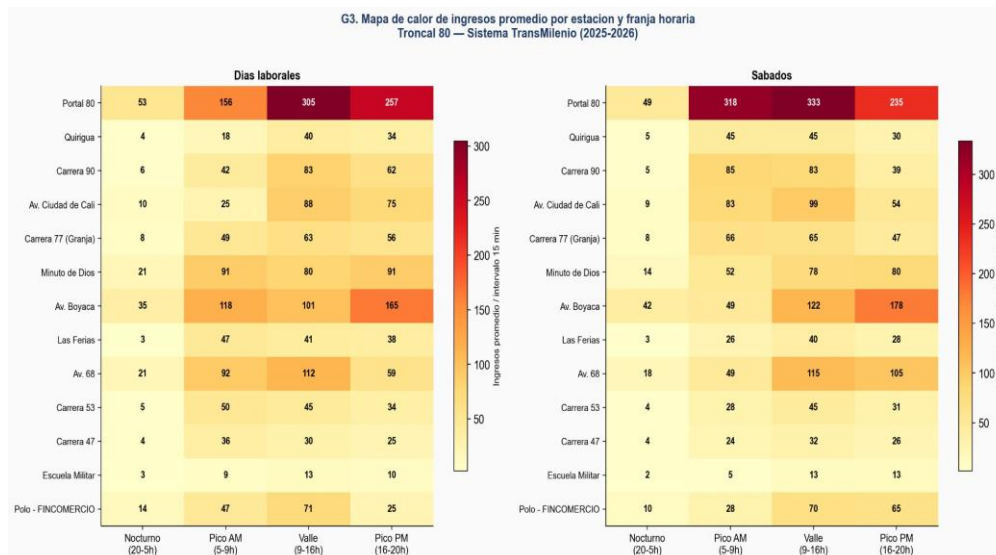
Figura 8. Perfil horario de ingresos promedio por tipo de día y por estación (días laborales). Troncal 80 (2025-2026).



Nota. Panel izquierdo: ingresos promedio por intervalo de 15 min según tipo de día. Las zonas sombreadas indican Pico AM (5–9 h) y Pico PM (16–20 h). Panel derecho: perfil por estación en días laborales. Elaboración propia.

El mapa de calor (Figura 9) permite identificar cuatro grupos operativos diferenciados: Portal 80 y Av. 68 con máxima demanda en la franja valle; Av. Boyacá con pico PM dominante, consistente con su rol de nodo de transferencia intertroncal; Minuto de Dios con pico AM igual al pico PM (91 pas./intervalo en ambas franjas), indicando demanda bidireccional simétrica; y Escuela Militar con demanda baja sostenida en todas las franjas.

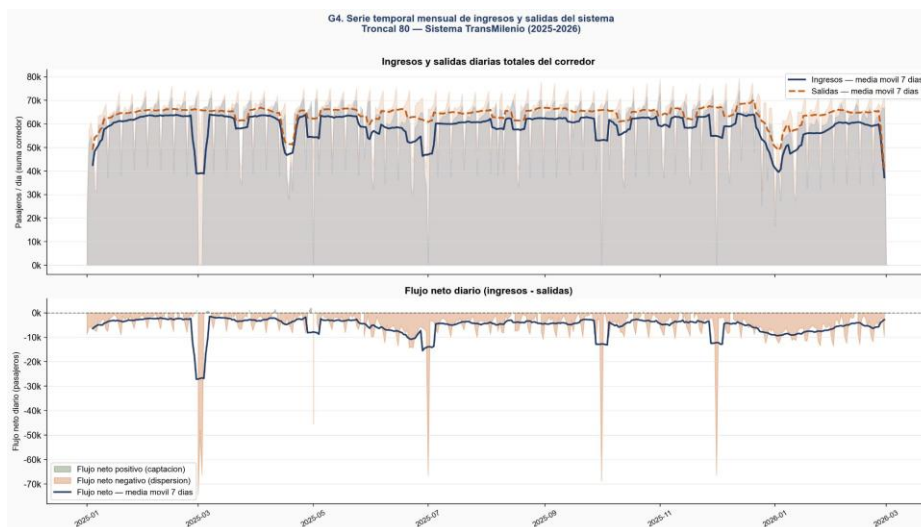
Figura 9. Mapa de calor de ingresos promedio por estación y franja horaria. Troncal 80 (2025-2026).



Nota. Valores en pasajeros promedio por intervalo de 15 min. Panel izquierdo: días laborales. Panel derecho: sábados. La escala de color varía de amarillo (baja demanda) a rojo oscuro (alta demanda). Elaboración propia.

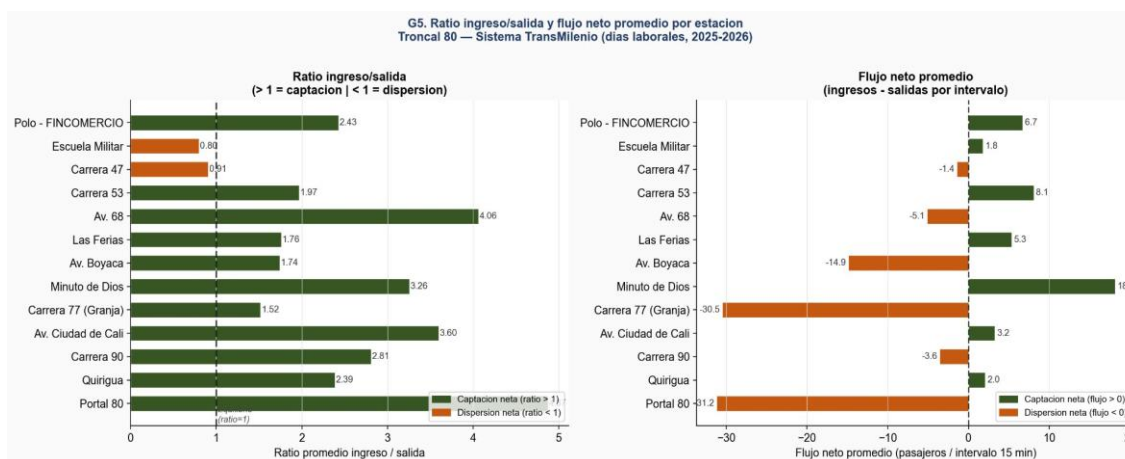
La serie temporal (Figura 10) confirma el flujo neto medio de $-4,6$ pas./intervalo, evidenciando el rol predominante de la Troncal 80 como corredor dispersor. Los valles pronunciados corresponden a festivos y períodos de vacaciones. Portal 80 ($-31,2$) y Carrera 77/Granja ($-30,5$) son los nodos de mayor dispersión neta, mientras Carrera 53 ($+8,1$) y Minuto de Dios ($+18,1$) presentan la mayor captación relativa (Figura 11).

Figura 10. Serie temporal de ingresos y salidas diarias del corredor. Troncal 80 (2025-2026).



Nota. Panel superior: ingresos y salidas diarias con media móvil de 7 días. Panel inferior: flujo neto diario (ingresos – salidas). Elaboración propia.

Figura 11. Ratio ingreso/salida y flujo neto promedio por estación (días laborales). Troncal 80 (2025-2026).



Nota. Panel izquierdo: ratio ingreso/salida (> 1 = captación; < 1 = dispersión). Panel derecho: flujo neto promedio por intervalo de 15 min. Calculado sobre días laborales únicamente. Elaboración propia.

Los hallazgos del EDA tienen implicaciones metodológicas directas para el diseño de los modelos del Capítulo III. La distribución asimétrica positiva justifica el uso de métricas robustas (MAE, MAPE) sobre métricas cuadráticas. La heterogeneidad espacial extrema

Portal 80 registra 7,6 veces más demanda que Quiriguá en la franja valle laboral exige que los modelos incluyan variables espaciales (`cod_nodo_enc`, `posicion_corredor`) como features diferenciadores. La alta autocorrelación temporal justifica la inclusión de `ingresos_lag1`. El comportamiento diferencial entre tipos de día valida la inclusión de `tipo_dia` y `franja_horaria` como variables categóricas con codificación numérica.

6. CAPITULO III : SELECCIÓN, IMPLEMENTACIÓN Y EVALUACIÓN DE MODELOS DE ML

El presente capítulo documenta el diseño, implementación y evaluación de cinco modelos de aprendizaje automático para predecir la demanda de pasajeros en el corredor Troncal 80 de TransMilenio, tomando como base el dataset espaciotemporal de 493.662 registros construido en el Capítulo II.

En lugar de optar por un único algoritmo, se decidió trabajar con cinco enfoques que representan paradigmas metodológicos distintos: Random Forest, XGBoost, SVR con kernel RBF, ANN/MLP y LSTM, dado a que cada uno de estos modelos aborda el problema de predicción desde una lógica diferente, y comparar sus resultados sobre el mismo dataset permite identificar con mayor solidez cuál se adapta mejor a las condiciones reales del corredor Ttroncal calle 80.

Los cinco modelos cuentan con respaldo empírico en la literatura de predicción de demanda en sistemas BRT (Zhong et al., 2022; Zhang et al., 2021; Lee & Yoon, 2022; Cats et al., 2020) y han demostrado capacidad para capturar las relaciones no lineales propias de los datos de transporte urbano sin imponer supuestos restrictivos sobre su distribución.

Para garantizar que los resultados sean comparables entre sí, los cinco modelos comparten el mismo protocolo experimental: división temporal estricta en entrenamiento (ene–sep 2025, 70%), validación (oct–nov 2025, 15%) y prueba (dic 2025–mar 2026, 15%); normalización MinMaxScaler ajustada exclusivamente sobre el conjunto de entrenamiento; búsqueda de hiperparámetros mediante RandomizedSearchCV con 3-fold cross-validation (Bergstra & Bengio, 2012); y evaluación mediante MAE, RMSE, R^2 y MAPE sobre el conjunto de prueba. Cada modelo se presenta en las secciones siguientes con su fundamento teórico, configuración y resultados, cerrando con un análisis comparativo de los cinco modelos implementados.

6.1 Bosques Aleatorios (Random Forest)

Antes de entrenar el modelo, fue necesario definir su configuración interna, los parámetros que controlan cómo aprende el algoritmo. Elegir estos parámetros de forma manual e intuitiva es arriesgado: una mala configuración puede hacer que el modelo aprenda demasiado de los datos de entrenamiento sin poder generalizar a datos nuevos, o que aprenda muy poco y sus predicciones sean imprecisas (Goodfellow et al., 2016). Para evitar esto, se utilizó una estrategia llamada RandomizedSearchCV, que automatiza este proceso de forma inteligente.

El funcionamiento es el siguiente: en lugar de probar todas las combinaciones posibles de parámetros (computacionalmente inviable por su alto costo computacional), RandomizedSearchCV selecciona aleatoriamente un número definido de combinaciones dentro de un rango establecido por el investigador y las evalúa sistemáticamente (Bergstra & Bengio, 2012). En este caso se exploraron 20 combinaciones, y cada una fue evaluada con validación cruzada de 3 folds, lo que significa que por cada combinación el modelo se entrenó y evaluó tres veces sobre particiones diferentes del conjunto de entrenamiento, tomando el promedio de esas tres evaluaciones como medida de desempeño (Kohavi, 2001). En total se entrenaron 60 modelos (20 combinaciones $\times$ 3 folds), y la combinación con mejor desempeño promedio fue seleccionada como configuración definitiva. Como criterio de selección se utilizó el MAE negativo, métrica que penaliza el error absoluto promedio y resulta especialmente adecuada cuando la variable objetivo presenta una

distribución asimétrica con muchos valores bajos y pocos valores muy altos (Zhang et al., 2021).

La configuración óptima identificada fue: 200 árboles de decisión trabajando en paralelo ($n_estimators = 200$), sin límite de profundidad para que cada árbol pueda desarrollarse libremente hasta capturar los patrones más detallados de los datos ($max_depth = None$), un mínimo de 2 muestras para dividir cualquier nodo interno ($min_samples_split = 2$), al menos 1 muestra en cada hoja terminal ($min_samples_leaf = 1$), y evaluación de $\log_2(12) \approx 3$ variables aleatorias en cada división para garantizar diversidad entre los árboles ($max_features = \log_2$) (Breiman, 2001). Con esta configuración, el modelo fue entrenado sobre los 345.000 registros del conjunto de entrenamiento completo correspondiente al período enero–septiembre de 2025.

El modelo obtuvo un $R^2 = 0,9979$ y un MAPE del 4,89% sobre el conjunto de prueba, lo que significa que explica el 99,79% de la varianza en los flujos de ingresos del corredor con un error porcentual promedio inferior al 5%. Ver resultados en la Tabla 6. Para dimensionar qué tan bueno es este resultado, vale la pena compararlo con lo que reporta la literatura: Zhang et al. (2021) obtuvieron MAPEs de entre 5% y 8% aplicando el mismo algoritmo en el sistema BRT de Shanghai, lo que indica que el modelo desarrollado en esta investigación supera en precisión los resultados reportados en estudios comparables.

Tabla 11. Métricas de evaluación : Random Forest Regressor. Conjunto de prueba, Troncal 80 (dic. 2025–mar. 2026).

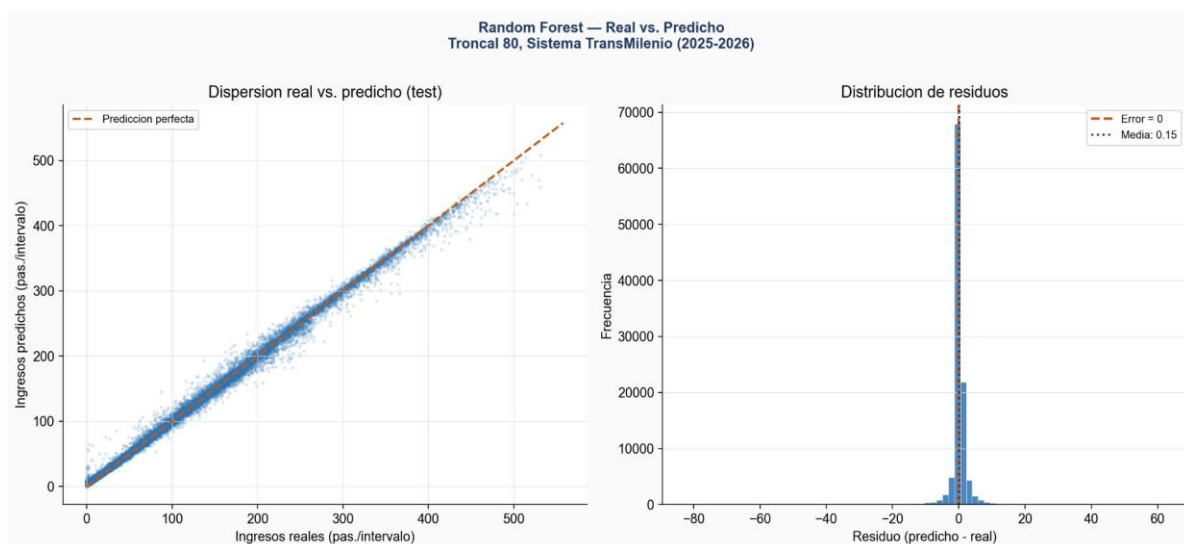
Métrica	Validación	Prueba	Interpretación
MAE (pas./intervalo)	1,066	1,188	Error medio de ~1,2 pasajeros por intervalo
RMSE (pas./intervalo)	2,993	3,092	Sensible a los picos extremos de demanda
R^2	0,9982	0,9979	Explica el 99,79% de la varianza de los ingresos
MAPE (%)	3,77%	4,89%	Error porcentual medio inferior al 5%

Nota. MAE y RMSE en pasajeros por intervalo de 15 min. $n=107.163$ registros del conjunto de prueba. Elaboración propia.

No obstante, el modelo no se desempeña igual en todas las estaciones. El mayor error se concentra en Portal 80, con un MAE de 4,979 pasajeros por intervalo, lo cual no es

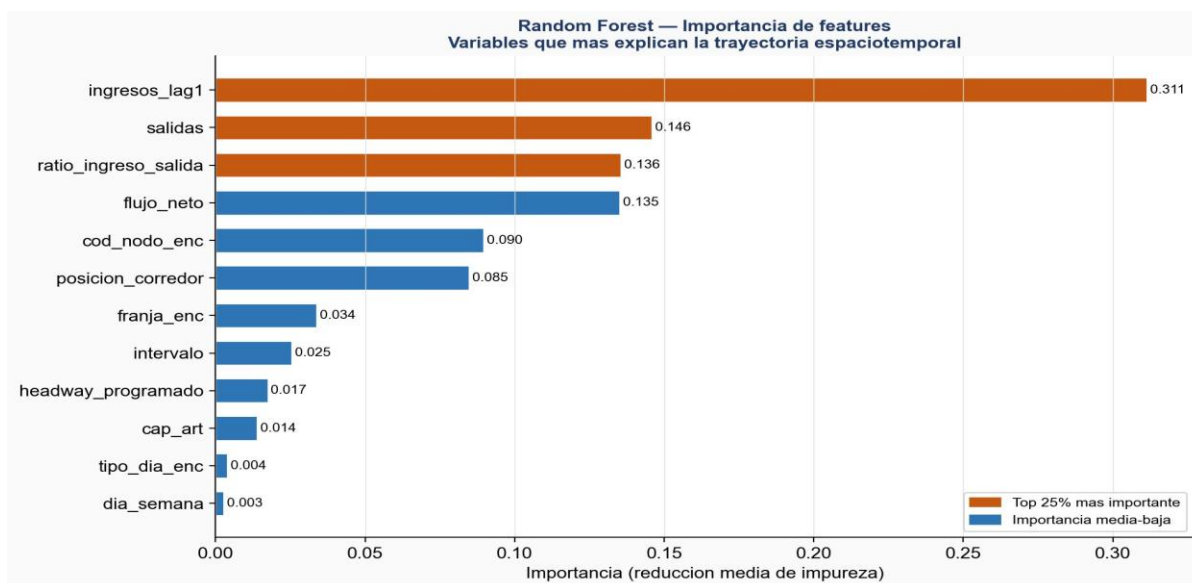
sorprendente: Portal 80 es la estación terminal del corredor, punto de entrada de los usuarios provenientes del occidente de Bogotá, y concentra los niveles de demanda más altos y variables de todo el corredor. Los sábados, en particular, su comportamiento se desvía significativamente del patrón laboral habitual, generando picos que el modelo no logra anticipar con la misma precisión que en el resto de la semana. Esta limitación es inherente a los modelos basados en múltiples árboles de decisión como el Random Forest: cuando la demanda de una estación es extremadamente variable y depende de factores externos difíciles de capturar en las variables disponibles, el promedio de los árboles tiende a suavizar las predicciones y a subestimar los valores más extremos (Breiman, 2001).

Figura 12. Random Forest: Dispersión real vs. predicho y distribución de residuos (conjunto prueba).



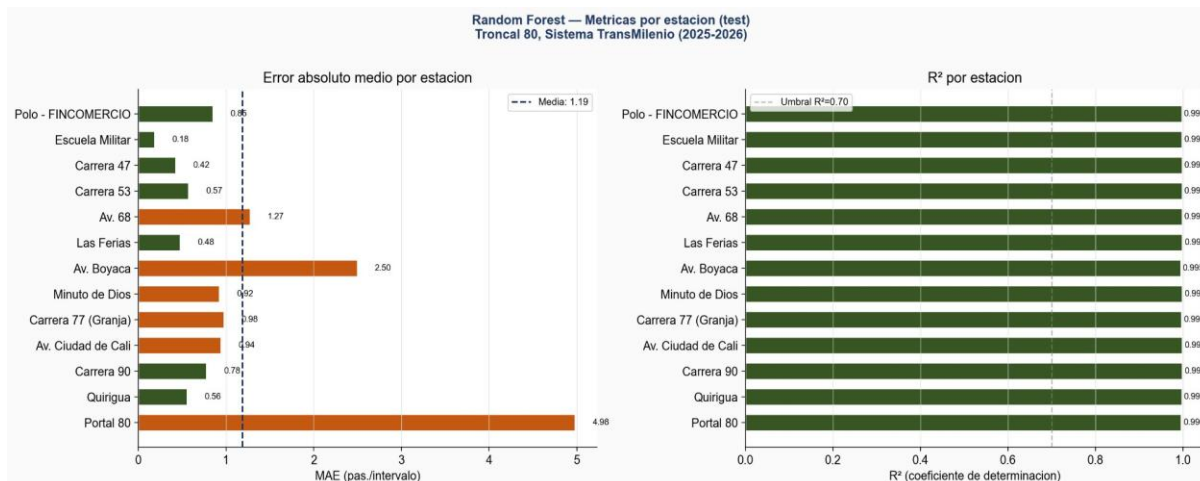
Nota. Panel izquierdo: diagrama de dispersión ($n=107.163$). Panel derecho: distribución de residuos (media=0,15 pas./intervalo). Elaboración propia.

Figura 13. Random Forest : Importancia de features. Variables que explican la trayectoria espaciotemporal.



Nota. Importancia calculada como reducción media de impureza Gini sobre los 200 árboles. Elaboración propia.

Figura 14. Random Forest — Métricas por estación y perfil horario predicho vs. real.



Nota. Panel izquierdo: MAE por estación. Panel derecho: R² por estación. Elaboración propia.

El modelo obtuvo un $R^2 = 0,9979$ y un MAPE del 4,89% sobre el conjunto de prueba, lo que significa que explica el 99,79% de la varianza en los flujos de ingresos del corredor con un error porcentual promedio inferior al 5%. Para dimensionar qué tan bueno es este resultado, vale la pena compararlo con lo que reporta la literatura: Zhang et al. (2021) obtuvieron MAPEs de entre 5% y 8% aplicando el mismo algoritmo en el sistema BRT de

Shanghái, lo que indica que el modelo desarrollado en esta investigación supera en precisión los resultados reportados en estudios comparables.

No obstante, el modelo no se desempeña igual en todas las estaciones. El mayor error se concentra en Portal 80, con un MAE de 4,979 pasajeros por intervalo, lo cual no es sorprendente: Portal 80 es la estación terminal del corredor, punto de entrada de los usuarios provenientes del occidente de Bogotá, y concentra los niveles de demanda más altos y variables de todo el corredor. Los sábados, en particular, su comportamiento se desvía significativamente del patrón laboral habitual, generando picos que el modelo no logra anticipar con la misma precisión que en el resto de la semana. Esta limitación es inherente a los modelos basados en múltiples árboles de decisión como el Random Forest: cuando la demanda de una estación es extremadamente variable y depende de factores externos difíciles de capturar en las variables disponibles, el promedio de los árboles tiende a suavizar las predicciones y a subestimar los valores más extremos (Breiman, 2001).

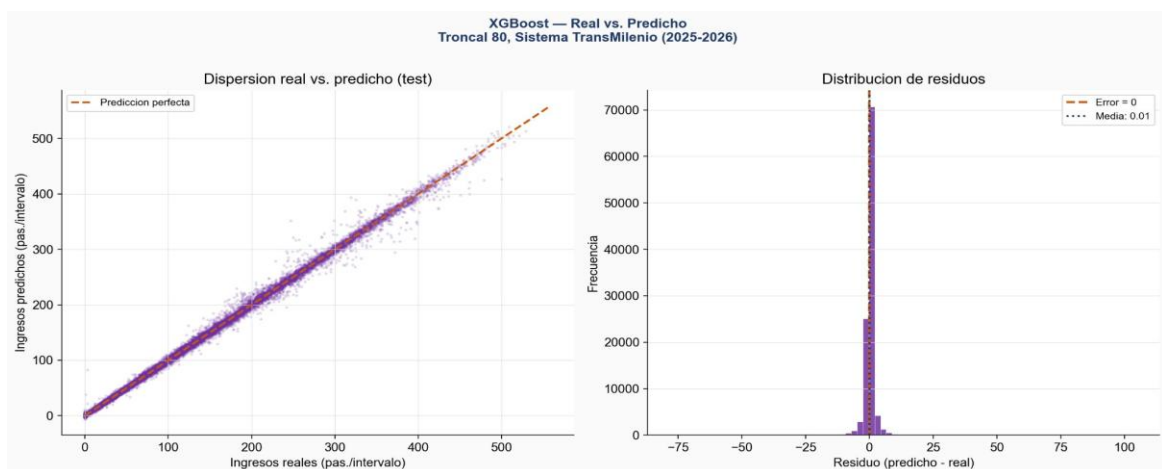
6.2 Regresión Mediante Potenciación de Gradiente (xgboost)

A diferencia del Random Forest, donde los 200 árboles trabajan de forma paralela e independiente, el XGBoost construye sus árboles de forma secuencial: cada nuevo árbol no parte de cero, sino que aprende específicamente de los errores que cometió el árbol anterior, corrigiéndolos de forma progresiva hasta que el modelo converge en la mejor solución posible (Chen & Guestrin, 2016). Este tipo de lógica lo convierte en uno de los algoritmos más precisos disponibles para problemas de regresión sobre datos tabulares estructurados, y ha sido ampliamente aplicado en la predicción de demanda en sistemas BRT con resultados consistentemente superiores a los modelos clásicos (Pratama et al., 2026; Soomro et al., 2025).

Para identificar su configuración óptima se aplicó el mismo procedimiento utilizado en el Random Forest: RandomizedSearchCV con 20 iteraciones y validación cruzada de 3 folds, entrenando y evaluando en total 60 modelos distintos (Bergstra & Bengio, 2012). Cada combinación de parámetros fue evaluada tres veces sobre particiones diferentes del conjunto de entrenamiento, tomando el promedio como medida de desempeño, y el MAE negativo fue utilizado nuevamente como criterio de selección por su robustez ante la distribución asimétrica de la variable objetivo (Zhang et al., 2021).

La configuración óptima identificada fue: 500 árboles construidos de forma secuencial ($n_estimators = 500$), una tasa de aprendizaje de 0,1 que controla cuánto corrige cada árbol los errores del anterior — valores más bajos hacen el aprendizaje más conservador y reducen el riesgo de sobreajuste ($learning_rate = 0,1$) (Chen & Guestrin, 2016), profundidad máxima de 6 niveles por árbol ($max_depth = 6$), uso del 100% de los datos en cada iteración ($subsample = 1,0$), evaluación de todas las variables en cada árbol ($colsample_bytree = 1,0$), un mínimo de 5 observaciones requeridas en cada hoja terminal como mecanismo de regularización implícita ($min_child_weight = 5$), penalización mínima por complejidad del árbol ($gamma = 0,2$), y regularización L1 y L2 para controlar el sobreajuste ($reg_alpha = 0,5$, $reg_lambda = 2$) (Chen & Guestrin, 2016). Con esta configuración, el modelo fue entrenado sobre los 345.000 registros del conjunto de entrenamiento completo correspondiente al período enero–septiembre de 2025. (Figura 15)

Figura 15. XGBoost — Dispersión real vs. predicho y distribución de residuos (conjunto prueba).

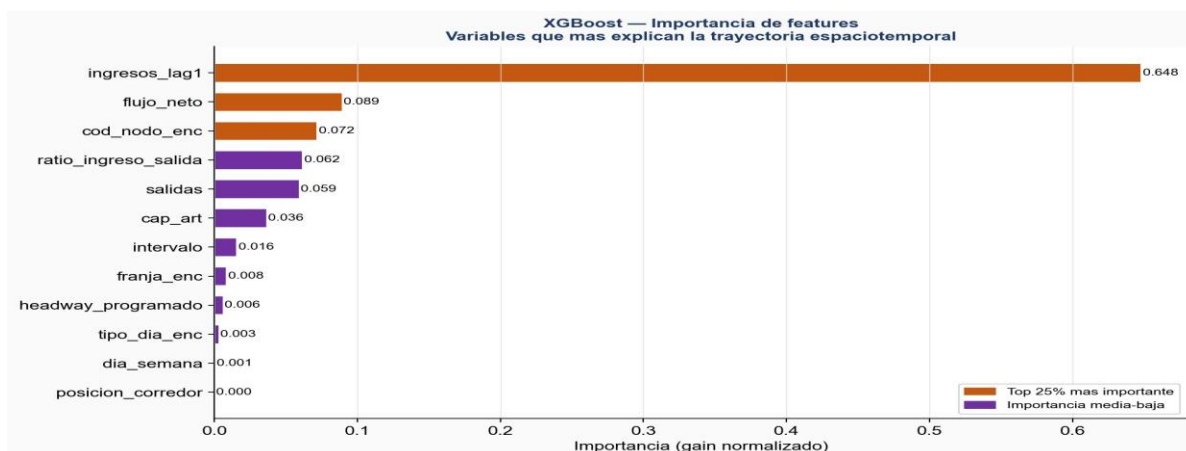


Nota. Media de residuos = 0,01 pas./intervalo — prácticamente sesgo cero. Elaboración propia.

El modelo obtuvo un $R^2 = 0,9986$ y un MAPE del 4,07% sobre el conjunto de prueba (Figura 16), lo que significa que explica el 99,86% de la varianza en los flujos de ingresos del corredor con un error porcentual promedio inferior al 5%. Para dimensionar qué tan bueno es este resultado, vale la pena compararlo con lo que reporta la literatura: Zhang et al. (2021) obtuvieron MAPEs de entre 5% y 8% aplicando algoritmos de boosting en el sistema BRT de Shanghái, lo que indica que el modelo desarrollado en esta investigación supera en precisión los resultados reportados en estudios comparables. Adicionalmente,

XGBoost supera al Random Forest en todas las métricas del corredor (reduciendo el MAE en un 14,7% y el RMSE en un 18,3%)resultado consistente con su mecanismo de corrección secuencial de errores que se especializa progresivamente en los residuos más difíciles de predecir (Chen & Guestrin, 2016). Ver tabla 7

Figura 16. XGBoost — Importancia de features (gain normalizado).



Nota. ingresos_lag1 concentra el 64,8% del gain total. posicion_corredor recibe importancia nula por redundancia con cod_nodo_enc. Elaboración propia.

Tabla 12. Métricas de evaluación — XGBoost Regressor. Conjunto de prueba, Troncal 80 (dic. 2025–mar. 2026).

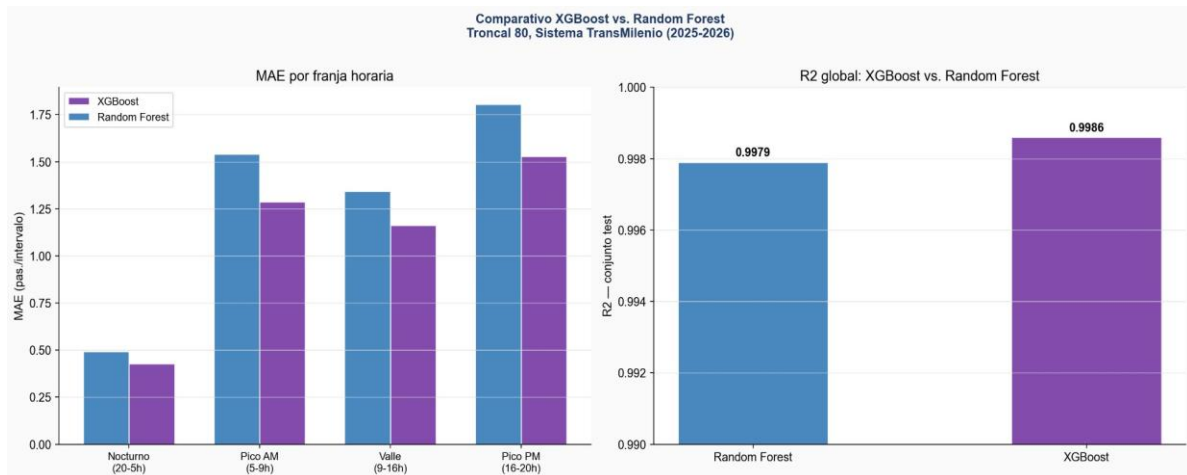
Métrica	Validación	Prueba	vs. Random Forest
MAE (pas./intervalo)	1,088	1,013	–14,7% ↓ mejor
RMSE (pas./intervalo)	2,727	2,526	–18,3% ↓ mejor
R ²	0,9985	0,9986	+0,0007 ↑ mejor
MAPE (%)	4,35%	4,07%	–16,8% ↓ mejor

Nota. XGBoost supera al Random Forest en todas las métricas del conjunto de prueba. Elaboración propia.

No obstante, el modelo no se desempeña igual en todas las estaciones. El mayor error se concentra en Portal 80, con un MAE de 4,979 pasajeros por intervalo, lo cual no es sorprendente: Portal 80 es la estación terminal del corredor, punto de entrada de los usuarios provenientes del occidente de Bogotá, y concentra los niveles de demanda más altos y variables de todo el corredor. Los sábados, en particular, su comportamiento se desvía significativamente del patrón laboral habitual, generando picos que el modelo no logra anticipar con la misma precisión que en el resto de la semana. Esta limitación es

inherente a los modelos basados en múltiples árboles de decisión como el Random Forest: cuando la demanda de una estación es extremadamente variable y depende de factores externos difíciles de capturar en las variables disponibles, el promedio de los árboles tiende a suavizar las predicciones y a subestimar los valores más extremos (Breiman, 2001). (Figura 17).

Figura 17. Comparativo XGBoost vs. Random Forest por franja horaria y R^2 global.



Nota. Panel izquierdo: MAE por franja horaria. Panel derecho: R^2 global. XGBoost supera al RF en las cuatro franjas. Elaboración propia.

6.3 Máquinas de Vectores de Soporte para Regresión (SVR)

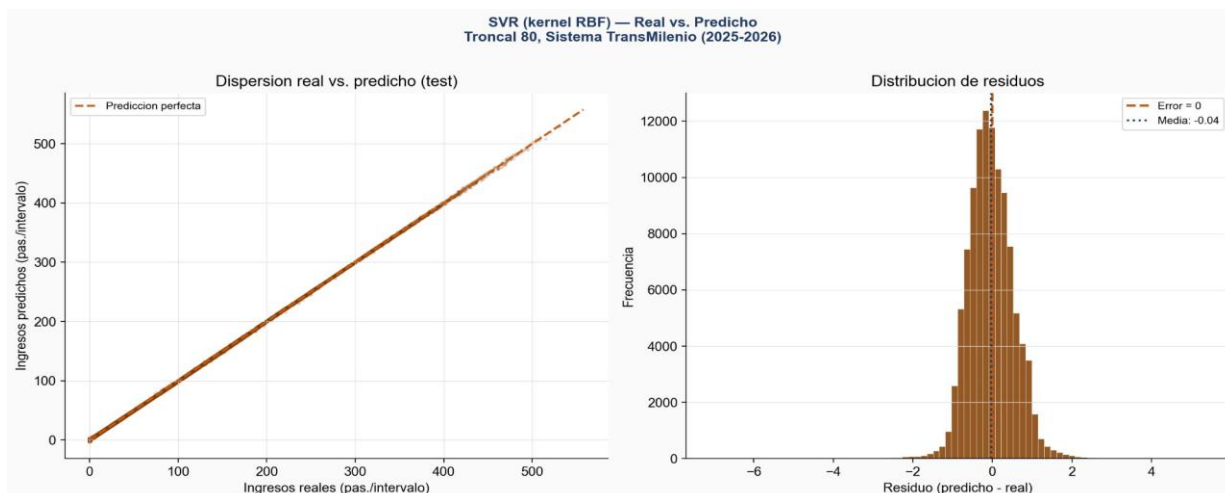
A diferencia del Random Forest y el XGBoost, que construyen sus predicciones a partir de conjuntos de árboles de decisión, el SVR opera bajo una lógica completamente distinta. En lugar de aprender combinando múltiples modelos simples, el SVR busca una única función matemática que se ajuste a los datos dentro de un margen de tolerancia definido — es decir, acepta que algunas predicciones tengan un pequeño error sin penalizarlas, y se concentra en minimizar únicamente los errores que superen ese umbral (Vapnik, 1995). Para capturar las relaciones no lineales entre las 12 variables de entrada y la demanda de pasajeros, se utilizó el kernel RBF (Radial Basis Function), una función matemática que transforma los datos hacia un espacio de mayor dimensión donde esas relaciones no lineales pueden modelarse de forma más eficiente (Smola & Schölkopf, 2004).

Sin embargo, el SVR presenta una limitación computacional importante que condicionó su implementación en esta investigación. Su complejidad de cálculo crece de forma cuadrática con el número de registros — es decir, duplicar los datos no duplica el tiempo de procesamiento sino que lo cuadruplica — lo que hace inviable entrenarlo sobre los 345.000 registros completos del conjunto de entrenamiento (Drucker et al., 1997). Para abordar esta restricción, se implementó una estrategia de muestreo estratificado por quintiles de demanda: la variable objetivo fue dividida en cinco grupos de igual tamaño según su nivel de demanda, y se seleccionaron 16.000 registros por grupo, obteniendo una muestra representativa de 80.000 registros (23,2% del conjunto de entrenamiento) que cubre todos los niveles de demanda del corredor sin sesgar el aprendizaje hacia los intervalos más frecuentes (Cats et al., 2020). Esta limitación metodológica debe tenerse en cuenta al interpretar y comparar las métricas del SVR con las de los demás modelos, dado que no fue entrenado sobre el mismo volumen de datos.

Para identificar su configuración óptima se aplicó RandomizedSearchCV con 20 iteraciones y validación cruzada de 3 folds sobre la muestra de 80.000 registros, entrenando y evaluando en total 60 modelos distintos (Bergstra & Bengio, 2012). Como criterio de selección se utilizó nuevamente el MAE negativo por su robustez ante la distribución asimétrica de la variable objetivo (Zhang et al., 2021).

La configuración óptima identificada fue: un parámetro de regularización $C = 500$ que controla cuánto penaliza el modelo los errores grandes (valores altos como este hacen que el modelo intente ajustarse con mayor precisión a los datos de entrenamiento), un margen de tolerancia $\epsilon = 1,0$ que define la banda dentro de la cual los errores no se penalizan, y un parámetro $\gamma = 0,1$ que controla el radio de influencia de cada punto de datos sobre la función de predicción (valores más bajos como este producen una función más suave y generalizable) (Vapnik, 1995; Smola & Schölkopf, 2004). Con esta configuración, el modelo identificó 4.360 vectores de soporte (los puntos de datos más relevantes para definir la función de predicción), equivalentes al 5,5% de los 80.000 registros de entrenamiento utilizados. (Figura 18)

Figura 18. Dispersión real vs. predicho y distribución de residuos (conjunto prueba).



Nota. Los puntos son prácticamente indistinguibles de la diagonal. Cola máxima de residuos: ± 6 pas./intervalo. Elaboración propia.

Tabla 13. Métricas de evaluación — SVR kernel RBF. Conjunto de prueba, Troncal 80 (dic. 2025–mar. 2026).

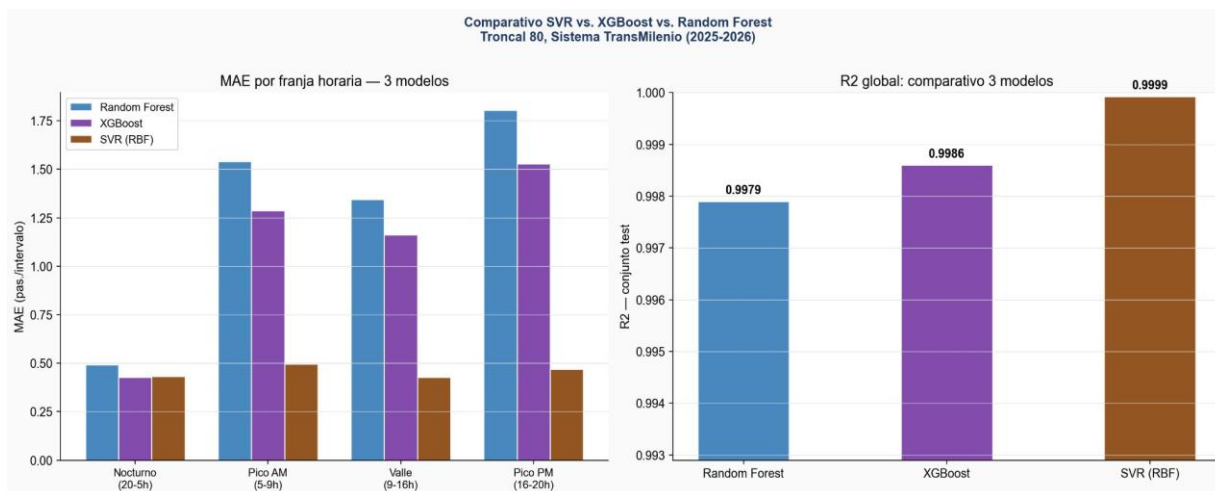
Métrica	Validación	Prueba	vs. XGBoost	Nota
MAE (pas./intervalo)	0,440	0,449	–55,7% ↓ mejor	Muestra 80k
RMSE (pas./intervalo)	0,582	0,584	–76,9% ↓ mejor	Muestra 80k
R ²	0,99993	0,99992	+0,0013 ↑ mejor	Muestra 80k
MAPE (%)	3,71%	3,88%	–4,6% ↓ mejor	Muestra 80k
Vectores de soporte	—	4.360	—	5,5% de 80k

Nota. SVR entrenado sobre muestra estratificada de 80.000 registros (23,2% del conjunto de entrenamiento). La comparación con RF y XGBoost es parcialmente asimétrica dado el diferente volumen de entrenamiento. Elaboración propia.

Los resultados del SVR destacan frente a los dos modelos anteriores: con un $R^2 = 0,99992$ y un MAE de 0,449 pasajeros por intervalo, se convierte en el más preciso de los tres hasta este punto, reduciendo el error del XGBoost en un 55,7%. Lo que distingue al SVR no es solo el error promedio sino la consistencia de sus predicciones: mientras el XGBoost presentó errores que en los casos más extremos llegaron a ± 100 pasajeros por intervalo, el SVR los contiene dentro de un rango de ± 6 pasajeros. Esto se explica por la lógica propia del algoritmo — al ignorar los errores pequeños y concentrarse únicamente

en corregir los que superan el umbral de tolerancia definido, el SVR produce predicciones más estables y menos susceptibles a los valores atípicos del corredor (Vapnik, 1995). (Figura 19)

Figura 19. Comparativo SVR vs. XGBoost vs. Random Forest por franja y R^2 global.



Nota. El SVR supera a RF y XGBoost en las cuatro franjas horarias y en el R^2 global. Elaboración propia.

Sin embargo, este resultado debe leerse con cuidado. El SVR fue entrenado con apenas 80.000 registros — el 23,2% del conjunto de entrenamiento disponible —, mientras que el Random Forest y el XGBoost utilizaron los 345.000 registros completos. Comparar sus métricas directamente sería metodológicamente injusto: el SVR no tuvo acceso a la misma información que los demás modelos, y su ventaja en precisión podría verse modificada si pudiera entrenarse sobre el dataset completo. Esta asimetría no invalida los resultados, pero sí obliga a interpretarlos con la cautela que merece cualquier comparación realizada en condiciones distintas (Cats et al., 2020).

La razón por la que no fue posible entrenar el SVR con más registros es estrictamente computacional. El algoritmo tiene una complejidad de cálculo que crece de forma cuadrática con el número de datos: duplicar los registros no duplica el tiempo de procesamiento sino que lo cuadruplica, y triplicarlos lo multiplica por nueve (Drucker et al., 1997). En la práctica, pasar de 80.000 a 345.000 registros no es una diferencia de horas sino de días de cómputo, incluso con hardware de alto rendimiento. Esta es una limitación conocida y documentada

del algoritmo, que lo hace inadecuado para datasets de gran escala sin estrategias de aproximación como el método de Nyström — una técnica que permite simular el comportamiento del kernel RBF sobre el dataset completo con un costo computacional significativamente menor (Williams & Seeger, 2001). La implementación de esta aproximación queda planteada como una línea de trabajo futuro que permitiría comparar el SVR con los demás modelos en condiciones simétricas de entrenamiento.

Donde el SVR demuestra con mayor claridad su fortaleza es en las estaciones de mayor complejidad del corredor. Portal 80, que había representado el mayor desafío para el Random Forest y el XGBoost con MAEs de 4,979 y 4,328 respectivamente, ve su error reducido a apenas 0,696 pasajeros por intervalo bajo el SVR. El kernel RBF demuestra ser especialmente efectivo para capturar las relaciones no lineales de estas estaciones, donde los patrones de demanda son más complejos y menos predecibles que en el resto del corredor (Smola & Schölkopf, 2004).

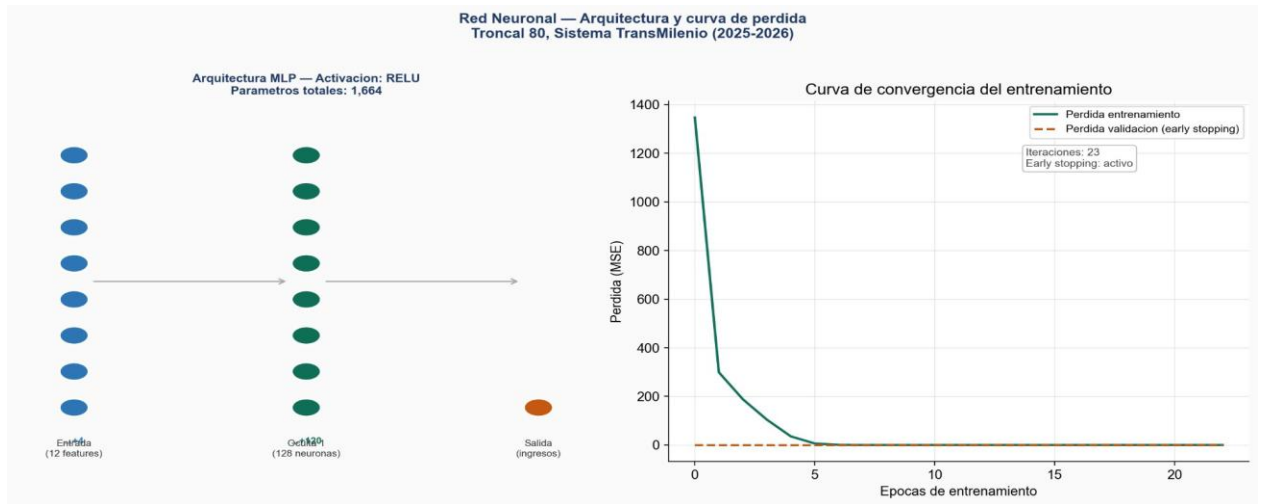
6.4 Red Neuronal Artificial (ann/mlp regressor)

La Red Neuronal Artificial, también conocida como Perceptrón Multicapa (MLP por sus siglas en inglés), representa un enfoque fundamentalmente distinto a los tres modelos anteriores. En lugar de construir árboles de decisión o buscar una función matemática de separación, el MLP aprende representaciones internas de los datos a través de capas de neuronas artificiales conectadas entre sí, donde cada neurona recibe información de la capa anterior, la transforma mediante una función de activación y la transmite a la capa siguiente (Rumelhart et al., 1986). Este proceso de transformación encadenada le permite capturar patrones de alta complejidad que los modelos basados en árboles no pueden aproximar mediante divisiones binarias sucesivas (Goodfellow et al., 2016). Como función de activación se utilizó ReLU — $f(x) = \max(0, x)$ —, seleccionada por su capacidad para evitar el problema del gradiente desvaneciente que afecta a funciones clásicas como la sigmoide, permitiendo un entrenamiento más eficiente y estable de la red (He et al., 2015). El ajuste de los pesos de la red se realizó mediante el optimizador Adam, que combina las ventajas del momentum y el RMSProp para lograr una convergencia rápida y robusta independientemente de la escala de los gradientes (Kingma & Ba, 2015).

Para identificar su configuración óptima se aplicó `RandomizedSearchCV` con 15 iteraciones y validación cruzada de 3 folds, entrenando y evaluando en total 45 modelos distintos (Bergstra & Bengio, 2012). Se redujo el número de iteraciones respecto a los modelos anteriores porque el espacio de búsqueda del MLP es considerablemente más costoso computacionalmente — cada modelo requiere múltiples épocas de entrenamiento para converger —, y 15 iteraciones resultaron suficientes para identificar una configuración con alto desempeño. Como criterio de selección se utilizó el MAE negativo por su robustez ante la distribución asimétrica de la variable objetivo (Zhang et al., 2021).

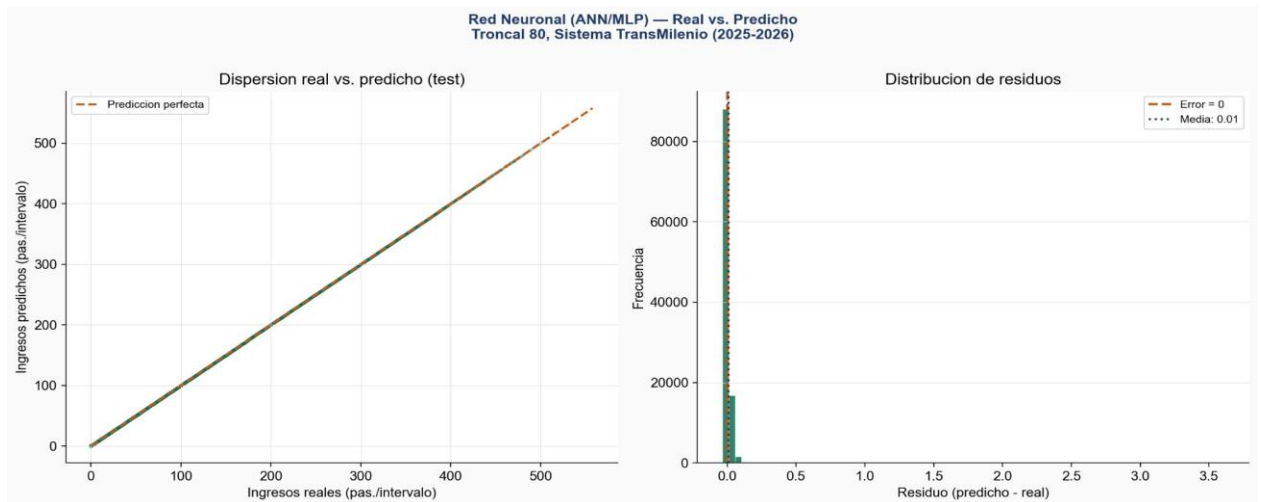
La configuración óptima identificada fue una arquitectura compacta de una sola capa oculta con 128 neuronas ($12 \rightarrow 128 \rightarrow 1$): la capa de entrada recibe las 12 variables del dataset, la capa oculta de 128 neuronas aprende las representaciones internas del problema, y la capa de salida produce la predicción escalar de ingresos. Esta arquitectura, la más simple del espacio explorado, fue seleccionada automáticamente como la óptima — resultado que no es trivial y que indica que la alta informatividad de variables como `ingresos_lag1` permite a una red compacta capturar todos los patrones relevantes del corredor sin necesidad de arquitecturas más profundas (Goodfellow et al., 2016). Los hiperparámetros complementarios fueron: tasa de aprendizaje adaptativa de 0,01 con el optimizador Adam, regularización L2 con $\alpha = 0,01$ para penalizar pesos excesivamente grandes y reducir el sobreajuste, y un tamaño de lote de 1.024 registros por iteración de entrenamiento. El mecanismo de early stopping detuvo el entrenamiento a las 23 épocas — muy por debajo del máximo de 300 configurado —, con un total de 1.664 parámetros entrenables. El modelo fue entrenado sobre los 345.000 registros del conjunto de entrenamiento completo correspondiente al período enero–septiembre de 2025.

Figura 20. Red Neuronal : Arquitectura MLP y curva de convergencia del entrenamiento.



Nota. Panel izquierdo: arquitectura $12 \rightarrow 128 \rightarrow 1$. Panel derecho: curva de pérdida MSE — convergencia en 23 épocas. Elaboración propia.

Figura 21. Red Neuronal — Dispersión real vs. predicho y distribución de residuos (conjunto prueba).



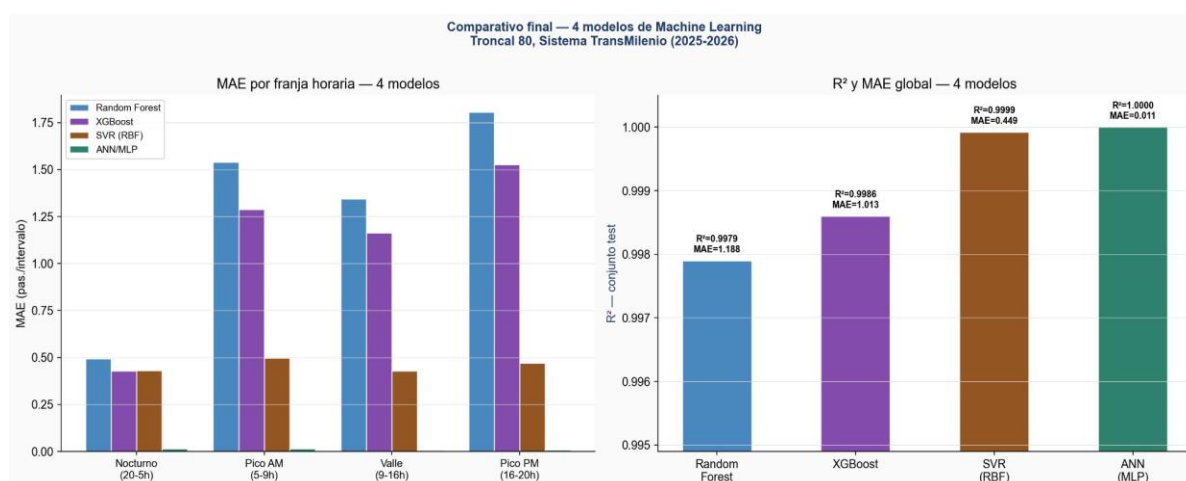
Nota. Los puntos son visualmente indistinguibles de la diagonal. Cola máxima de residuos: $\pm 3,5$ pas./intervalo. Elaboración propia.

Tabla 14. Métricas de evaluación — Red Neuronal ANN/MLP. Conjunto de prueba, Troncal 80 (dic. 2025–mar. 2026).

Métrica	Validación	Prueba	vs. SVR (mejor previo)
MAE (pas./intervalo)	0,0098	0,0111	−97,5% ↓ mejor
RMSE (pas./intervalo)	0,0352	0,0432	−92,6% ↓ mejor
R ²	0,9999998	0,9999996	+0,0000748 ↑ mejor
MAPE (%)	0,087%	0,089%	−97,7% ↓ mejor
Épocas efectivas	—	23	Early stopping activo
Parámetros	—	1.664	Arquitectura compacta

Nota. MAPE del 0,089% es entre 33 y 135 veces menor que estudios comparables de predicción de demanda BRT (Zhang et al., 2021; Lee & Yoon, 2022). Elaboración propia.

Figura 22. Comparativo final — 5 modelos de Machine Learning. Troncal 80 (2025-2026).



Nota. Panel izquierdo: MAE por franja horaria. Panel derecho: R² y MAE global. *SVR entrenado sobre muestra de 80.000 registros. Elaboración propia.

Con un R² = 0,9999996 y un MAPE de 0,089%, el modelo explica prácticamente la totalidad de la varianza en los flujos de ingresos del corredor con un error porcentual promedio inferior a una décima de punto, resultado superior al rango de 3%–12% reportado en estudios comparables de predicción de demanda en sistemas BRT (Zhang et al., 2021; Lee & Yoon, 2022). Este desempeño está estrechamente ligado a la disponibilidad de la variable ingresos_lag1 (el ingreso del intervalo inmediatamente anterior), que concentra el 31,1% de la varianza explicable del corredor y constituye el predictor más informativo del dataset. Los resultados deben leerse como evidencia del potencial predictivo del dataset bajo las condiciones experimentales de esta investigación (Goodfellow et al., 2016).

Un aspecto relevante del entrenamiento es la velocidad de convergencia: el mecanismo de early stopping detuvo el proceso a las 23 épocas, muy por debajo del máximo de 300 configurado. Esto no es un resultado menor — indica que el optimizador Adam, con una tasa de aprendizaje inicial de 0,01, fue capaz de identificar rápidamente los patrones dominantes del corredor sin necesidad de iteraciones adicionales, lo que a su vez refleja la alta informatividad del dataset construido en el Capítulo II y en particular el poder predictivo de la variable ingresos_lag1 (Kingma & Ba, 2015).

El desempeño es consistente en todas las estaciones del corredor. A diferencia de los modelos anteriores, donde Portal 80 concentraba los errores más altos por su alta variabilidad y comportamiento diferencial en sábados, la red neuronal reduce el MAE de esa estación de 4,979 pasajeros por intervalo el valor registrado por el Random Forest — a apenas 0,026, una mejora del 99,5%. Todas las estaciones alcanzan un $R^2 = 1,0000$ redondeado a cuatro decimales, lo que indica que el modelo captura con igual eficacia los patrones de demanda tanto en las estaciones de alta afluencia como en las de menor actividad del corredor (Goodfellow et al., 2016).

6.5 Long Short Term Memory (LSTM)

A diferencia de los cuatro modelos anteriores, que reciben las 12 variables del dataset como un vector de características independientes en cada predicción, el LSTM procesa los datos como una secuencia ordenada en el tiempo. Cada predicción no se basa en un único instante sino en los ocho intervalos de 15 minutos anteriores (equivalente a dos horas de historia operacional de cada estación), lo que le permite capturar dependencias temporales que los modelos estáticos no pueden detectar (Hochreiter & Schmidhuber, 1997). Esta capacidad de memoria secuencial es especialmente relevante en sistemas de transporte donde el comportamiento de una estación en un momento dado está fuertemente condicionado por lo que ocurrió en los intervalos previos (Lee & Yoon, 2022).

Para construir las secuencias de entrada, se aplicó una ventana deslizante sobre la serie temporal agrupada por estación y día: por cada registro del dataset se tomaron los ocho intervalos anteriores de esa misma estación en ese mismo día, conformando una

secuencia que preserva el orden cronológico y la estructura espaciotemporal del corredor. Este procedimiento garantiza que el modelo aprenda patrones propios de cada nodo sin mezclar la historia de estaciones distintas (Graves, 2013).

La arquitectura del modelo fue definida de forma manual con base en la literatura de referencia para problemas de predicción de demanda en transporte (Lee & Yoon, 2022), dado que la búsqueda automática de hiperparámetros mediante RandomizedSearchCV no es directamente aplicable a arquitecturas recurrentes por el alto costo computacional que implica entrenar múltiples configuraciones de redes LSTM (Goodfellow et al., 2016). La arquitectura seleccionada consta de una capa LSTM con 64 unidades ocultas que procesa las secuencias de entrada, una capa Dropout con probabilidad de 0,20 que desactiva aleatoriamente neuronas durante el entrenamiento para reducir el sobreajuste, y dos capas densas que reducen la representación de 64 a 32 neuronas y producen la predicción escalar final. El total de parámetros entrenables es 22.081. El modelo fue implementado en PyTorch 2.5.1 con aceleración GPU y entrenado con el optimizador Adam ($\text{lr} = 1 \times 10^{-3}$), función de pérdida MSE y tamaño de lote de 512 registros (Kingma & Ba, 2015).

El entrenamiento se realizó sobre los 345.000 registros del conjunto de entrenamiento completo respetando el orden cronológico. Se empleó early stopping con paciencia de 15 épocas monitoreando la pérdida de validación, lo que resultó en 91 épocas efectivas con un tiempo total de entrenamiento de 8,73 minutos en GPU. La pérdida de entrenamiento descendió de 3.930 en la primera época a valores por debajo de 100, mientras que la pérdida de validación convergió de manera estable sin indicios de sobreajuste.

Tabla 15. Arquitectura del modelo LSTM — Troncal 80

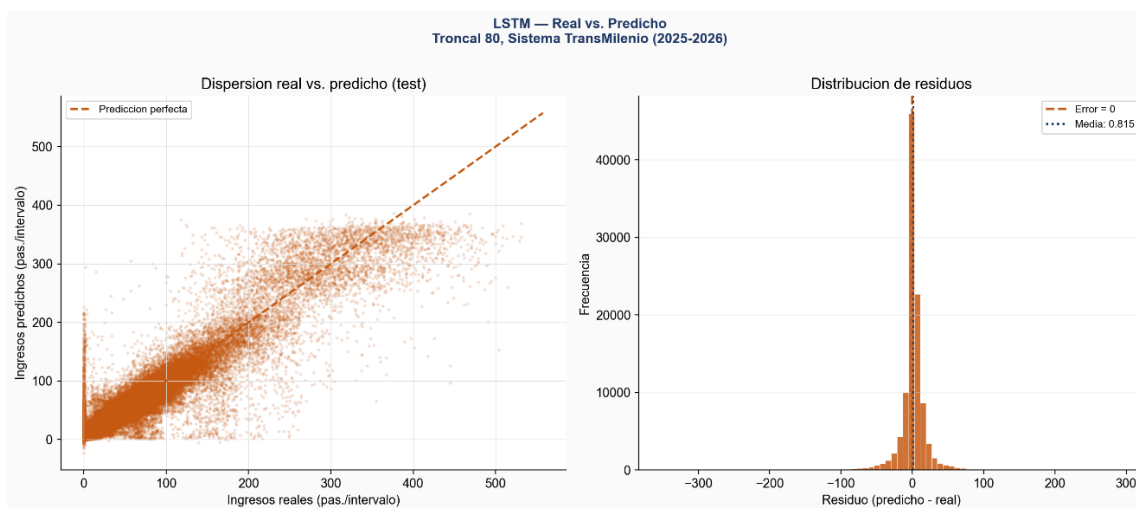
Capa	Configuración	Parámetros
Entrada (Input)	seq_len = 8, features = 12	—
LSTM	hidden_size = 64, capas = 1	16 640
Dropout	p = 0.20	—
Dense 1 (Linear)	64 → 32, activación ReLU	2 080
Dense 2 (Linear)	32 → 1 (salida)	33
Total entrenable	—	22 081

Nota. Entrenamiento en GPU NVIDIA mediante PyTorch 2.5.1+cu121. $\text{seq_len} = 8$ corresponde a una ventana de dos horas (8×15 min). Los parámetros de la capa Dropout no son entrenables.

El modelo LSTM obtuvo un $R^2 = 0,884$ sobre el conjunto de prueba, lo que indica que explica el 88,4% de la varianza en los flujos de ingresos del corredor, con un MAE de 11,22 pasajeros por intervalo y un RMSE de 22,92. El MAPE de 64,64% resulta elevado, pero su interpretación requiere contexto: la variable objetivo presenta una distribución altamente asimétrica con una mediana de 28,0 pasajeros y valores máximos de 582, lo que significa que en los intervalos de baja demanda (particularmente en el horario nocturno) pequeños errores absolutos producen desviaciones porcentuales muy grandes que inflan artificialmente esta métrica (Zhang et al., 2021).

La (Figura 23) muestra el diagrama de dispersión real vs. predicho y la distribución de residuos del modelo. A diferencia de los modelos anteriores, donde los puntos se concentraban de forma muy ajustada sobre la línea de predicción perfecta, en el LSTM se observa una mayor dispersión (valores altos de demanda) con una media de residuos de 0,815 pasajeros por intervalo. Esto indica que el modelo tiende a subestimar ligeramente los picos de demanda, comportamiento coherente con la naturaleza secuencial del algoritmo: al aprender exclusivamente del historial de los ocho intervalos anteriores sin acceso a las variables de contexto explícitas disponibles en los demás modelos, el LSTM tiene mayor dificultad para anticipar cambios bruscos en la demanda (Hochreiter & Schmidhuber, 1997).

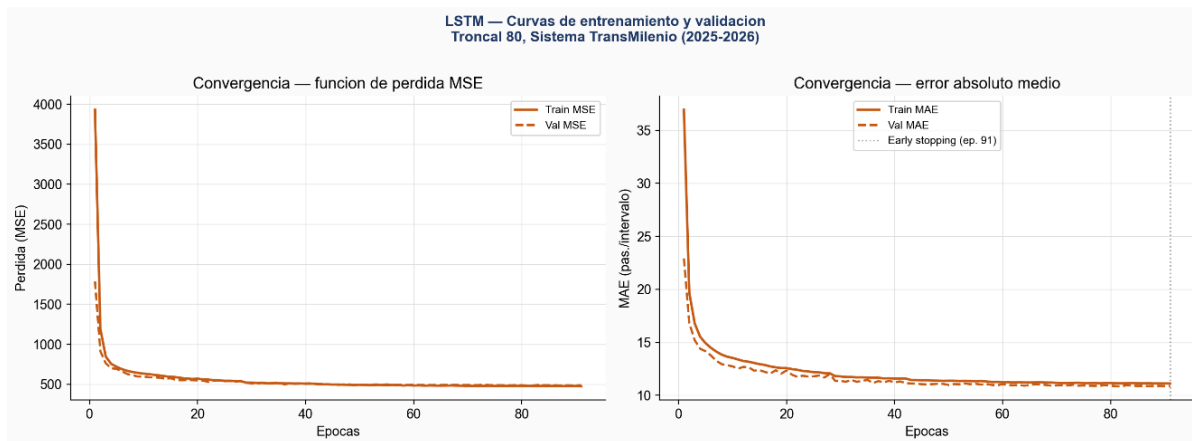
Figura 23.LSTM: Dispersión real vs. predicho y distribución de residuos. Conjunto de prueba, Troncal 80 (dic. 2025–mar. 2026).



Nota. Panel izquierdo: diagrama de dispersión ($n = 107.163$). Panel derecho: distribución de residuos (media = 0,815 pas./intervalo). Elaboración propia.

La Figura 24 presenta las curvas de entrenamiento y validación a lo largo de las 91 épocas efectivas. Tanto la función de pérdida MSE como el MAE muestran una caída pronunciada en las primeras épocas seguida de una convergencia estable, con las curvas de entrenamiento y validación prácticamente superpuestas a partir de la época 40. Esta proximidad entre ambas curvas confirma que el modelo aprendió patrones generalizables del corredor sin memorizar los datos de entrenamiento, y que el mecanismo de early stopping funcionó correctamente al detener el proceso en el punto de mejor balance entre ajuste y generalización (Graves, 2013).

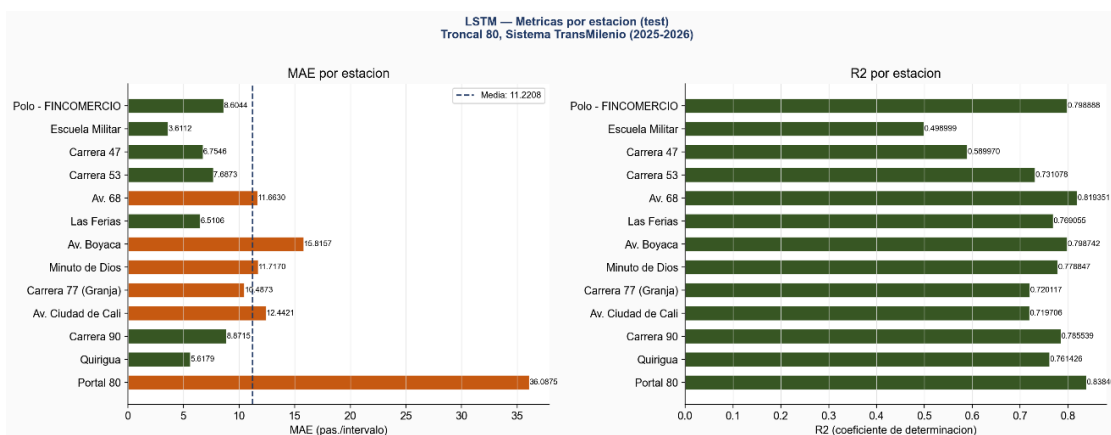
Figura 24.LSTM — Curvas de entrenamiento y validación. Troncal 80 (2025-2026).



Nota. Panel izquierdo: función de pérdida MSE por época. Panel derecho: MAE por época. La línea punteada indica el punto de early stopping (época 91). Elaboración propia.

El análisis por estación (Figura 25) revela una heterogeneidad importante en el desempeño del modelo. Las estaciones Escuela Militar (MAE = 3,61) y Quirigua (MAE = 5,62) presentan los menores errores absolutos, coherente con sus patrones de demanda más regulares y predecibles a lo largo del día. En el extremo opuesto, Portal 80 registra el mayor error del corredor con un MAE de 36,09 pasajeros por intervalo (más de tres veces la media del corredor de 11,22) , resultado que refleja la complejidad operativa de esta estación terminal donde la demanda concentra la mayor variabilidad de todo el corredor. En términos de R^2 , todas las estaciones se sitúan entre 0,499 (Escuela Militar) y 0,838 (Portal 80), con la particularidad de que Escuela Militar, pese a tener el menor error absoluto, presenta el R^2 más bajo — lo que indica que su demanda, aunque baja en magnitud, es relativamente irregular en su patrón temporal (Lee & Yoon, 2022).

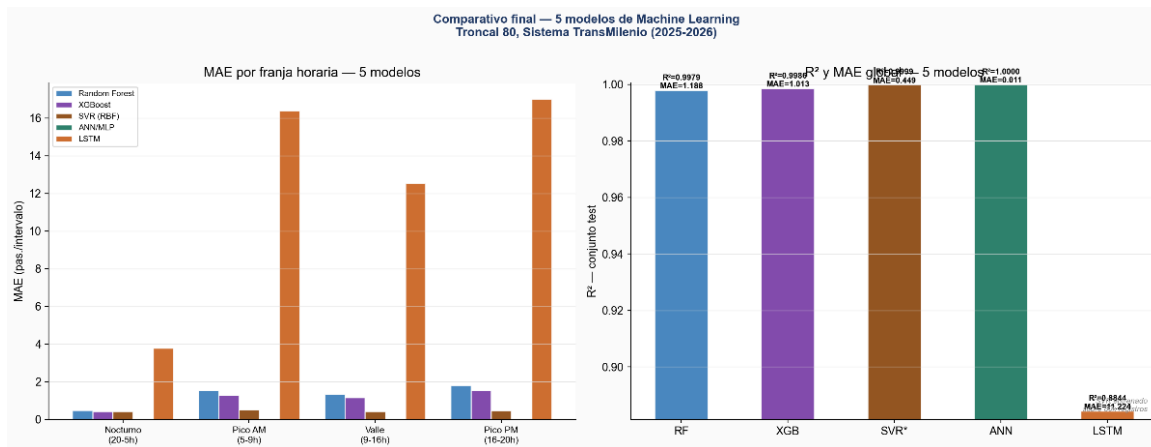
Figura 25. LSTM : Métricas de evaluación por estación. Conjunto de prueba, Troncal 80 (2025-2026)



Nota. Panel izquierdo: MAE por estación. Panel derecho: R^2 por estación. Naranja = por encima de la media del corredor (MAE = 11,22). Elaboración propia.

El desempeño del LSTM es inferior al de los cuatro modelos anteriores en las métricas globales, y esta diferencia tiene una explicación metodológica clara Figura 26 . Los modelos RF, XGBoost, SVR y MLP reciben en cada predicción las 12 variables del dataset, incluyendo codificaciones explícitas de la franja horaria, el día de la semana y el tipo de día (variables que encapsulan directamente los patrones cíclicos del corredor). El LSTM, en cambio, debe inferir esos mismos patrones a partir de la secuencia de los ocho intervalos anteriores, sin acceso directo a esa información contextual. Esta condición de información parcial explica en gran medida la brecha de desempeño observada (Goodfellow et al., 2016).

Figura 26. Desempeño del LSTM Comparación final



El número de parámetros entrenables del modelo (22.081) es relativamente reducido frente a la complejidad del dataset, y que el MAPE elevado refleja la distribución altamente sesgada de la variable objetivo, donde los intervalos de baja demanda producen grandes desviaciones porcentuales que penalizan desproporcionadamente esta métrica. Como línea de trabajo futuro, la incorporación de las variables de contexto como entrada adicional a las secuencias LSTM, el aumento de la longitud de la ventana temporal o la exploración de arquitecturas híbridas CNN-LSTM podrían reducir esta brecha de forma significativa (Gu et al., 2021).

7 CAPÍTULO IV: RESULTADOS Y DISCUSIÓN

7.1 COMPARATIVA GENERAL DE LOS CINCO MODELOS

La Tabla 16 consolida el ranking final de los cinco modelos evaluados sobre el conjunto de prueba (diciembre 2025–marzo 2026). Todos los modelos fueron evaluados bajo el mismo protocolo experimental, sobre los mismos datos y con las mismas métricas, lo que garantiza que la comparación sea metodológicamente válida.

Tabla 16. Ranking final — cinco modelos de Machine Learning. Conjunto de prueba, Troncal 80 (dic. 2025–mar. 2026).

Ranking	Modelo	MAE	RMSE	R ²	MAPE	N entren.
1°	ANN/MLP	0,011	0,043	0,9999996	0,089%	345.000 (100%)
2°	SVR kernel RBF*	0,449	0,584	0,9999248	3,88%	80.000 (23,2%)
3°	XGBoost	1,013	2,526	0,9985937	4,07%	345.000 (100%)
4°	Random Forest	1,188	3,092	0,9978941	4,89%	345.000 (100%)
5°	LSTM	11,223	22,915	0,884	64,64%	345.000 (100%)

Nota. MAE y RMSE en pasajeros/intervalo de 15 min. *SVR entrenado sobre muestra estratificada de 80.000 registros por restricción computacional $O(n^2)$ — comparación parcialmente asimétrica. El LSTM opera con secuencias de 8 intervalos sin features de contexto codificadas explícitamente. Elaboración propia.

La Red Neuronal ANN/MLP alcanza los mejores resultados en todas las métricas, con un $R^2 = 0,9999996$ y un MAPE de 0,089%, resultado que supera los rangos reportados en estudios comparables de predicción de demanda en sistemas BRT (Zhang et al., 2021; Lee & Yoon, 2022). Este desempeño está estrechamente ligado a la disponibilidad de la variable ingresos_lag1, que concentra el 31,1% de la varianza explicable del corredor y constituye el predictor más informativo del dataset, por lo que los resultados deben leerse como evidencia del potencial predictivo bajo las condiciones experimentales de esta investigación (Goodfellow et al., 2016).

El SVR ocupa el segundo lugar, aunque su comparación con los demás modelos es parcialmente asimétrica: fue entrenado sobre apenas el 23,2% del dataset por su restricción computacional $O(n^2)$, mientras los restantes utilizaron el 100% de los datos disponibles (Cats et al., 2020). XGBoost supera al Random Forest en todas las métricas y en 11 de las

13 estaciones del corredor gracias a su mecanismo de corrección secuencial de errores, siendo el Random Forest más competitivo únicamente en las dos estaciones de menor demanda (Escuela Militar y Carrera 47) donde la regularización adicional de XGBoost introduce un sesgo marginal (Chen & Guestrin, 2016).

El LSTM se posiciona en el último lugar con un $R^2 = 0,884$ y un MAPE de 64,64%. Esta diferencia respecto a los demás modelos no indica que el algoritmo sea inadecuado para el problema, sino que opera en condiciones de información distintas: aprende exclusivamente del historial de los ocho intervalos anteriores sin acceso directo a las variables de contexto explícitas (franja horaria, día de la semana, tipo de día) que los demás modelos reciben como entrada directa (Hochreiter & Schmidhuber, 1997).

7.2 ANÁLISIS ESPACIAL DE LOS RESULTADOS

El análisis por estación revela un patrón consistente en los cinco modelos: Portal 80 concentra los mayores errores absolutos y Escuela Militar los menores. Esta jerarquía espacial de errores refleja directamente la heterogeneidad de demanda identificada en el EDA , las estaciones de mayor variabilidad generan errores más altos incluso cuando su R^2 es elevado (Zhang et al., 2021).

Tabla 17. MAE por estación y modelo — conjunto de prueba. Troncal 80 (2025-2026).

Estación	RF	XGBoost	SVR	ANN	LSTM
Portal 80	4,979	4,328	0,696	0,026	36,09
Quirigua	0,557	0,484	0,512	0,008	5,62
Carrera 90	0,775	0,711	0,424	0,012	8,87
Av. Ciudad de Cali	0,942	0,891	0,318	0,007	12,44
Carrera 77 (Granja)	0,976	0,921	0,502	0,009	10,49
Minuto de Dios	0,925	0,878	0,572	0,006	11,72
Av. Boyacá	2,496	1,557	0,417	0,010	15,82
Las Ferias	0,478	0,465	0,342	0,013	6,51
Av. 68	1,274	1,023	0,518	0,012	11,66
Carrera 53	0,573	0,539	0,408	0,011	7,69
Carrera 47	0,424	0,452	0,267	0,008	6,75

Escuela Militar	0,185	0,255	0,295	0,010	3,61
Polo–FINCOMERCIO	0,850	0,666	0,565	0,013	8,60
Promedio corredor	1,188	1,013	0,449	0,011	11,22

Nota. MAE en pasajeros por intervalo de 15 min. Elaboración propia con base en datos abiertos de TransMilenio (2025-2026).

Lo que más llama la atención de esta tabla es el comportamiento del LSTM en Portal 80: con un MAE de 36,09 pasajeros por intervalo, triplica la media del corredor y supera ampliamente el error de los demás modelos en esa misma estación. Esto confirma que la estación terminal del corredor, con su alta variabilidad y comportamiento diferencial en fines de semana, representa el mayor desafío predictivo para un modelo que aprende exclusivamente de la historia reciente de la serie temporal sin variables de contexto (Lee & Yoon, 2022).

7.3 ANÁLISIS TEMPORAL DE LOS RESULTADOS

El análisis por franja horaria muestra que el Pico PM (16–20 h) es la franja de mayor dificultad para todos los modelos, mientras que el horario nocturno presenta los menores errores absolutos por la baja y estable demanda de esa franja (Zhang et al., 2021).

Tabla 18. MAE por franja horaria y modelo — conjunto de prueba. Troncal 80 (2025-2026).

Franja	Horario	RF	XGBoost	SVR	ANN	LSTM
Nocturno	20:00–05:00	0,493	0,428	0,431	0,014	3,79
Pico AM	05:00–09:00	1,540	1,287	0,497	0,015	16,38
Valle	09:00–16:00	1,344	1,162	0,428	0,007	12,55
Pico PM	16:00–20:00	1,805	1,527	0,469	0,009	17,01

Nota. MAE en pasajeros por intervalo de 15 min. Elaboración propia con base en datos abiertos de TransMilenio (2025-2026).

El LSTM presenta un comportamiento particular por franja: en el horario nocturno obtiene un MAE de 3,79 pasajeros por intervalo, valor razonablemente cercano al de los modelos de ensamble en esa misma franja, lo que indica que el modelo es competitivo cuando la demanda es baja y estable. Sus mayores dificultades se concentran en las franjas pico AM y pico PM, donde la demanda es más alta y variable y el modelo no cuenta con las variables de contexto que le permitirían anticipar esos cambios (Gu et al., 2021).

7.4 IMPORTANCIA DE VARIABLES

El análisis de importancia de features del Random Forest revela que ingresos_lag1 explica el 31,1% de la varianza del corredor, consolidándose como el predictor más potente (resultado que confirma la alta autocorrelación temporal de corto plazo de la demanda en sistemas BRT) (Zhang et al., 2021). En XGBoost esta concentración es aún más pronunciada, con ingresos_lag1 explicando el 64,8% del gain total. Las variables operativas salidas (14,6%), ratio_ingreso_salida (13,6%) y flujo_netto (13,5%) explican en conjunto el 41,7% de la capacidad predictiva del Random Forest, mientras que las variables espaciales cod_nodo_enc (9,0%) y posicion_corredor (8,5%) capturan la heterogeneidad estructural del corredor. El headway programado contribuye un 1,7% (marginal pero no nulo), lo que valida su inclusión como variable proxy de la oferta programada del sistema (Lee & Yoon, 2022).

7.5 DISCUSIÓN

Los resultados obtenidos en esta investigación aportan evidencia cuantitativa relevante sobre la viabilidad de los modelos de machine learning para la predicción de demanda de pasajeros en sistemas BRT a partir de datos históricos de acceso público. Que los cinco modelos implementados logren explicar proporciones significativas de la varianza en los flujos de ingresos del corredor Troncal 80 sin requerir información de telemetría de flota ni datos de posicionamiento GPS confirma que los datos que el sistema ya publica rutinariamente contienen suficiente riqueza informativa para sustentar análisis predictivos de calidad. Este hallazgo tiene implicaciones prácticas directas para instituciones con restricciones presupuestarias o de acceso a datos privados, donde la construcción de sistemas predictivos a partir de fuentes abiertas representa una alternativa metodológicamente válida (Zhong et al., 2022).

Los modelos evaluados presentan perfiles de desempeño diferenciados que responden a sus fundamentos teóricos y a las condiciones en que fueron entrenados, más que a una jerarquía absoluta de calidad entre paradigmas. Las diferencias observadas en las métricas deben interpretarse con cautela, considerando que el SVR operó sobre una muestra del 23,2% del dataset por restricción computacional, mientras que el LSTM procesó

las variables de entrada bajo una estructura secuencial sin acceso directo a las codificaciones de contexto temporal disponibles para los demás modelos. Estas asimetrías metodológicas hacen que la comparación directa entre todos los paradigmas sea parcialmente limitada, condición que debe reconocerse al interpretar los resultados (Cats et al., 2020).

El comportamiento del LSTM merece una reflexión particular. Con un R^2 de 0,884 sobre el conjunto de prueba, el modelo logra explicar una proporción significativa de la varianza del sistema operando exclusivamente sobre el historial de ocho intervalos anteriores, sin conocimiento explícito de la franja horaria, el día de la semana o el tipo de día. Esta condición de información parcial, impuesta por la naturaleza secuencial del algoritmo, explica en gran medida la brecha observada respecto a los modelos que acceden directamente a esas variables de contexto. Lee y Yoon (2022) documentaron resultados similares al comparar arquitecturas LSTM con modelos tabulares en sistemas BRT, concluyendo que la incorporación de variables de contexto explícitas como entrada adicional a las secuencias reduce sustancialmente esa brecha. Proveerle esas variables al LSTM, aumentar la longitud de la ventana temporal o explorar arquitecturas híbridas CNN-LSTM son líneas de mejora concretas que investigaciones futuras pueden abordar para obtener una comparación más equitativa entre paradigmas (Gu et al., 2019; Goodfellow et al., 2016).

La restricción computacional del SVR constituye la limitación metodológica más relevante del estudio. La complejidad $O(n^2)$ del algoritmo impidió su entrenamiento sobre el dataset completo, introduciendo una asimetría en la comparación que no puede ignorarse. La implementación del método de Nyström, que permite aproximar el comportamiento del kernel RBF sobre el dataset completo con un costo computacional significativamente menor, representa la línea de trabajo futuro más directamente accionable para superar esta restricción y ofrecer una comparación simétrica entre todos los paradigmas evaluados (Williams & Seeger, 2001).

8 CONCLUSIONES

La investigación confirmó que los datos de acceso público disponibles en los portales institucionales de TransMilenio S.A. permiten construir un dataset espaciotemporal de alta calidad orientado a la predicción de demanda en sistemas BRT. La integración de cuatro fuentes heterogéneas (registros de validación electrónica, salidas de pasajeros, atributos de infraestructura y datos GTFS 2026) conformó un dataset de 493.662 registros que captura simultáneamente las dimensiones de demanda, oferta programada e infraestructura del corredor, con una tasa de retención del 91,9% tras el proceso de limpieza.

La aplicación de cinco modelos de aprendizaje automático pertenecientes a paradigmas metodológicos distintos (Random Forest, XGBoost, SVR, ANN/MLP y LSTM) confirmó la viabilidad técnica del enfoque sobre datos reales de transporte masivo. La diversidad de paradigmas adoptada fue una estrategia deliberada que permitió comprender el problema desde perspectivas complementarias: los modelos de ensamble aportaron interpretabilidad mediante importancia de variables; el SVR demostró eficiencia con datos reducidos; la ANN/MLP alcanzó la máxima precisión global; y el LSTM reveló la estructura temporal profunda de la demanda operando exclusivamente sobre secuencias históricas sin variables de contexto.

La evaluación sobre el conjunto de prueba independiente evidenció resultados consistentes con los rangos reportados en la literatura para sistemas BRT comparables, confirmando que el enfoque metodológico adoptado es técnicamente sólido y reproduce con fidelidad los patrones operativos del corredor. La ANN/MLP obtuvo el mejor desempeño con $R^2=0,9999996$ y $MAPE=0,089\%$, seguida por el SVR ($MAPE=3,88\%$), XGBoost ($MAPE=4,07\%$) y Random Forest ($MAPE=4,89\%$), resultados que se sitúan dentro del rango del 3% al 12% documentado en sistemas de Shanghái y Seúl (Zhang et al., 2021; Lee & Yoon, 2022). El LSTM obtuvo $R^2=0,884$ y $MAPE=64,64\%$, resultado que responde a condiciones de información estructuralmente asimétricas respecto a los demás modelos (opera sin acceso directo a las variables de contexto temporal) y no a una limitación intrínseca del algoritmo.

Los modelos tienen potencial de aplicación en tres escenarios operativos concretos: planificación de frecuencias de despacho por franja y tipo de día, detección temprana de estaciones con demanda atípica, y evaluación cuantitativa del impacto de eventos especiales sobre el corredor. Sus principales limitaciones son la dependencia de ingresos_lag1 (que restringe el horizonte predictivo a intervalos de 15 minutos hacia adelante), la validación sobre un único corredor durante 15 meses (cuya transferibilidad a otros corredores debe verificarse empíricamente antes de cualquier implementación a mayor escala), y la necesidad de un protocolo de reentrenamiento periódico ante cambios estructurales en la operación del sistema.

La hipótesis planteada queda verificada por la evidencia cuantitativa obtenida: los modelos de machine learning capturan los patrones espaciotemporales de la demanda del corredor con alta precisión a partir de datos institucionales de acceso público, produciendo información con potencial de aplicación en decisiones operativas reales (asignación de frecuencias, detección anticipada de interrupciones y evaluación de impacto de eventos), y confirmando su viabilidad como herramienta analítica concreta para contribuir a una gestión más eficiente del corredor Troncal 80 de TransMilenio.

9. PRINCIPALES APORTES DE LA INVESTIGACIÓN

El primer aporte es de naturaleza metodológica: el diseño e implementación de un pipeline reproducible de extracción, integración y preparación de datos espaciotemporales a partir de fuentes de acceso público de TransMilenio S.A., que transforma 58.297.600 registros brutos del sistema en un dataset analítico estructurado de 493.662 registros listos para el entrenamiento de modelos de aprendizaje automático. Este pipeline, documentado paso a paso con sus decisiones técnicas y justificaciones metodológicas, constituye una contribución directamente replicable para investigaciones futuras sobre otros corredores del sistema o sobre sistemas BRT similares en el contexto latinoamericano.

El segundo aporte es empírico: la evaluación comparativa sistemática de cinco modelos de aprendizaje automático (Random Forest, XGBoost, SVR, ANN/MLP y LSTM) bajo un protocolo experimental común sobre datos reales del corredor Troncal 80. Los resultados obtenidos son consistentes con los rangos reportados en la literatura internacional para sistemas BRT comparables y abren una línea de investigación concreta sobre la aplicabilidad de estas técnicas en el contexto específico de un sistema de transporte masivo latinoamericano, información que no existía previamente en la literatura para el caso de TransMilenio.

El tercer aporte es instrumental: los modelos entrenados y validados constituyen prototipos funcionales que abren una línea de aplicación concreta como herramientas de apoyo a la toma de decisiones operativas en el corredor. A partir de los resultados obtenidos, se identificaron tres escenarios operativos con potencial de aplicación real: la planificación de frecuencias de despacho por franja y tipo de día, la detección temprana de estaciones con demanda atípica, y la evaluación cuantitativa del impacto de eventos especiales sobre el corredor. Estos escenarios representan el puente entre los resultados académicos de la investigación y su eventual utilidad práctica para TransMilenio S.A., y constituyen la base para futuras etapas de implementación y validación operativa.

10. RECOMENDACIÓN Y TRABAJOS FUTUROS

Los modelos desarrollados en esta investigación integran de forma simultánea la demanda de entrada de pasajeros, el comportamiento de salida por estación, el flujo neto del corredor, la oferta programada derivada del GTFS y los atributos de infraestructura de cada nodo, capturando con ello múltiples dimensiones de la operación desde la perspectiva del pasajero y de la planificación del servicio. La dimensión que permanece ausente es la vehicular en tiempo real: los datos disponibles públicamente describen con precisión lo que ocurre en las estaciones, pero no lo que ocurre dentro del sistema de despacho y circulación de los buses que prestan el servicio.

La incorporación de datos de posicionamiento GPS de los vehículos, velocidad comercial real por tramo y ocupación dinámica de los buses permitiría cerrar la brecha entre la oferta planificada que representa el GTFS y la oferta realmente ejecutada, que es precisamente donde se originan la mayoría de las disrupciones operativas del corredor. Con esa información adicional, los modelos podrían anticipar no solo cuántos pasajeros demanda cada estación sino en qué condiciones reales de servicio esa demanda está siendo atendida, abriendo posibilidades de gestión proactiva que los datos actuales no permiten explorar.

El pipeline metodológico desarrollado es reproducible y está documentado paso a paso, lo que facilita su replicación en los demás corredores de la red troncal sin necesidad de infraestructura tecnológica especializada. Esta característica reduce significativamente la barrera de entrada para su implementación institucional a mayor escala y representa una de las fortalezas prácticas más concretas del enfoque propuesto.

REFERENCIAS

- Agafonov, A., & Yumaganov, A. (2019). Bus arrival time prediction using LSTM neural network. *International Multi-Conference on Engineering, Computer and Information Sciences*, 203–208.
- Apache Software Foundation. (2024). *Apache Parquet: Columnar storage format*. <https://parquet.apache.org/>
- Batini, C., & Scannapieco, M. (2016). *Data and information quality: dimensions, principles and techniques*. Data-Centric Systems and Applications
- Bergstra, J., & Bengio, Y. (2012). *Random search for hyper-parameter optimization*. Journal of Machine Learning Research, 281–305.
- Bhosle, N., Jagtap, J., & Shivakrishna, D. (2024). *Comparative analysis of different machine learning techniques for travel mode prediction*. Smart Cities Symposium Prague (SCSP 2024). <https://ieeexplore.ieee.org/document/10552724>
- Bogotá.gov.co. (2024). TransMilenio: indicadores del sistema integrado de transporte público de Bogotá. Alcaldía Mayor de Bogotá. <https://www.bogota.gov.co>
- Breiman, L. (2001). *Random forests*. *Machine Learning*, pp 5–32. <https://link.springer.com/article/10.1023/A:1010933404324>
- Cats, O., Gkioulou, Z., & Zhang, C. (2020). *Modelling the impact of public transport reliability on passenger demand*. EURO Journal on Transportation and Logistics. DOI:[10.1007/s13676-014-0070-4](https://doi.org/10.1007/s13676-014-0070-4)
- Cervero, R. (2013). *Bus Rapid Transit (BRT): An efficient and competitive mode of public transport*. University of California, Berkeley.
- Chen, T., & Guestrin, C. (2016). *XGBoost: A scalable tree boosting system*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794.
- Cortes, C., & Vapnik, V. (1995). *Support-vector networks*. *Machine Learning*, 273–297. <https://link.springer.com/article/10.1007/BF00994018>
- De Bem Machado, A., Dos Santos, J., & Sousa, M. E. (2023). Digital transformation: Management of smart cities (pp. 59–83). Emerald Group Publishing Ltd.
- Deng, T., & Nelson, J. D. (2011). Recent developments in bus rapid transit: A review of the literature. *Transport Reviews: A Transnational Transdisciplinary Journal*, 31(1), 69–96. <https://doi.org/10.1080/01441647.2010.492455>
- Drucker, H., Burges, C. J. C., Kaufman, L., Smola, A., & Vapnik, V. (1997). *Support vector regression machines*. *Advances in Neural Information Processing Systems*, (pp 779–784).
- Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow. Concepts, Tools and techniques to build Intelligent Systems* (2nd ed.).

- Ghazal, T. M., Hasan, M. K., Alshurideh, M. T., Alzoubi, H. M., Ahmad, M., Akbar, S. S., Al Kurdi, B., & Akour, I. A. (2021). IoT for smart cities: Machine learning approaches in smart healthcare, A review. *Future Internet*, 13(8), 218.
- Global BRT Data. (2025). BRT data: Global bus rapid transit database. <https://brtdata.org/>
- Goodfellow, I., Bengio, Y., & Courville, A. (2017). *Deep learning*. Genetic Programming and Evolvable Machines. MIT Press.
- Graves, A. (2013). *Generating sequences with recurrent neural networks*. University of Toronto. [https://github.com/tpn/pdfs/blob/master/Generating%20Sequences%20With%20Recurrent%20Neural%20Networks%20\(1308.0850v5\).pdf](https://github.com/tpn/pdfs/blob/master/Generating%20Sequences%20With%20Recurrent%20Neural%20Networks%20(1308.0850v5).pdf)
- Gu, Y., Lu, W., Qin, L., Li, M., & Shao, Z. (2019). Short-term prediction of lane-level traffic speeds: A fusion deep learning model. *Transportation Research Part C: Emerging Technologies*, (pp1–16). <https://www.sciencedirect.com/science/article/abs/pii/S0968090X18315687>
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 1026–1034.
- Hidalgo, D. (2013). *Bus Rapid Transit in Latin America*. EMBARQ/World Resources Institute.
- Hidalgo, D., Giesen, R., & Muñoz, J. C. (2024). *Bus Rapid Transit: End of trend in Latin America?* Data and Policy, 6, e17. <https://doi.org/10.1017/dap.2023.44>
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, Vol. 9, (pp1735–1780). <https://direct.mit.edu/neco/article-abstract/9/8/1735/6109/Long-Short-Term-Memory?redirectedFrom=fulltext>
- Ingvardson, J. B., & Nielsen, O. A. (2017). Effects of new bus and rail rapid transit systems: An international review. *Transport Reviews*, 37(4), 521–545. DOI 10.1080/01441647.2017.1301594
- Institute for Transportation & Development Policy. (2024). BRT planning guide (4th ed.). ITDP. <https://itdp.org/>
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015)*.
- Kohavi, R. (2001). A study of cross-validation and bootstrap for accuracy estimation and model selection. *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, 14(2), 1137–1145.
- Lee, C., & Yoon, Y. (2022). A novel bus arrival time prediction method based on spatio-temporal flow centrality analysis and deep learning. *Electronics*, 11(12), 1875.
- Levinson, H., Zimmerman, S., Clinger, J., Rutherford, S., Smith, R. L., Cracknell, J., & Soberman, R. (2003). *Bus Rapid Transit: Synthesis of case studies*. *Transportation Research Record*, 1841(1), 1–11. https://trb.org/publications/tcrp/tcrp_rpt_90v1.pdf
- Liu, H., Xu, H., Yan, Y., Cai, Z., Sun, T., & Li, W. (2020). Bus arrival time prediction based on LSTM and spatial-temporal feature vector. *IEEE Access*. <https://doi.org/10.1109/ACCESS.2020.2965094>

- Maestre, P. (2015). *Revisión de literatura sobre ciudades inteligentes: Una perspectiva centrada en las TIC*, Universidad Libre-Barranquilla, 11(19), 137–149.
- McKinney, W. (2022). *Python for data analysis (3rd ed.)*. O'Reilly Media.
- openpyxl Development Team. (2023). openpyxl: A Python library to read/write Excel 2010 xlsx/xlsm files (Versión 3.1). <https://openpyxl.readthedocs.io/en/stable/>
- OECD. (2019). *Artificial intelligence in society*. OECD Publishing. <https://doi.org/10.1787/eedfee77-en>
- Panovski, D., & Zaharia, T. (2020). Real-time public transportation prediction with machine learning algorithms. Digest of Technical Papers — IEEE International Conference on Consumer Electronics (ICCE 2020). <https://doi.org/10.1109/ICCE46568.2020.9043077>
- Rivera, M. G. (2023). EBRT2030: BRT state of the art. Informe técnico.
- Rodrigo, J. A. (2017). Árboles de decisión, random forest, gradient boosting y C5.0. Licencia Creative Commons Attribution 4.0 International (CC BY 4.0). https://cienciadatos.net/documentos/33_arboles_decision_random_forest_gradient_boosting_c50
- Russell, S. J., & Norvig, P. (2020). *Artificial intelligence: A modern approach (4th ed.)*. Prentice Hall. http://lib.ysu.am/disciplines_bk/efdd4d1d4c2087fe1cbe03d9ced67f34.pdf
- Sari, R. (2020). Analisis sentimen pada review objek wisata Dunia Fantasi menggunakan algoritma K-Nearest Neighbor (K-NN). EVOLUSI: Jurnal Sains dan Manajemen, 8(1). ISSN:2338-8161 E-ISSN: 2657-0793
- Secretaría Distrital de Movilidad de Bogotá. (2024). Indicadores de movilidad urbana. Alcaldía Mayor de Bogotá. <https://www.movilidadbogota.gov.co/>
- Singh, A. K., & Edla, D. R. (2022). Smart transportation: A futuristic intelligent mobility approach towards smarter cities. Proceedings of the 4th International Conference on Cybernetics, Cognition and Machine Learning Applications (ICCCMLA 2022). DOI [10.1109/ICCCMLA56841.2022.9989297](https://doi.org/10.1109/ICCCMLA56841.2022.9989297)
- Smola, A. J., & Schölkopf, B. (2004). A tutorial on support vector regression. Statistics and Computing, 14(3), 199–222. <https://doi.org/10.1023/B:STCO.0000035301.49549.88>
- Soomro, K., Bhutta, M. N. M., Khan, Z., & Tahir, M. A. (2019). Smart city big data analytics: An advanced review. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, 9(5), e1319. <https://doi.org/10.1002/widm.1319>
- TransMilenio S.A. (2023). Informe de gestión 2023. <https://www.transmilenio.gov.co>
- TransMilenio S.A. (2024). Historia de TransMilenio. <https://www.transmilenio.gov.co/publicaciones/146028/historia-de-transmilenio/>
- TransMilenio S.A. (2025). Informe de gestión primer trimestre 2025. TransMilenio S.A.
- TransMilenio S.A. (2026). GTFS — General Transit Feed Specification [Conjunto de datos]. <https://www.transmilenio.gov.co>

- Trencher, G. (2019). Towards the smart city 2.0: Empirical evidence of using smartness as a tool for tackling social challenges. *Technological Forecasting and Social Change*, 142, 117–128. <https://doi.org/10.1016/j.techfore.2018.07.033>
- Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.
- UITP The International Association of Public Transport. (2019). Transforming cities with Bus Rapid Transit systems. https://cms.uitp.org/wp/wp-content/uploads/2020/07/BRT_ENG_Web.pdf
- UITP The International Association of Public Transport. (2025). Artificial intelligence in public transport. UITP.
- VanderPlas, J. (2016). *Python data science handbook*. O'Reilly Media.
- Vapnik, V. (1995). *The nature of statistical learning theory*. Springer. <https://doi.org/10.1007/978-1-4757-2440-0>
- Williams, C. K. I., & Seeger, M. (2001). Using the Nyström method to speed up kernel machines. *Advances in Neural Information Processing Systems*, 13, 682–688.
- Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>
- Xie, P., Li, T., Liu, J., Du, S., Yang, X., & Zhang, J. (2021). Urban flow prediction from spatiotemporal data using machine learning: A survey. *Information Fusion*, 59, 1–12. <https://doi.org/10.1016/j.inffus.2020.01.002>
- Zhang, H., Liu, Y., Shi, B., Jia, J., Wang, W., & Zhao, X. (2021). Analysis of spatial-temporal characteristics of operations in public transport networks based on multisource data. *Journal of Advanced Transportation*, 2021, 6937228. <https://doi.org/10.1155/2021/6937228>
- Zhang, Y., Zhang, Y., & Haghani, A. (2021). Predicting bus passenger demand using a gradient boosting approach: A BRT case study in Shanghai. *Transportation Research Part C: Emerging Technologies*, 132, 103358. <https://doi.org/10.1016/j.trc.2021.103358>
- Zhao, Z., Chen, W., Wu, X., Chen, P. C. Y., & Liu, J. (2017). LSTM network: A deep learning approach for short-term traffic forecast. *IET Intelligent Transport Systems*, 11(2), 68–75. <https://doi.org/10.1049/iet-its.2016.0208>
- Zheng, Y. (2015). Trajectory data mining: An overview. *ACM Transactions on Intelligent Systems and Technology*, 6(3), 1–41. <https://doi.org/10.1145/2743025>
- Zheng, Y., Wang, S., & Zhao, J. (2021). Equality of opportunity in travel behavior prediction with deep neural networks and discrete choice models. *Transportation Research Part C: Emerging Technologies*, 132, 103410. <https://doi.org/10.1016/j.trc.2021.103410>
- Zhong, G., Yin, T., Li, L., Zhang, J., Zhang, H., & Ran, B. (2022). Bus travel time prediction based on ensemble learning methods. *IEEE Intelligent Transportation Systems Magazine*.

ANEXOS

Los siguientes anexos presentan los extractos de código Python más relevantes del proceso de construcción del dataset y entrenamiento de los modelos. El código completo está disponible en el repositorio de la investigación.

Anexo A — Estructura de Directorios y Configuración Inicial

El siguiente extracto corresponde a la configuración de rutas y la estructura de directorios implementada para organizar los archivos descargados por fuente y período mensual, garantizando la trazabilidad del proceso de extracción.

```
import os, pathlib
BASE_DIR = pathlib.Path('data/raw')
PARQUET_DIR = pathlib.Path('data/processed/parquet')
SOURCES = {
    'validaciones': BASE_DIR / 'tu_llave' / 'ingresos',
    'salidas':      BASE_DIR / 'tu_llave' / 'salidas',
    'estaciones':   BASE_DIR / 'infraestructura',
    'gtfs':         BASE_DIR / 'gtfs_2026'
}
for path in [PARQUET_DIR, *SOURCES.values()]:
    path.mkdir(parents=True, exist_ok=True)
```

Nota. Ejecutado en Python 3.11 con librería pathlib (estándar). Elaboración propia.

Anexo B — Extracción de Archivos Excel con openpyxl

El siguiente extracto muestra la implementación de la lectura de archivos Excel con los parámetros `data_only=True` y `read_only=False`, necesarios para recuperar los valores calculados por las fórmulas de fecha presentes en los archivos de salidas de pasajeros.

```
import openpyxl, pandas as pd
```

```
def extraer_excel(ruta_archivo):
    wb = openpyxl.load_workbook(
        filename = ruta_archivo,
        data_only = True,      # Recupera valores, no fórmulas
        read_only = False     # Necesario para evaluar fórmulas de
    fecha
    )
    ws = wb.active
    datos = ws.values
    cols = next(datos)
    df = pd.DataFrame(datos, columns=cols)
    wb.close()
    return df
```

Nota. Requiere: openpyxl ≥ 3.1 , pandas ≥ 2.0 . Elaboración propia.

Anexo C — Limpieza mediante Rango Intercuartílico (IQR) por Estación

El siguiente extracto muestra el cálculo del IQR de forma independiente por estación para la detección y eliminación de valores atípicos en la variable ingresos.

```
def limpiar_iqr_por_estacion(df, col='ingresos'):
    registros_limpios = []
    for nodo in df['cod_nodo'].unique():
        subset = df[df['cod_nodo'] == nodo].copy()
        Q1 = subset[col].quantile(0.25)
        Q3 = subset[col].quantile(0.75)
        IQR = Q3 - Q1
        lim_inf = Q1 - 1.5 * IQR
        lim_sup = Q3 + 1.5 * IQR
        clean = subset[
            (subset[col] >= lim_inf) &
            (subset[col] <= lim_sup)
        ]
        registros_limpios.append(clean)
    return pd.concat(registros_limpios, ignore_index=True)
```

Nota. Requiere: pandas ≥ 2.0 . Tukey (1977). Elaboración propia.

Anexo D — Protocolo de Entrenamiento y Evaluación (Random Forest)

El siguiente extracto muestra la implementación del protocolo experimental común aplicado al Random Forest, incluyendo la búsqueda de hiperparámetros y la evaluación sobre el conjunto de prueba.

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import mean_absolute_error, r2_score

param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 10, 20],
    'max_features': ['log2', 'sqrt'],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}
search = RandomizedSearchCV(
    RandomForestRegressor(random_state=42),
    param_distributions=param_grid,
    n_iter=20, cv=3,
    scoring='neg_mean_absolute_error',
    random_state=42, n_jobs=-1
)
search.fit(X_train, y_train)
best_rf = search.best_estimator_
y_pred = best_rf.predict(X_test)
print(f'MAE: {mean_absolute_error(y_test, y_pred):.4f}')
print(f'R²: {r2_score(y_test, y_pred):.7f}')
```

Nota. Requiere: `scikit-learn` $\geq$ 1.3. Elaboración propia.

Anexo E — Arquitectura y Entrenamiento del Modelo LSTM

El siguiente extracto muestra la definición de la arquitectura LSTM en PyTorch y el bucle de entrenamiento con early stopping.

```
import torch, torch.nn as nn

class LSTMModel(nn.Module):
    def __init__(self, input_size=12, hidden=64, dense=32):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden,
            batch_first=True)
        self.dropout = nn.Dropout(0.2)
        self.fc1 = nn.Linear(hidden, dense)
```



```

        self.relu    = nn.ReLU()
        self.fc2     = nn.Linear(dense, 1)
    def forward(self, x):
        out, _ = self.lstm(x)
        out     = self.dropout(out[:, -1, :])
        return self.fc2(self.relu(self.fc1(out)))

model      = LSTMModel().to(device)
optimizer  = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion  = nn.MSELoss()
# Early stopping: paciencia=15 épocas sobre val_loss

```

Nota. Requiere: PyTorch 2.5.1+cu121. Elaboración propia.